OBSERVATION

We (mostly) assumed all the data structures that we have discussed so far are single-threaded.

A modern DBMS needs to allow multiple threads to safely access data structures to take advantage of additional CPU cores and hide disk I/O stalls.

OBSERVATION

We (mostly) assumed all the data structures that we have discussed so far are single-threaded.

A modern DBMS needs to allow multiple threads to safely access data structures to take advantage of additional CPU cores and hide disk I/O stalls.

TinyDB Redis KX
pickleDB VOLTDB H-Store

They Don't Do This!

CONCURRENCY CONTROL

A **concurrency control** protocol is the method that the DBMS uses to ensure "correct" results for concurrent operations on a shared object.

A protocol's correctness criteria can vary:

→ **Logical Correctness:** Can a thread see the data that it is supposed to see?

→ **Physical Correctness:** Is the internal representation of the object sound?

LOCKS VS. LATCHES

Locks (Transactions)

→ Protect the database's logical contents from other transactions.
→ Held for transaction's duration.
→ Need to be able to rollback changes.

Latches (Workers)

→ Protect the critical sections of the DBMS's internal data structure from other workers (e.g., threads).
→ Held for operation duration.
→ Do <u>not</u> need to be able to rollback changes.

LOCKS VS. LATCHES

	Locks	*Latches*
Separate…	Transactions	Workers (threads, processes)
Protect…	Database Contents	In-Memory Data Structures
During…	Entire Transactions	Critical Sections
Modes…	Shared, Exclusive, Update, Intention	Read, Write
Deadlock	Detection & Resolution	Avoidance
…by…	Waits-for, Timeout, Aborts	Coding Discipline
Kept in…	Lock Manager	Protected Data Structure

Source: Goetz Graefe

LOCKS VS. LATCHES

Lecture #15

	Locks	*Latches*
Separate…	Transactions	Workers (threads, processes)
Protect…	Database Contents	In-Memory Data Structures
During…	Entire Transactions	Critical Sections
Modes…	Shared, Exclusive, Update, Intention	Read, Write
Deadlock	Detection & Resolution	Avoidance
…by…	Waits-for, Timeout, Aborts	Coding Discipline
Kept in…	Lock Manager	Protected Data Structure

Source: Goetz Graefe

LATCH MODES

Read Mode

→ Multiple threads can read the same object at the same time.
→ A thread can acquire the read latch if another thread has it in read mode.

Write Mode

→ Only one thread can access the object.
→ A thread cannot acquire a write latch if another thread has it in any mode.

Compatibility Matrix

	Read	Write
Read	✓	X
Write	X	X

LATCH IMPLEMENTATION GOALS

Small memory footprint.

Fast execution path when no contention.

Decentralized management of latches.

Avoid expensive system calls.

Source: Filip Pizlo

LATCH IMPLEMENTATIONS

Test-and-Set Spinlock

Blocking OS Mutex

Reader-Writer Locks

Advanced approaches:

→ Adaptive Spinlock (Apple ParkingLot)
→ Queue-based Spinlock (MCS Locks)
→ Optimistic Lock Coupling (The Germans)

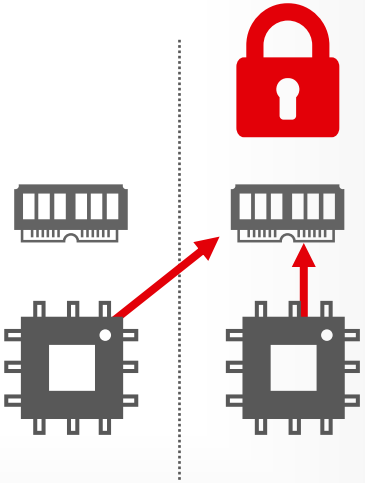
LATCH IMPLEMENTATIONS

Approach #1: Test-and-Set Spin Latch (TAS)

→ Very efficient (single instruction to latch/unlatch)
→ Non-scalable, not cache friendly, not OS friendly.
→ Example: **`std::atomic<T>`**

std::atomic<bool>

```
std::atomic_flag latch;
 ⋮
while (latch.test_and_set(…)) {
   // Retry? Yield? Abort?
}
```



COMPARE-AND-SWAP

Atomic instruction that compares contents of a memory location **M** to a given value **V**

→ If values are equal, installs new given value **V'** in **M**

→ Otherwise, operation fails

See C++11 Atomics

M

20

Address

New Value

```
__sync_bool_compare_and_swap(&M, 20, 30)
```

Compare Value

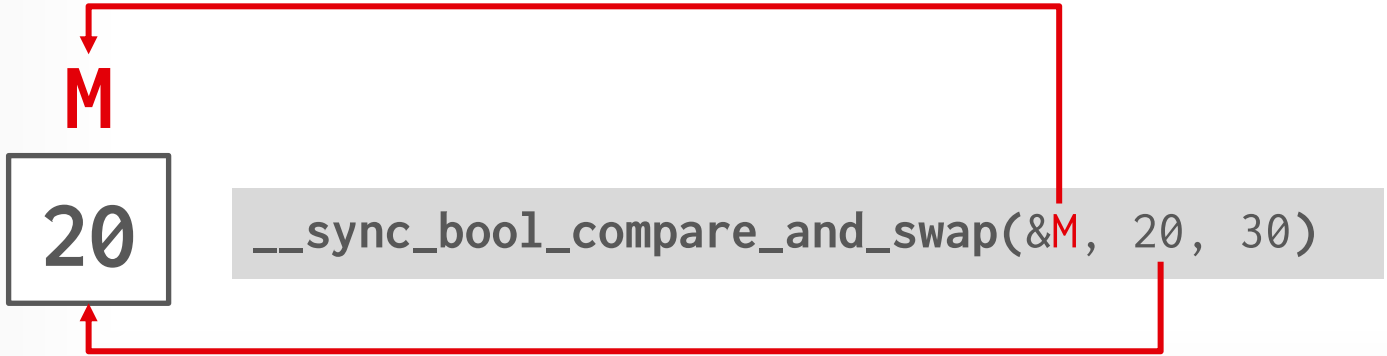
COMPARE-AND-SWAP

Atomic instruction that compares contents of a memory location **M** to a given value **V**

→ If values are equal, installs new given value **V'** in **M**

→ Otherwise, operation fails

See C++11 Atomics



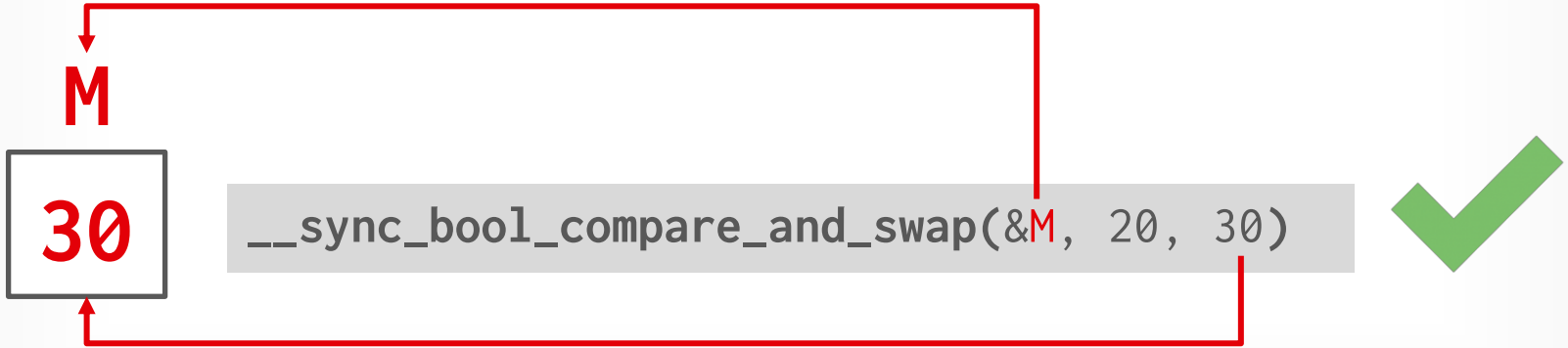
COMPARE-AND-SWAP

Atomic instruction that compares contents of a memory location **M** to a given value **V**

→ If values are equal, installs new given value **V'** in **M**

→ Otherwise, operation fails

See C++11 Atomics

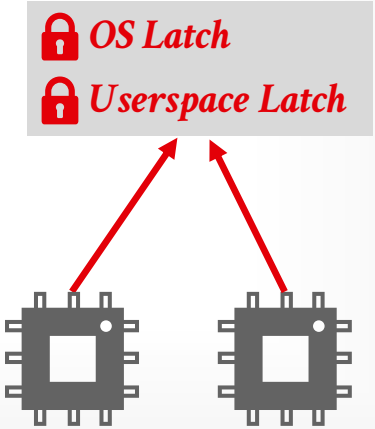


LATCH IMPLEMENTATIONS

Approach #2: Blocking OS Mutex

→ Simple to use
→ Non-scalable (about 25ns per lock/unlock invocation)
→ Example: **std::mutex** → **pthread_mutex_t** → **futex**

```
std::mutex m;
 ⋮
m.lock();
// Do something special...
m.unlock();
```

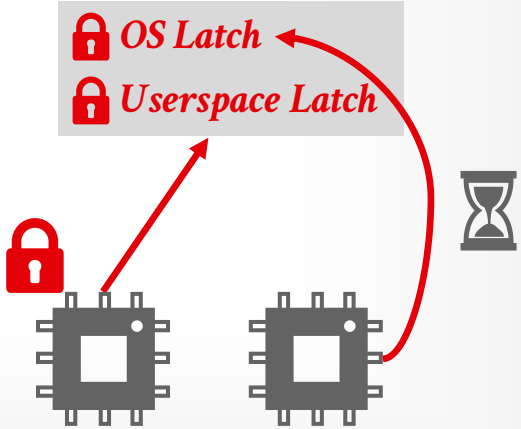


LATCH IMPLEMENTATIONS

Approach #2: Blocking OS Mutex

→ Simple to use
→ Non-scalable (about 25ns per lock/unlock invocation)
→ Example: **std::mutex** → **pthread_mutex_t** → **futex**

```
std::mutex m;
 ⋮
m.lock();
// Do something special...
m.unlock();
```



LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latches

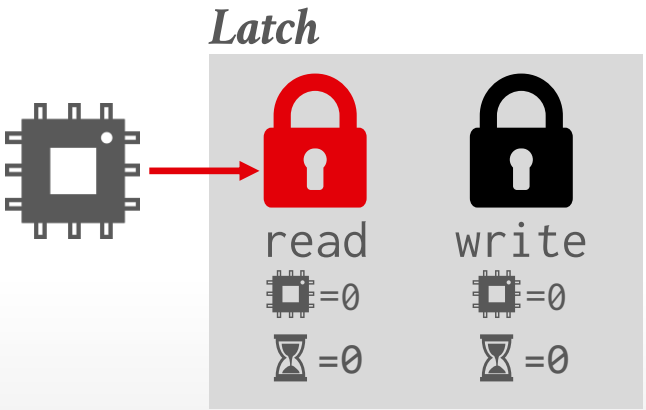
→ Allows for concurrent readers. Must manage read/write queues to avoid starvation.

→ Can be implemented on top of spinlocks.

→ Example: `std::shared_mutex` → `pthread_rwlock_t`

→ `pthread_mutex_t`

→ `pthread_cond_t`



LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latches

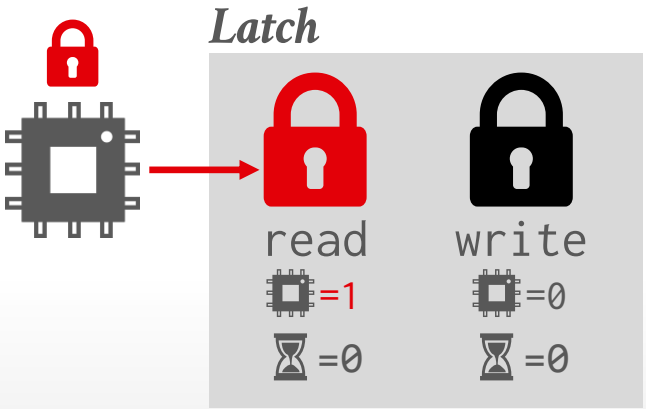
→ Allows for concurrent readers. Must manage read/write queues to avoid starvation.

→ Can be implemented on top of spinlocks.

→ Example: `std::shared_mutex` → `pthread_rwlock_t`

→ `pthread_mutex_t`

→ `pthread_cond_t`



LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latches

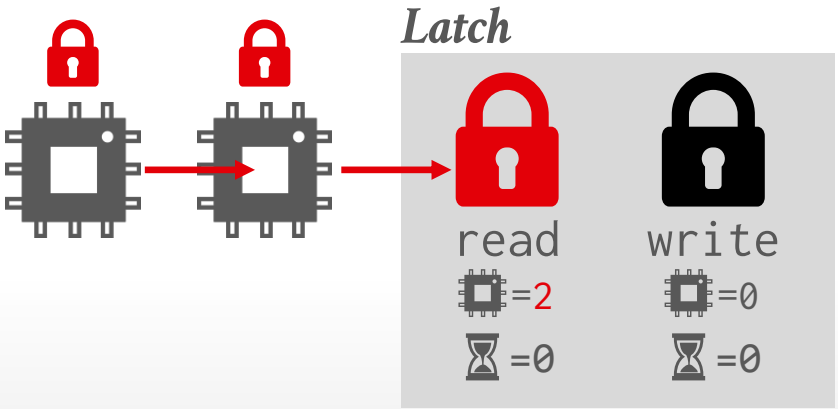
→ Allows for concurrent readers. Must manage read/write queues to avoid starvation.

→ Can be implemented on top of spinlocks.

→ Example: `std::shared_mutex` → `pthread_rwlock_t`

→ `pthread_mutex_t`

→ `pthread_cond_t`



LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latches

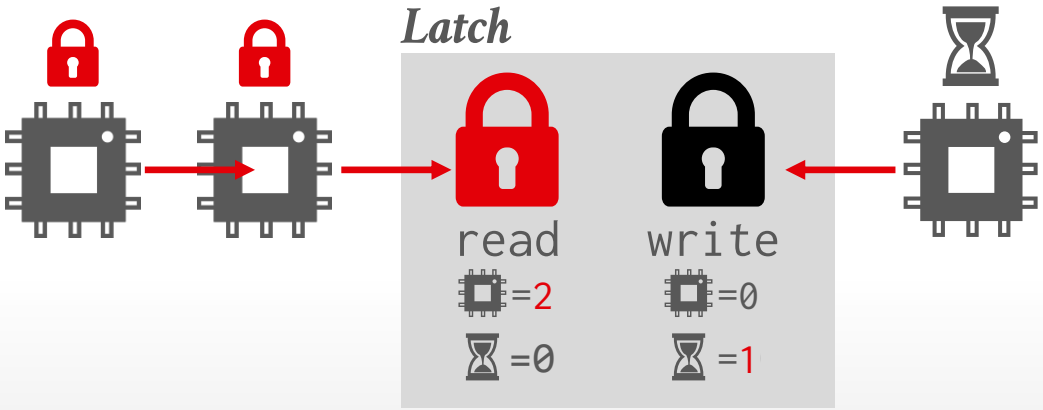
→ Allows for concurrent readers. Must manage read/write queues to avoid starvation.

→ Can be implemented on top of spinlocks.

→ Example: `std::shared_mutex` → `pthread_rwlock_t`

→ `pthread_mutex_t`

→ `pthread_cond_t`



LATCH IMPLEMENTATIONS

Approach #3: Reader-Writer Latches

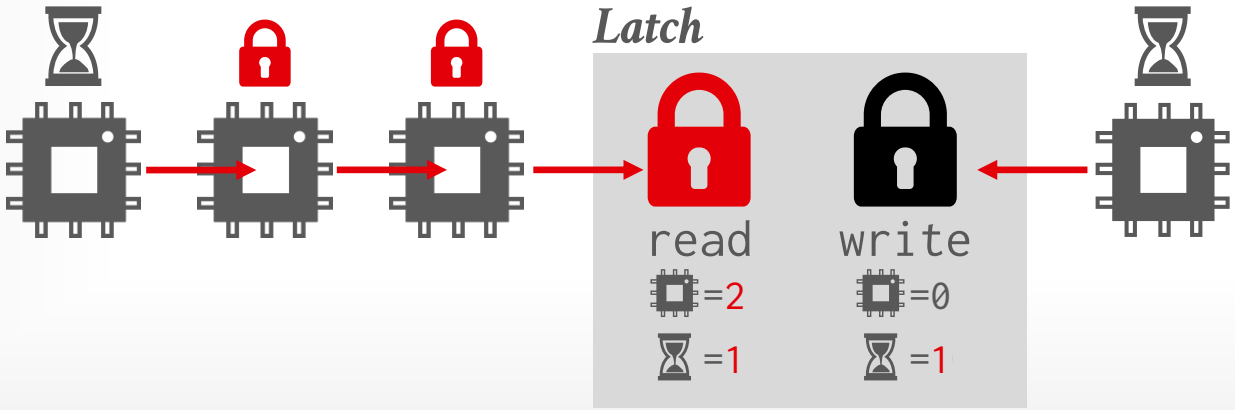
→ Allows for concurrent readers. Must manage read/write queues to avoid starvation.

→ Can be implemented on top of spinlocks.

→ Example: **`std::shared_mutex`** → **`pthread_rwlock_t`**

→ **`pthread_mutex_t`**

→ **`pthread_cond_t`**



HASH TABLE LATCHING

Easy to support concurrent access due to the limited ways threads access the data structure.

→ All threads move in the same direction and only access a single page/slot at a time.

→ Deadlocks are not possible.

To resize the table, take a global write latch on the entire table (e.g., in the header page).

HASH TABLE LATCHING

Approach #1: Global Latch

→ Use a single latch to protect the entire data structure.

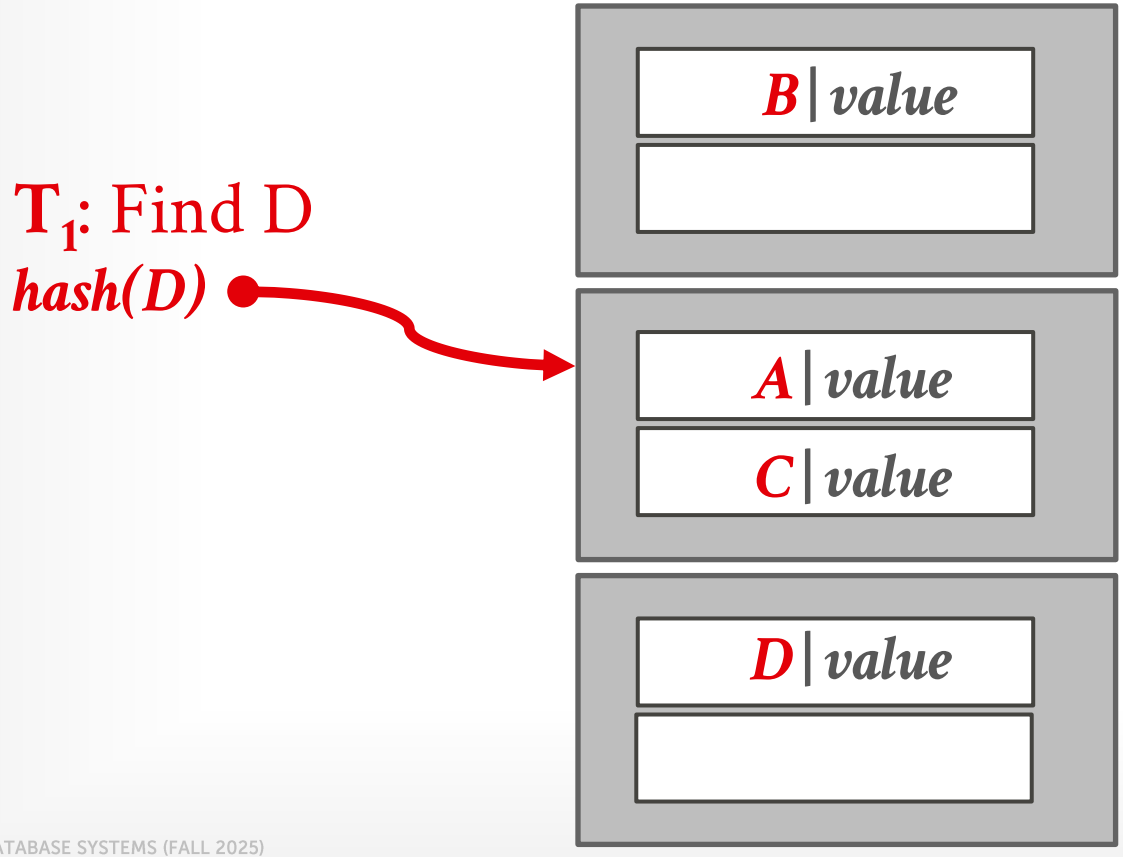
Approach #2: Page/Block Latches

→ Each page/block has an embedded reader-writer latch.
→ Threads acquire latch before accessing a page/block.
→ Release latch before moving to the next page/block.

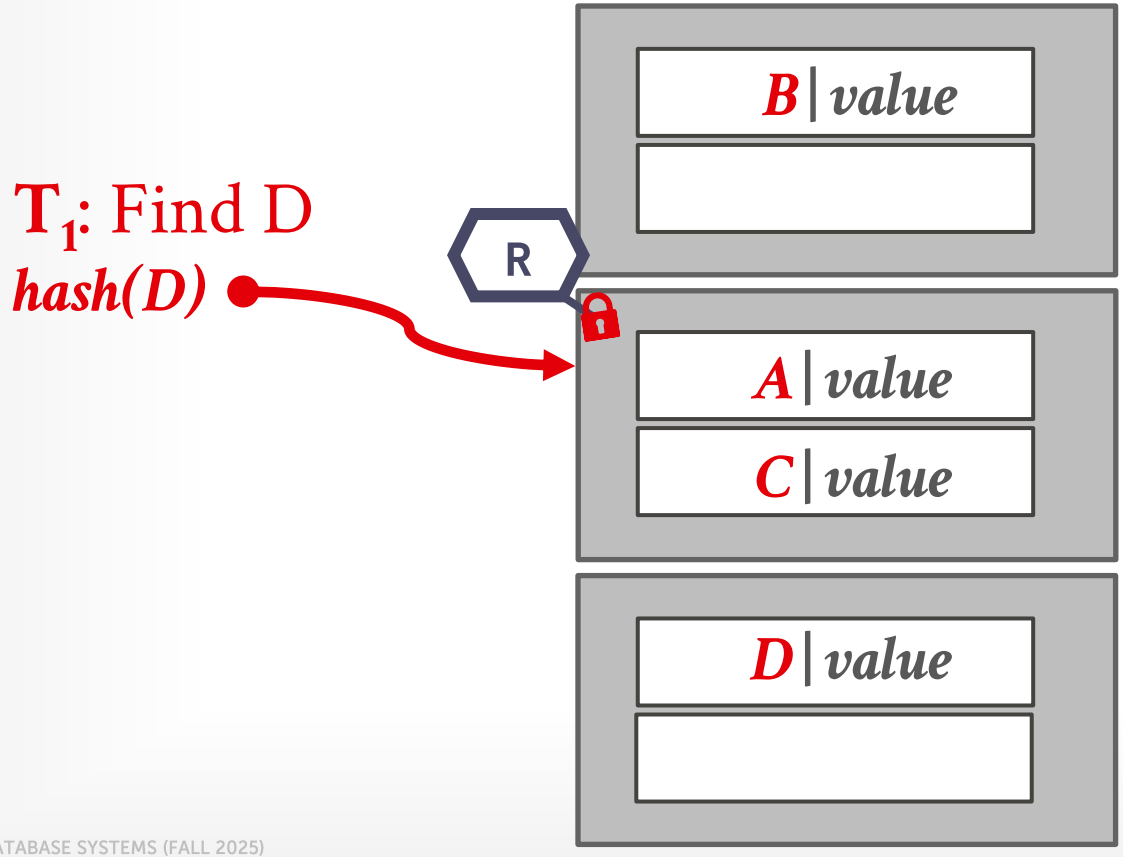
Approach #3: Slot Latches

→ Each slot has its own latch.
→ Can use a single-mode latch to reduce meta-data and computational overhead.

HASH TABLE: PAGE/BLOCK LATCHES



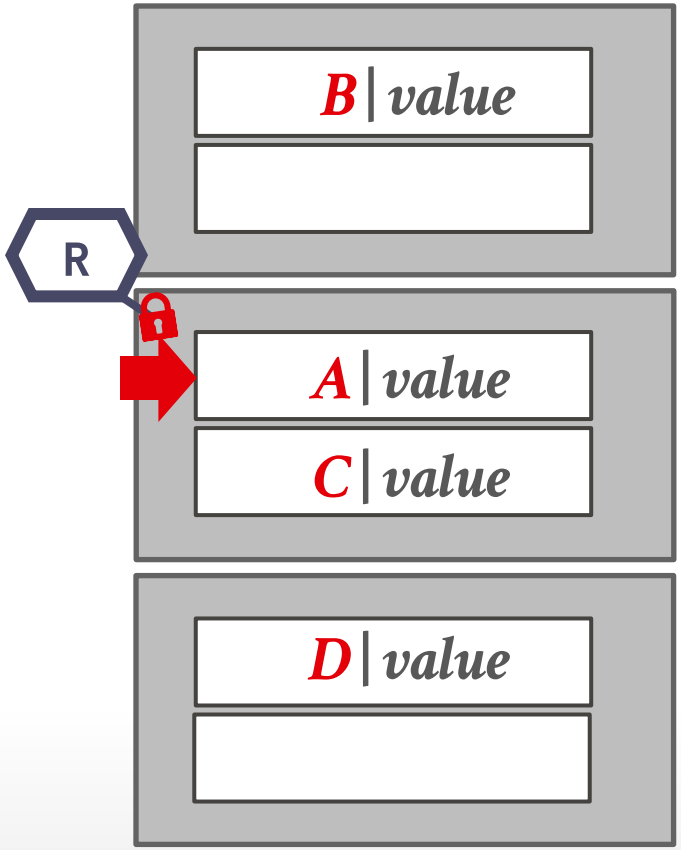
HASH TABLE: PAGE/BLOCK LATCHES



HASH TABLE: PAGE/BLOCK LATCHES

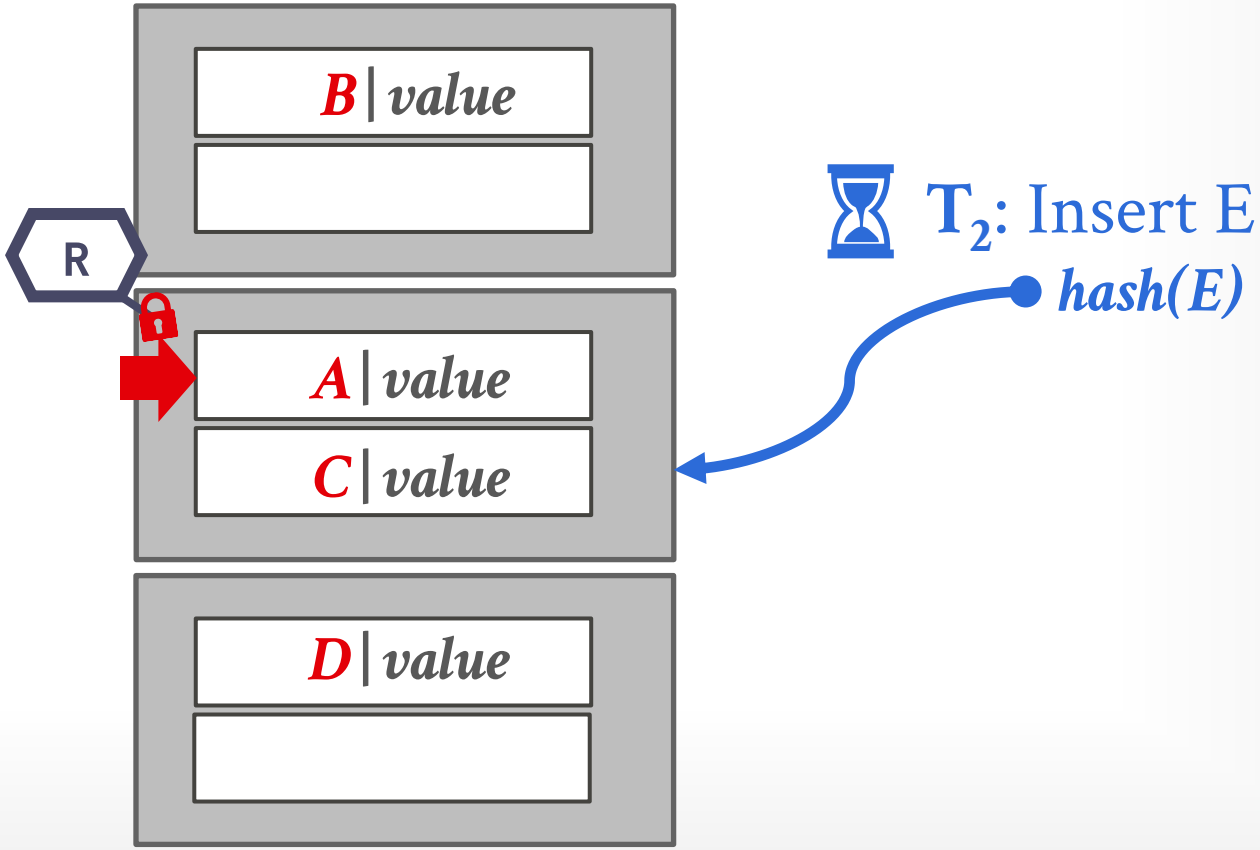
T_1: Find D

hash(D)



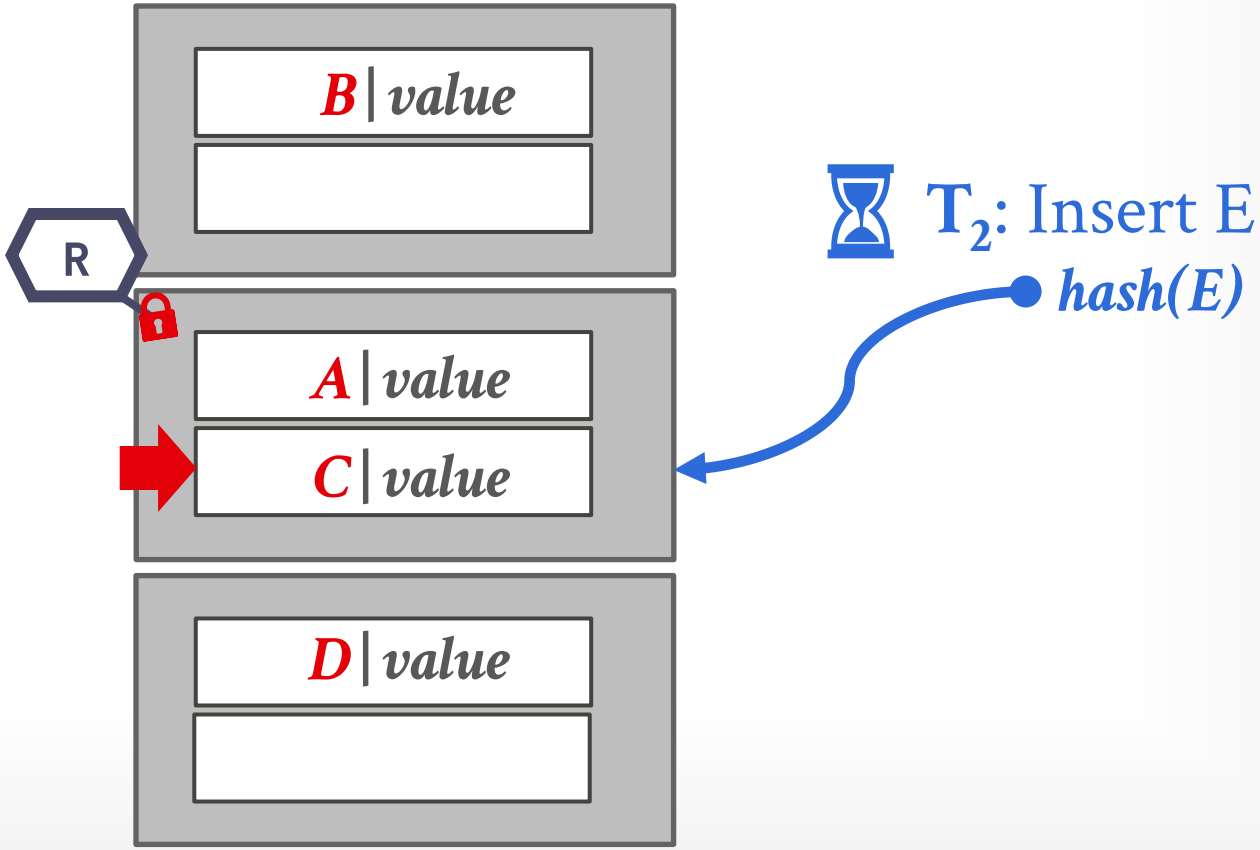
HASH TABLE: PAGE/BLOCK LATCHES

T_1: Find D
hash(D)



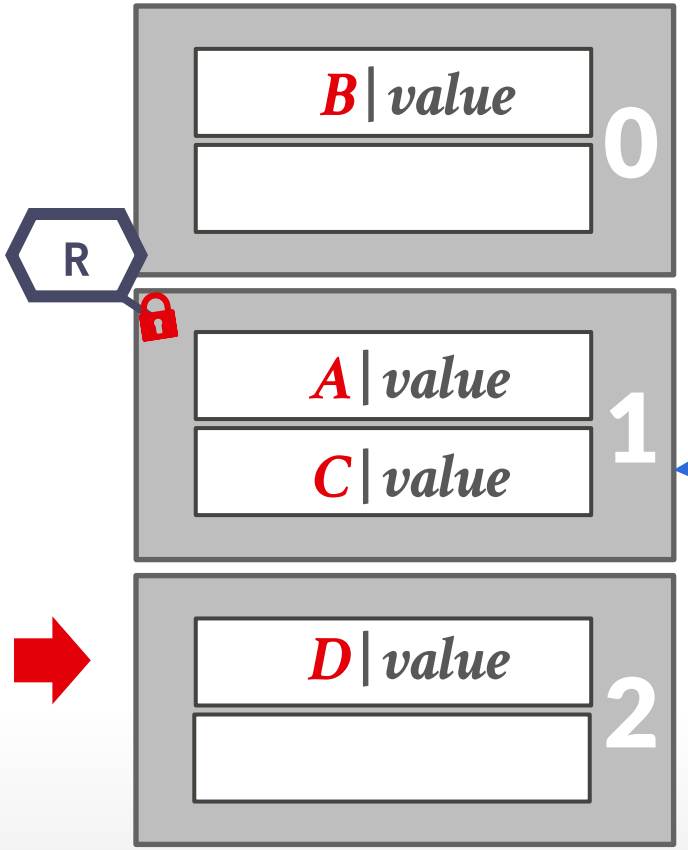
HASH TABLE: PAGE/BLOCK LATCHES

T_1: Find D
hash(D)



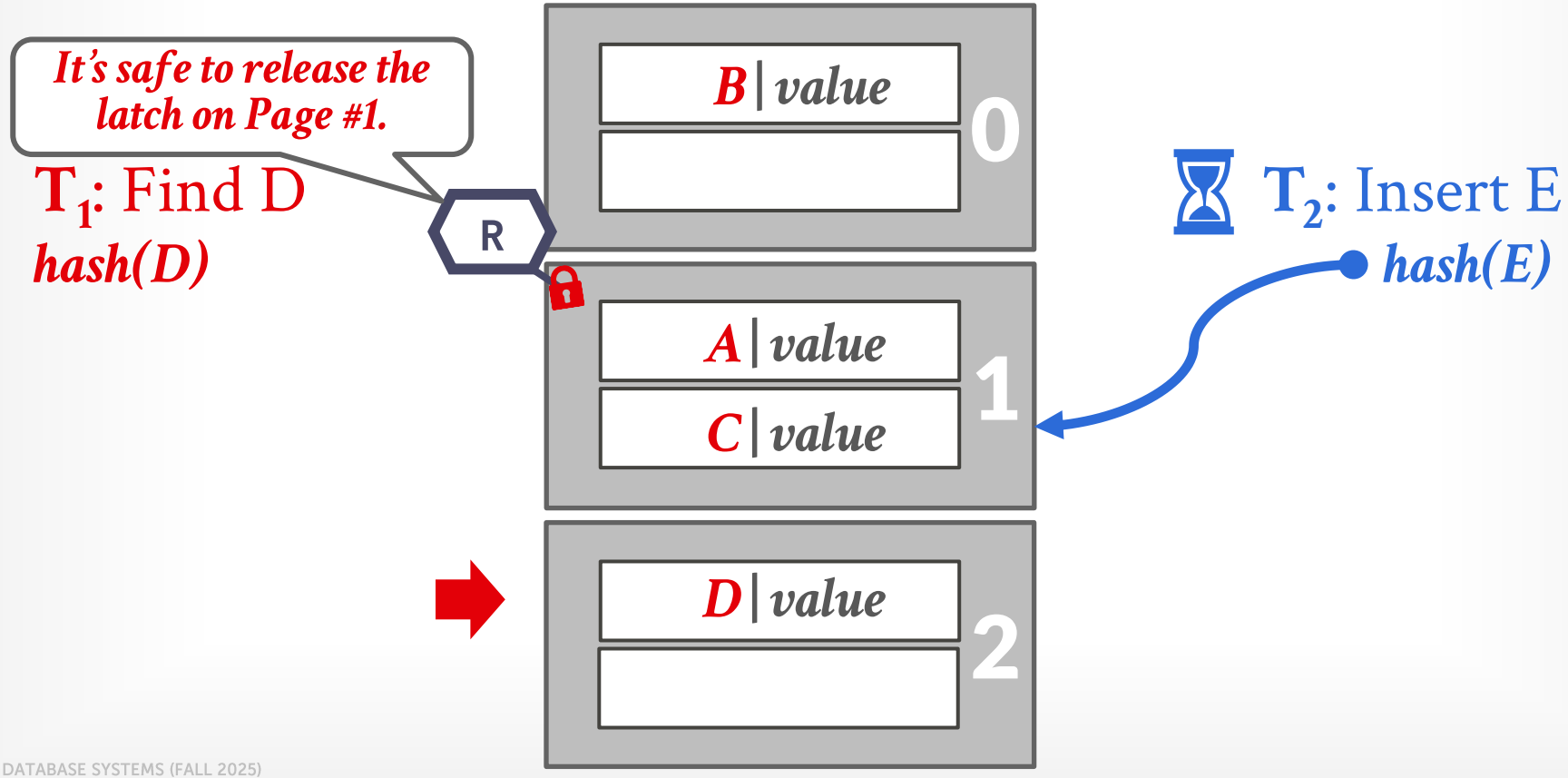
HASH TABLE: PAGE/BLOCK LATCHES

T_1: Find D
hash(D)



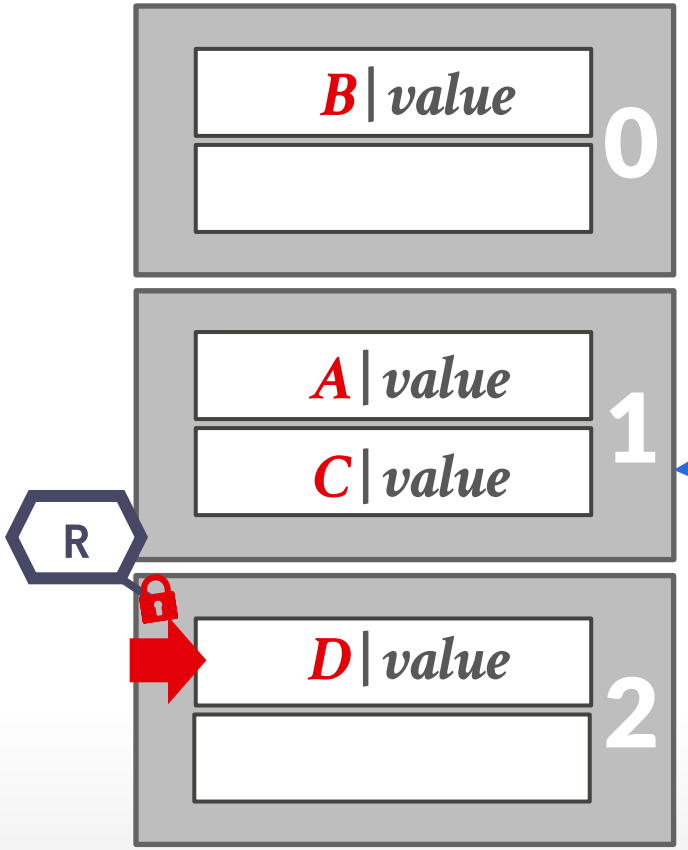
T_2: Insert E
hash(E)

HASH TABLE: PAGE/BLOCK LATCHES



HASH TABLE: PAGE/BLOCK LATCHES

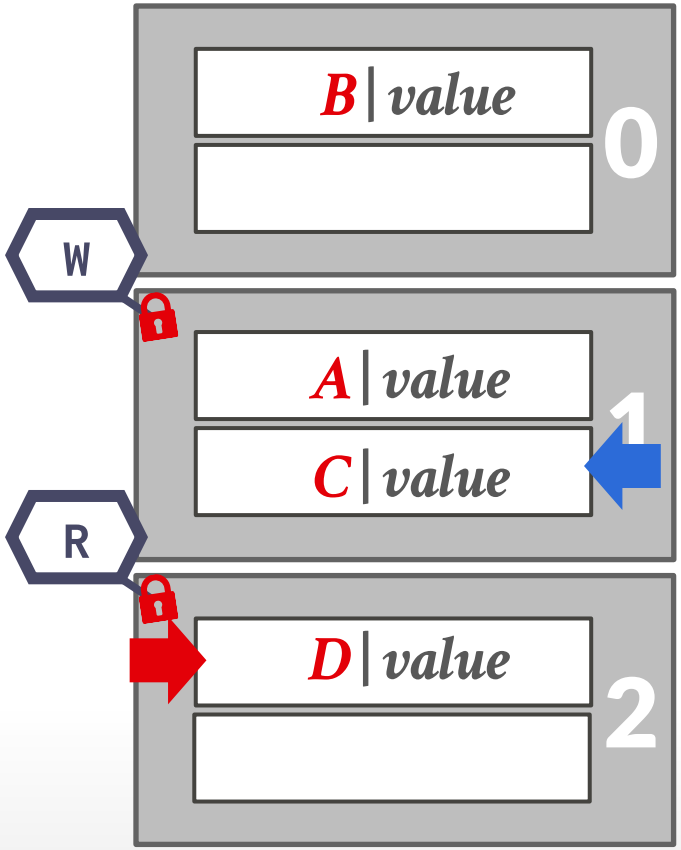
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: PAGE/BLOCK LATCHES

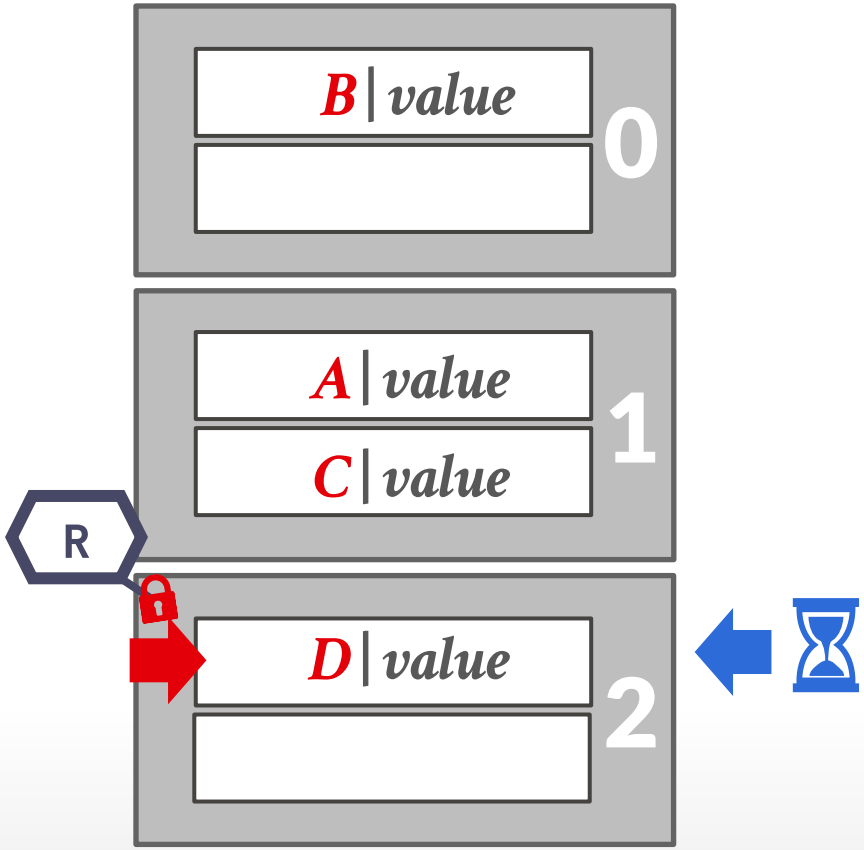
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: PAGE/BLOCK LATCHES

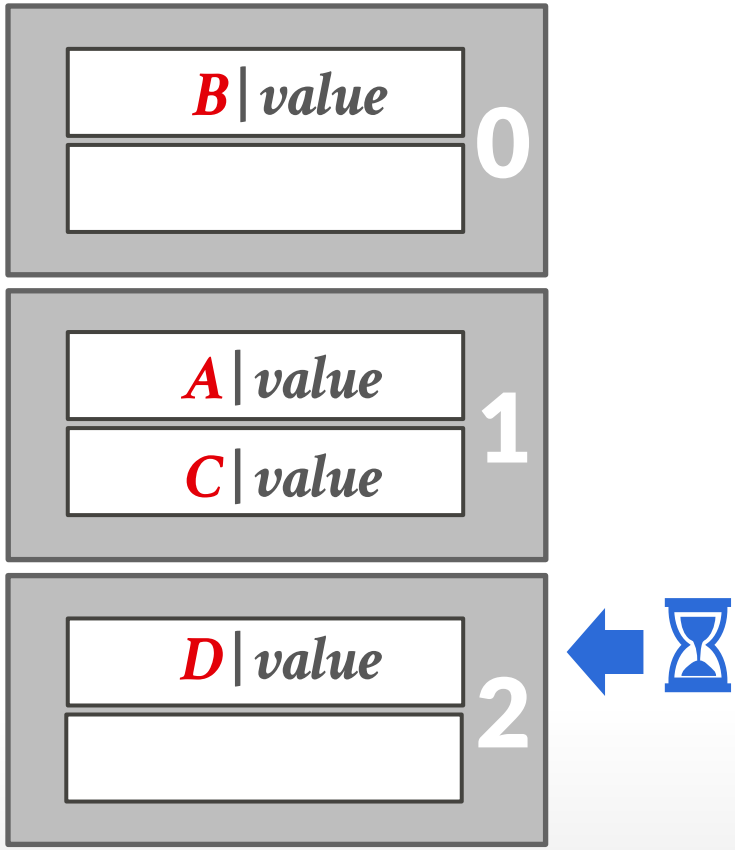
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: PAGE/BLOCK LATCHES

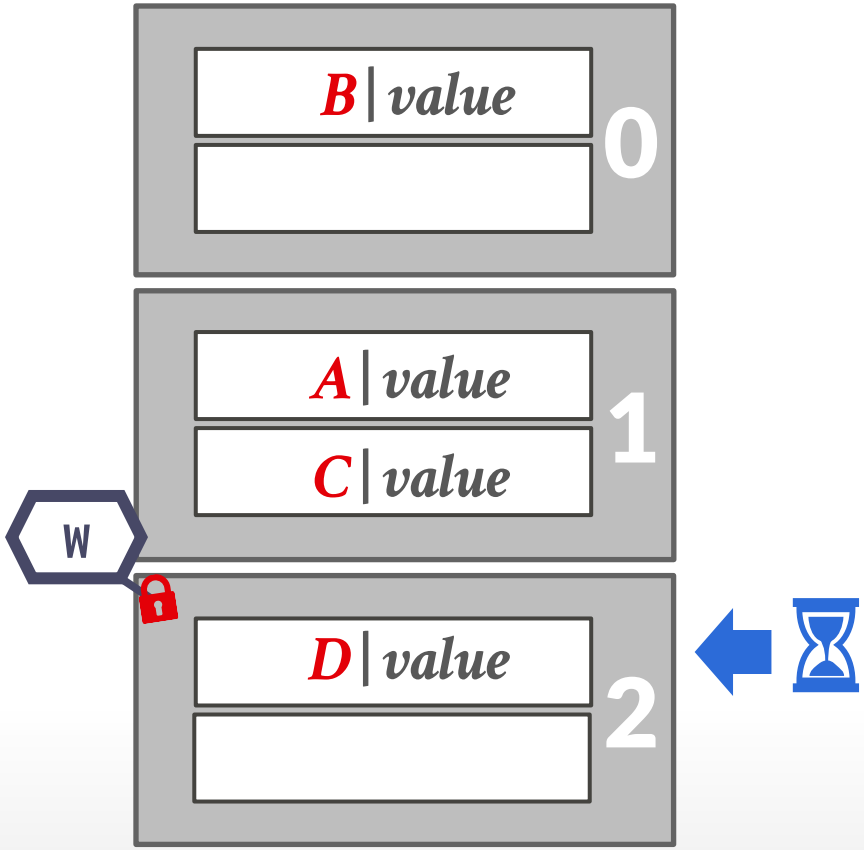
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: PAGE/BLOCK LATCHES

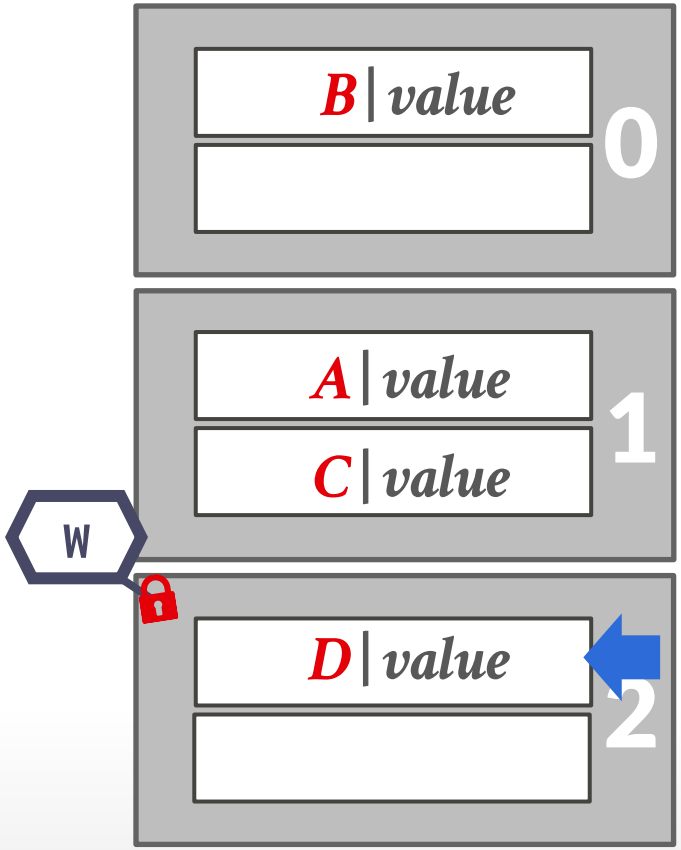
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: PAGE/BLOCK LATCHES

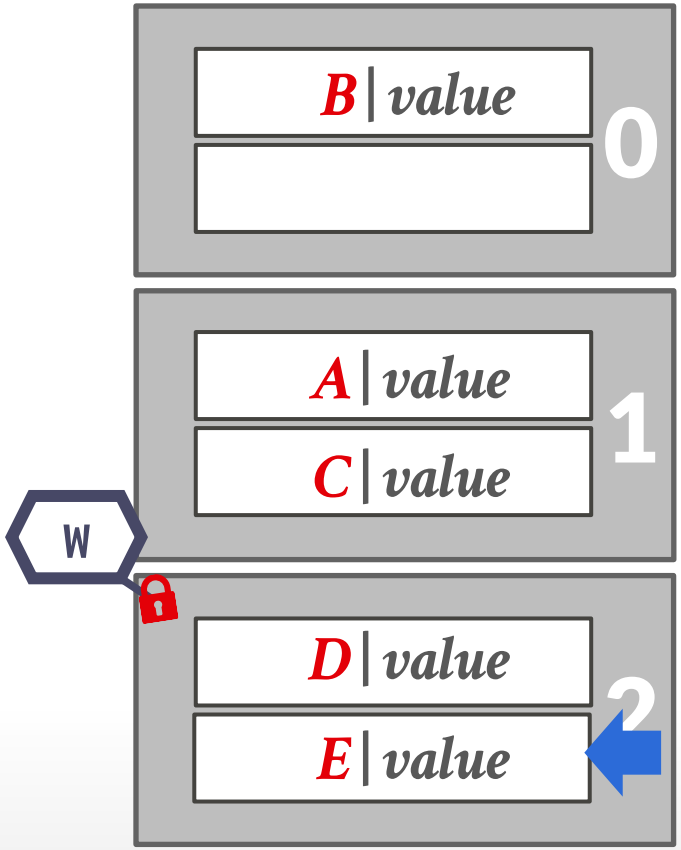
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: PAGE/BLOCK LATCHES

T_1: Find D
hash(D)

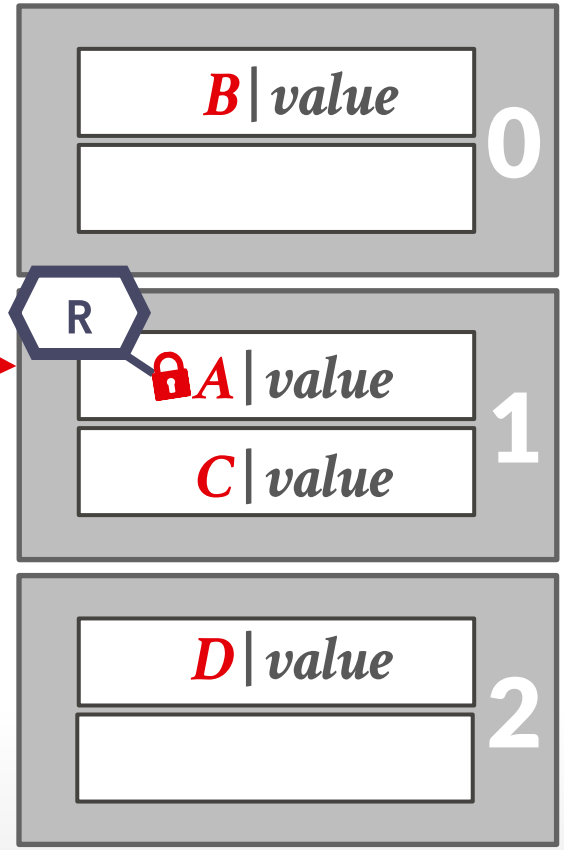


T_2: Insert E
hash(E)

HASH TABLE: SLOT LATCHES

T_1: Find D

hash(D)

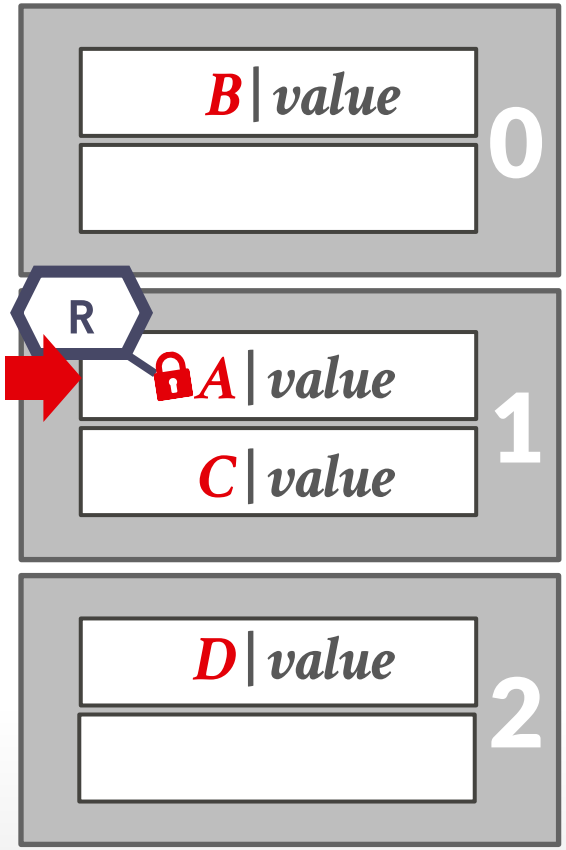


T_2: Insert E

hash(E)

HASH TABLE: SLOT LATCHES

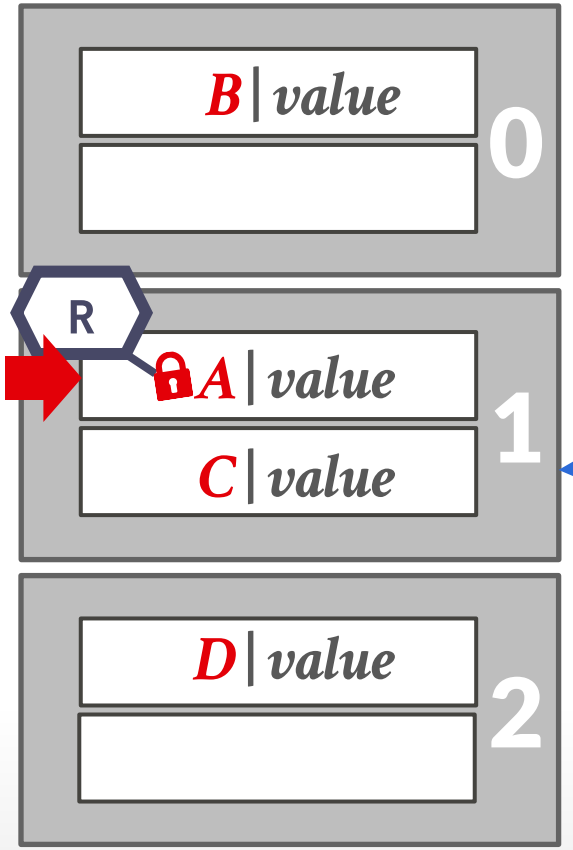
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: SLOT LATCHES

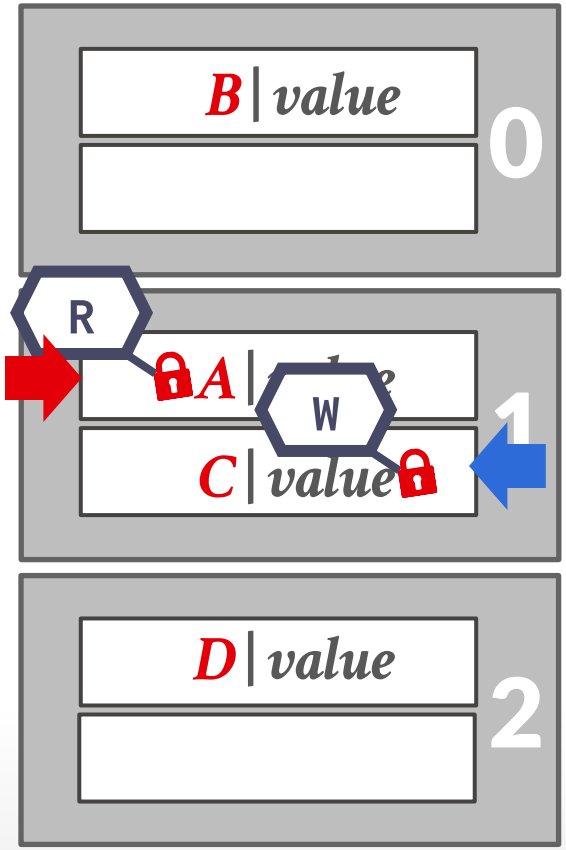
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: SLOT LATCHES

T_1: Find D
hash(D)



T_2: Insert E
hash(E)

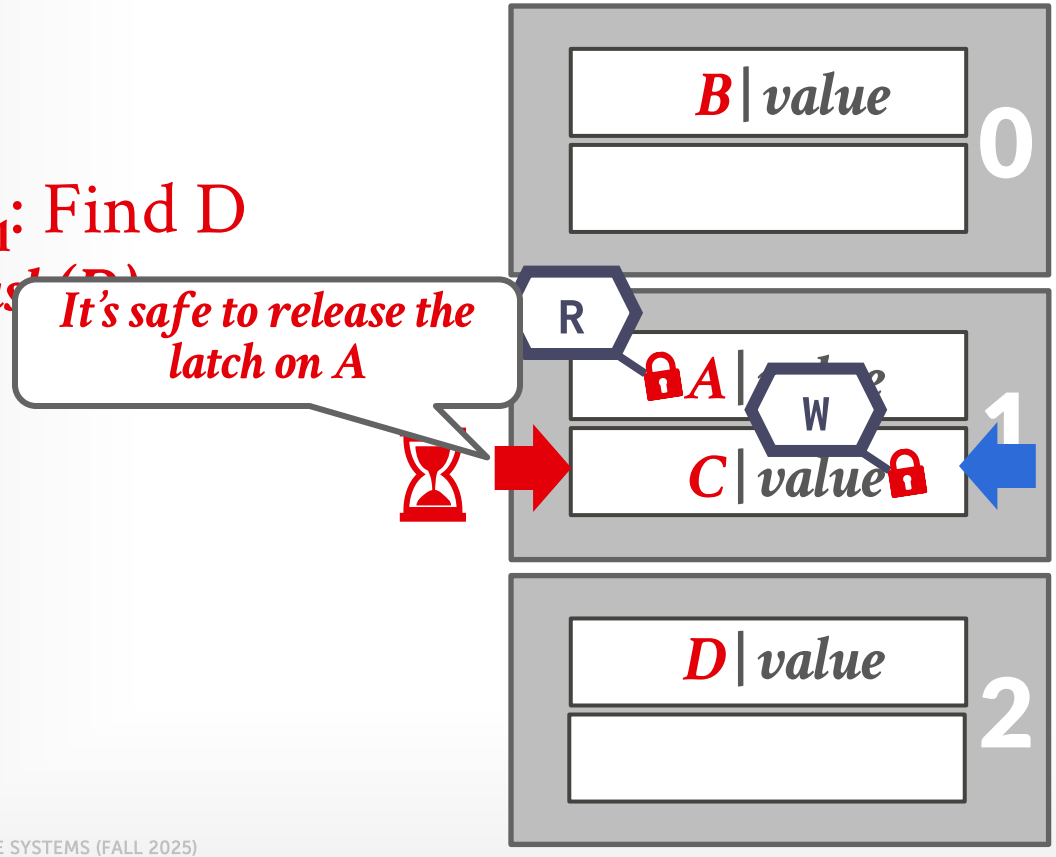
HASH TABLE: SLOT LATCHES

T_1: Find D

hash(D)

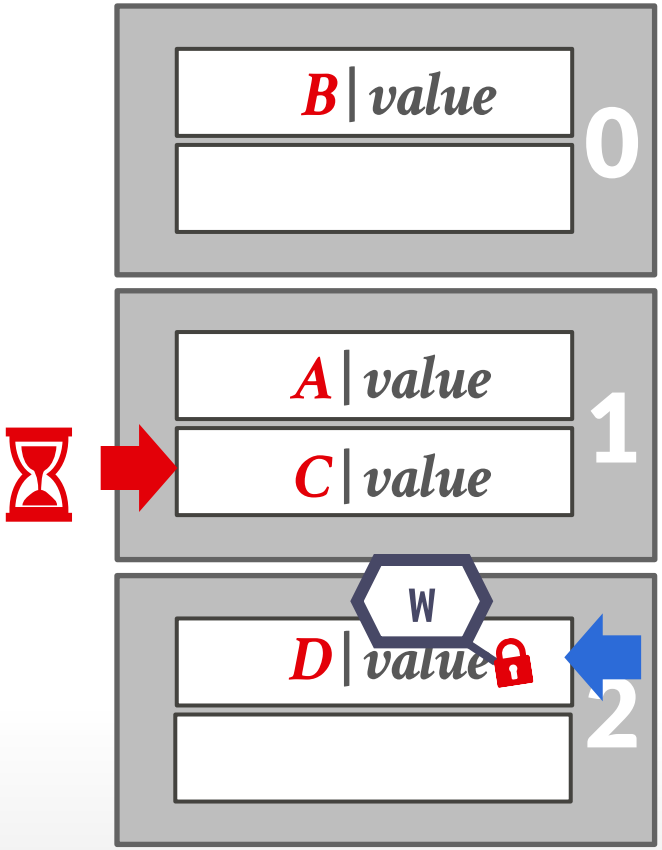
T_2: Insert E

hash(E)



HASH TABLE: SLOT LATCHES

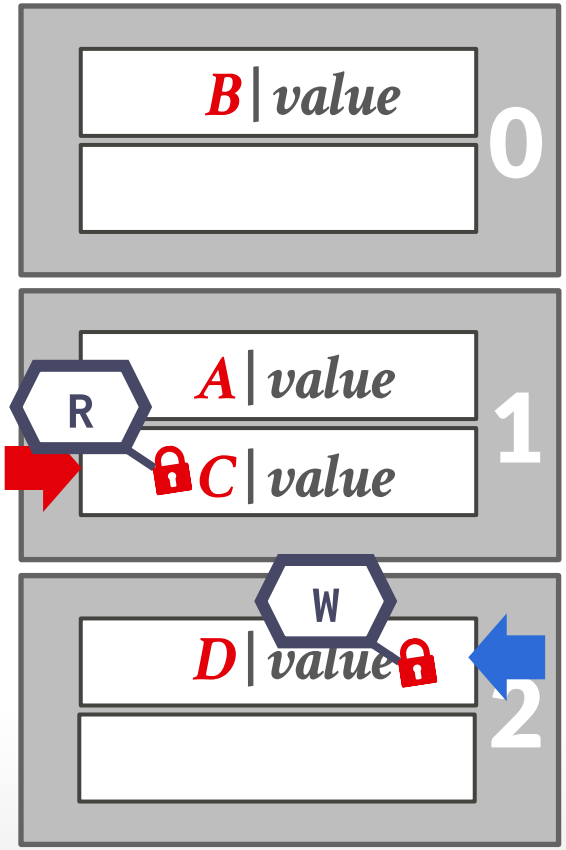
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: SLOT LATCHES

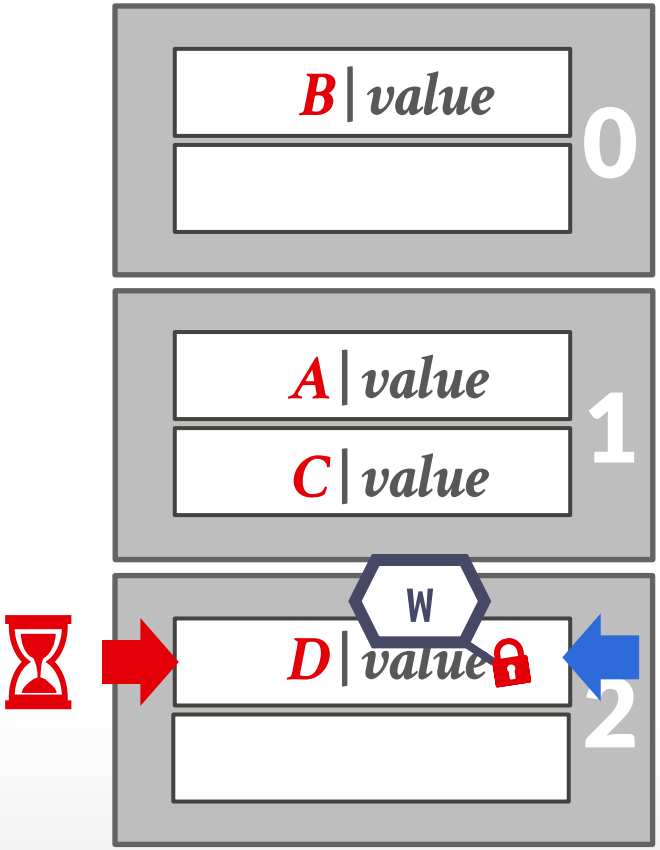
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: SLOT LATCHES

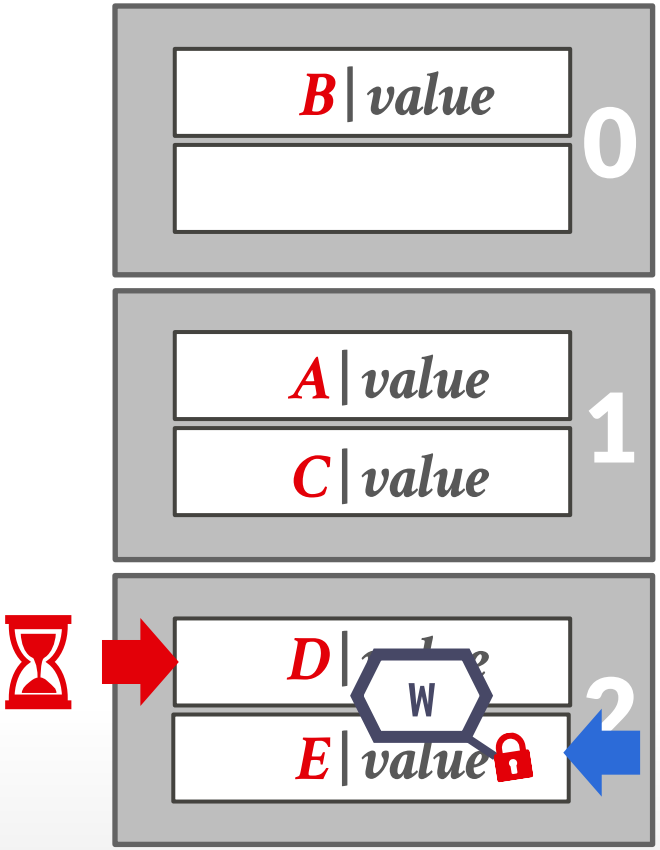
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: SLOT LATCHES

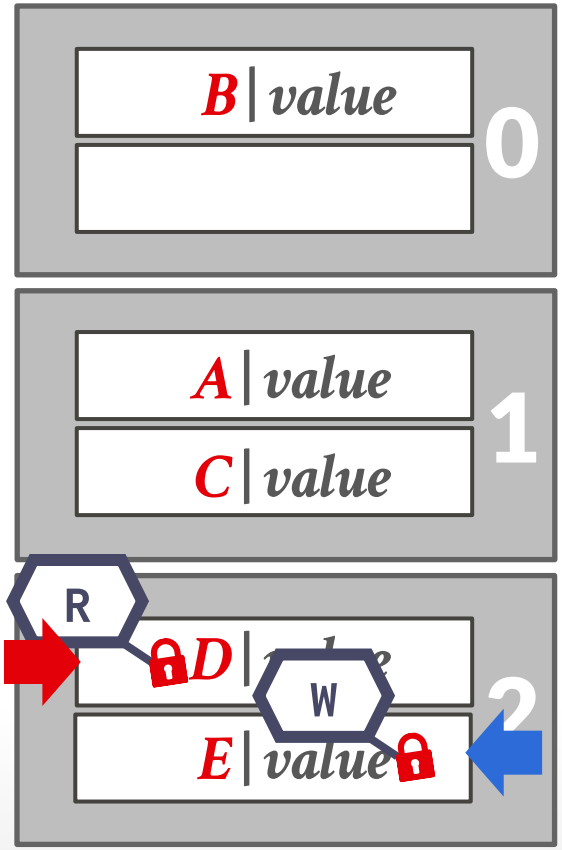
T_1: Find D
hash(D)



T_2: Insert E
hash(E)

HASH TABLE: SLOT LATCHES

T_1: Find D
hash(D)



T_2: Insert E
hash(E)

B+TREE CONCURRENCY CONTROL

We want to allow multiple threads to read and update a B+Tree at the same time.

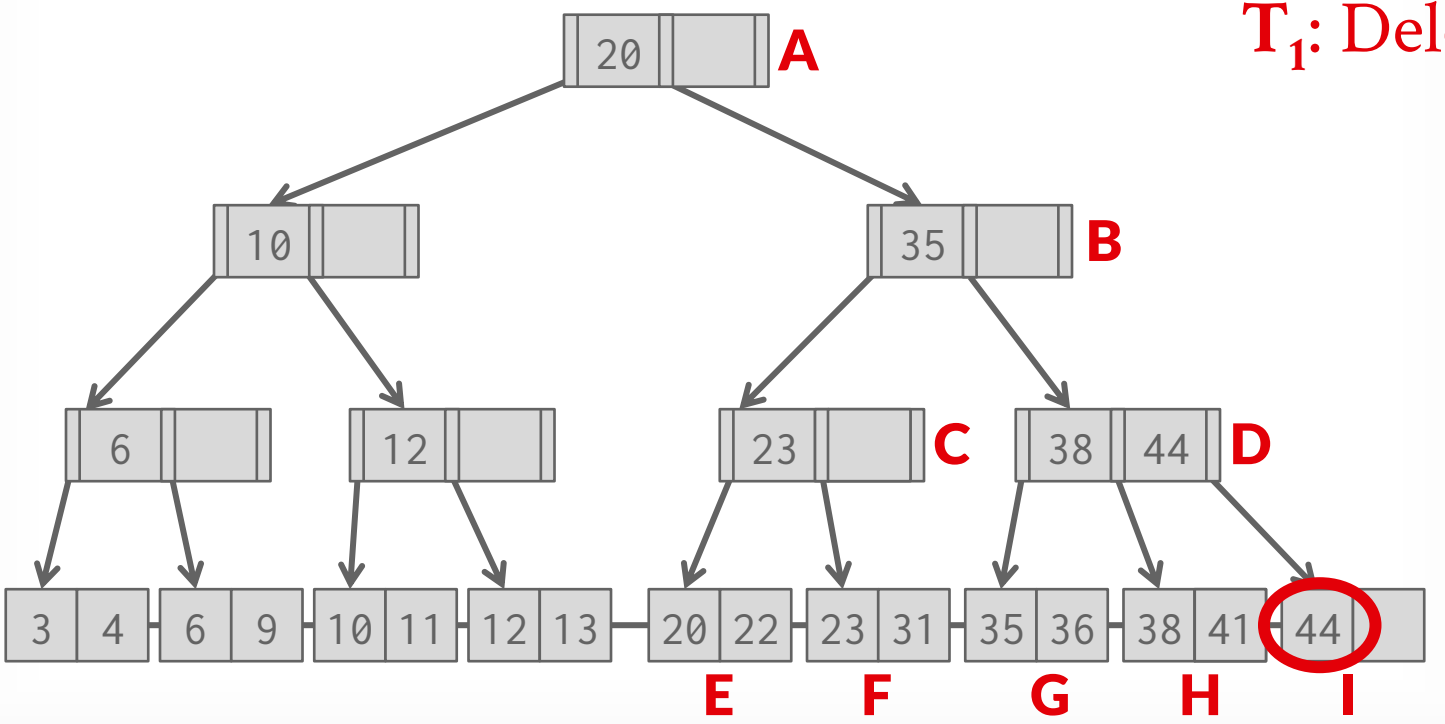
We need to protect against two types of problems:

→ Threads trying to modify the contents of a node at the same time.

→ One thread traversing the tree while another thread splits/merges nodes.

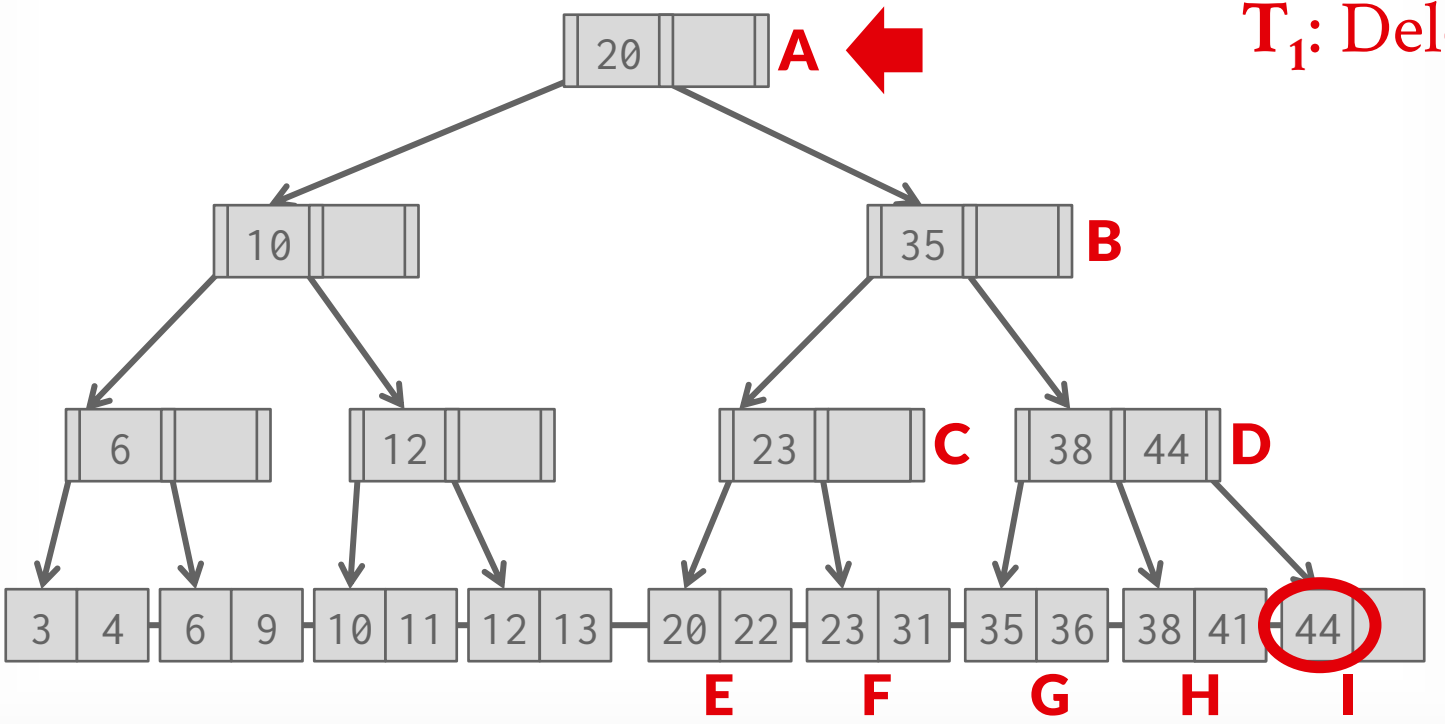
B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44



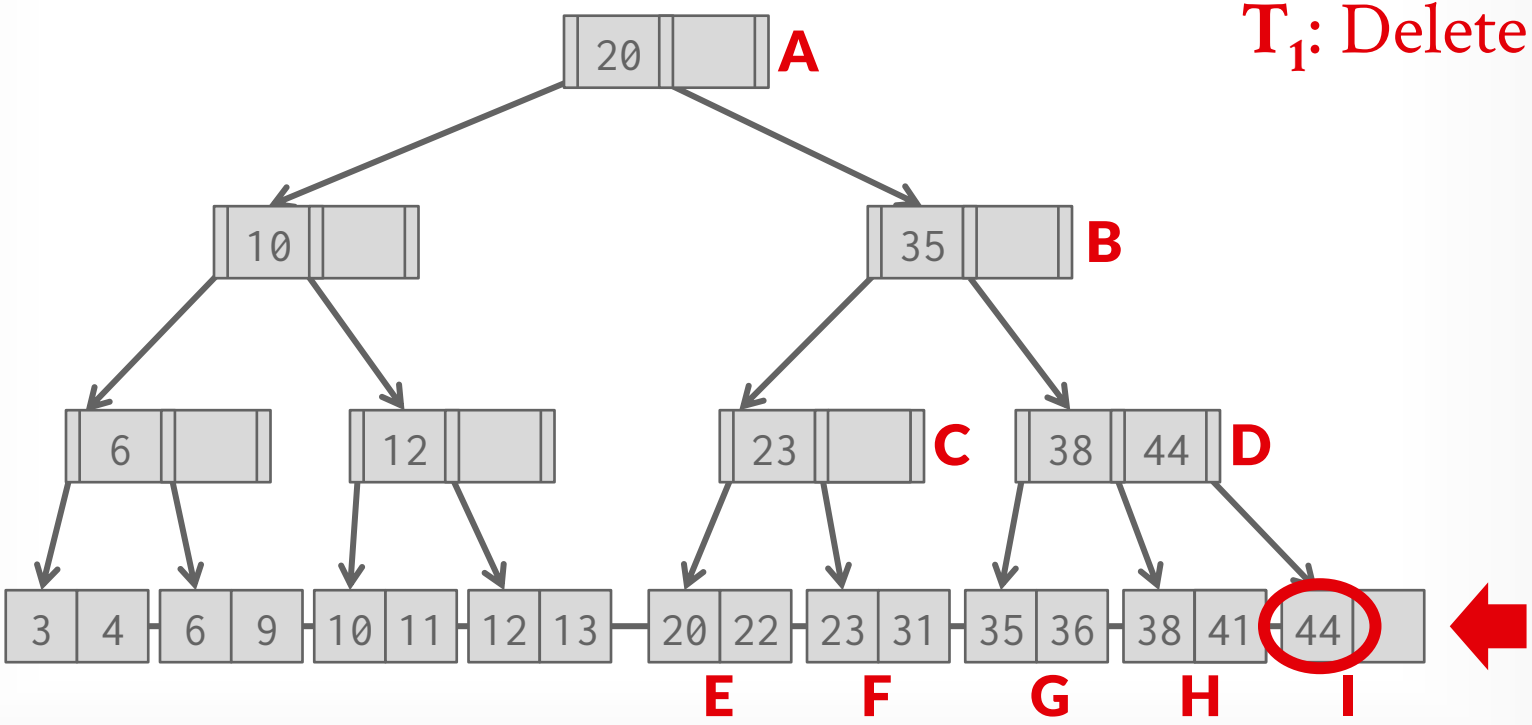
B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44



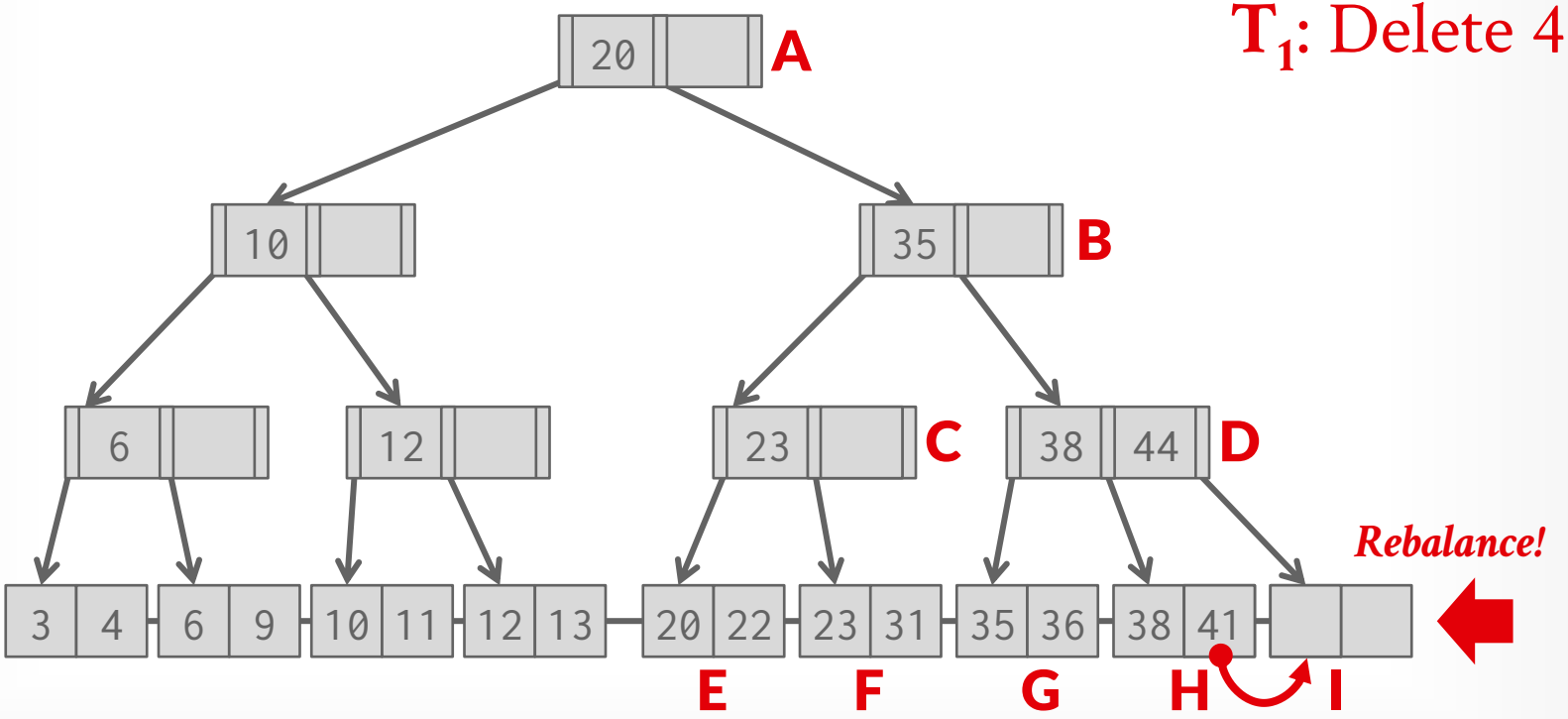
B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44



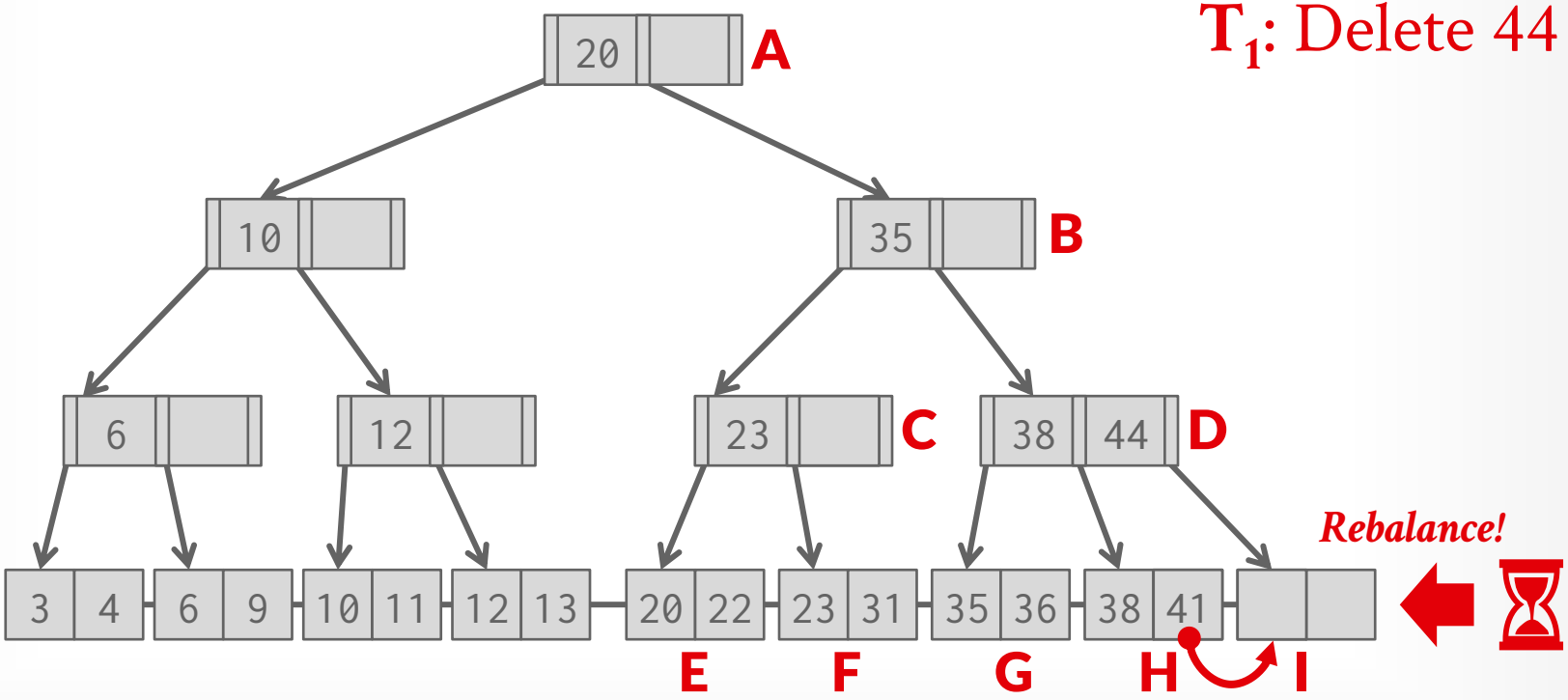
B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44



B+TREE MULTI-THREADED EXAMPLE

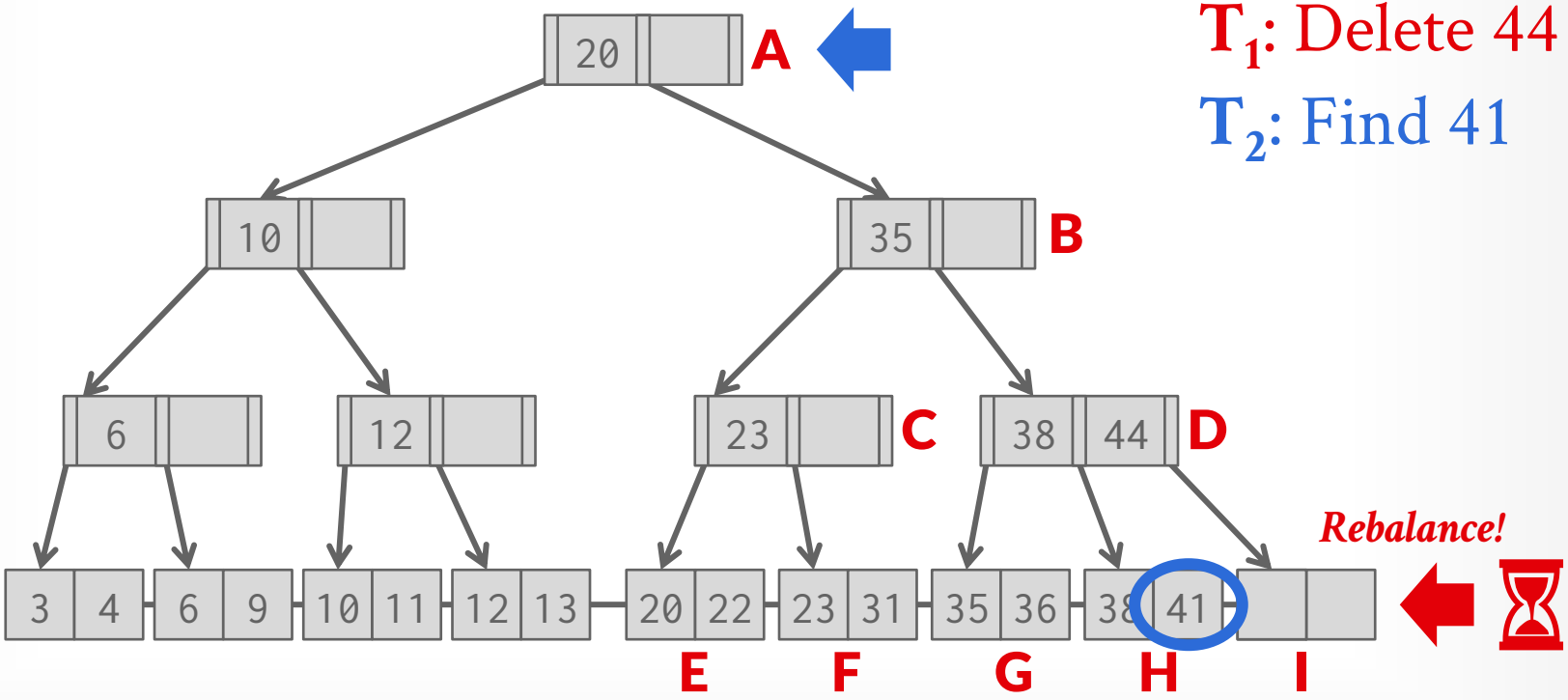
T_1: Delete 44



B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44

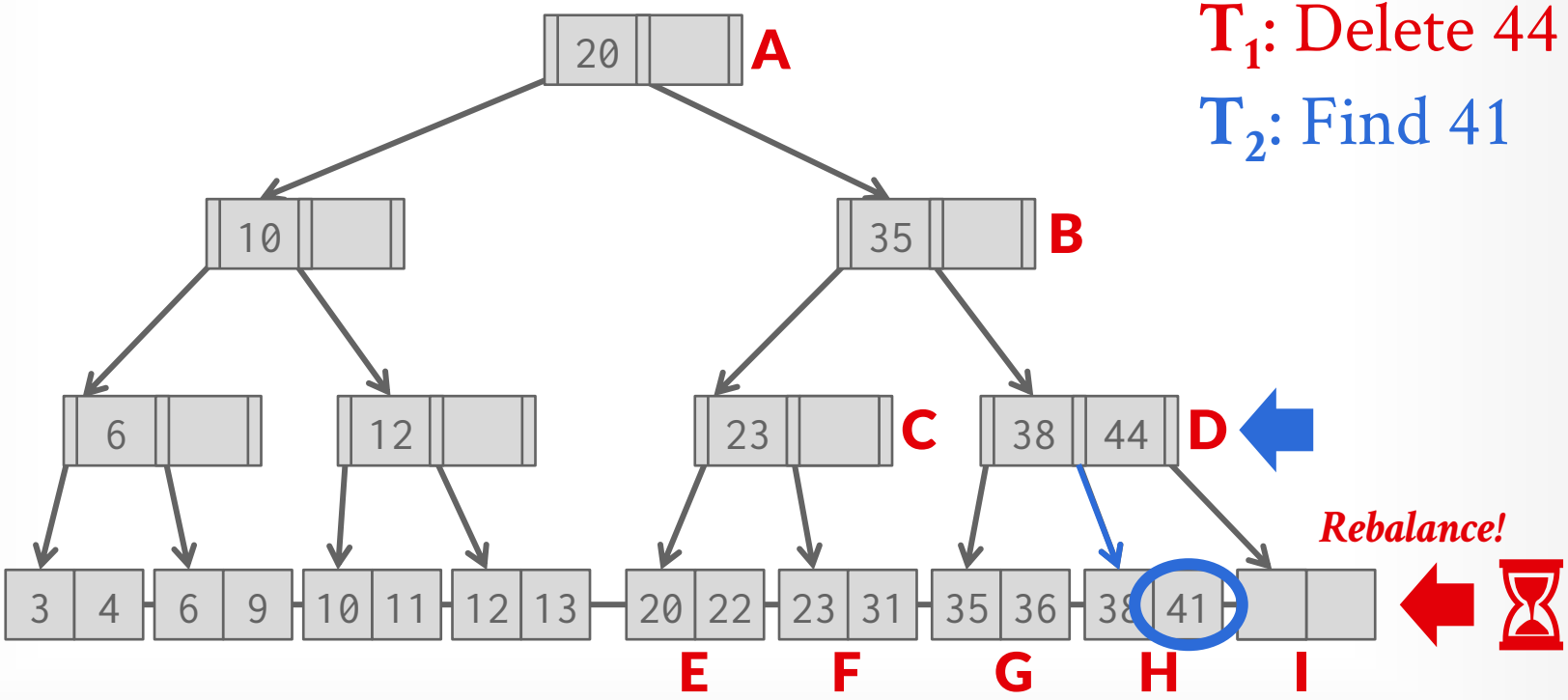
T_2: Find 41



B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44

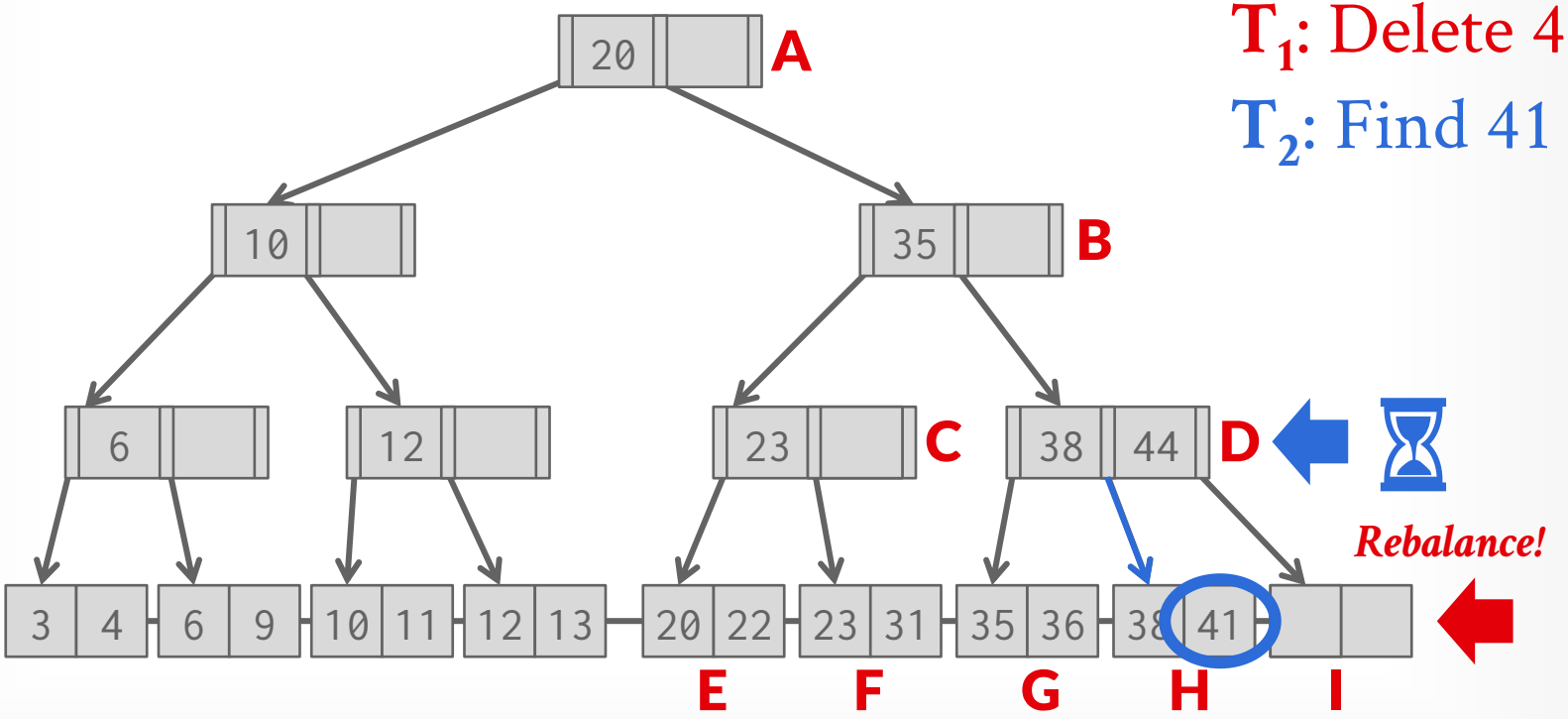
T_2: Find 41



B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44

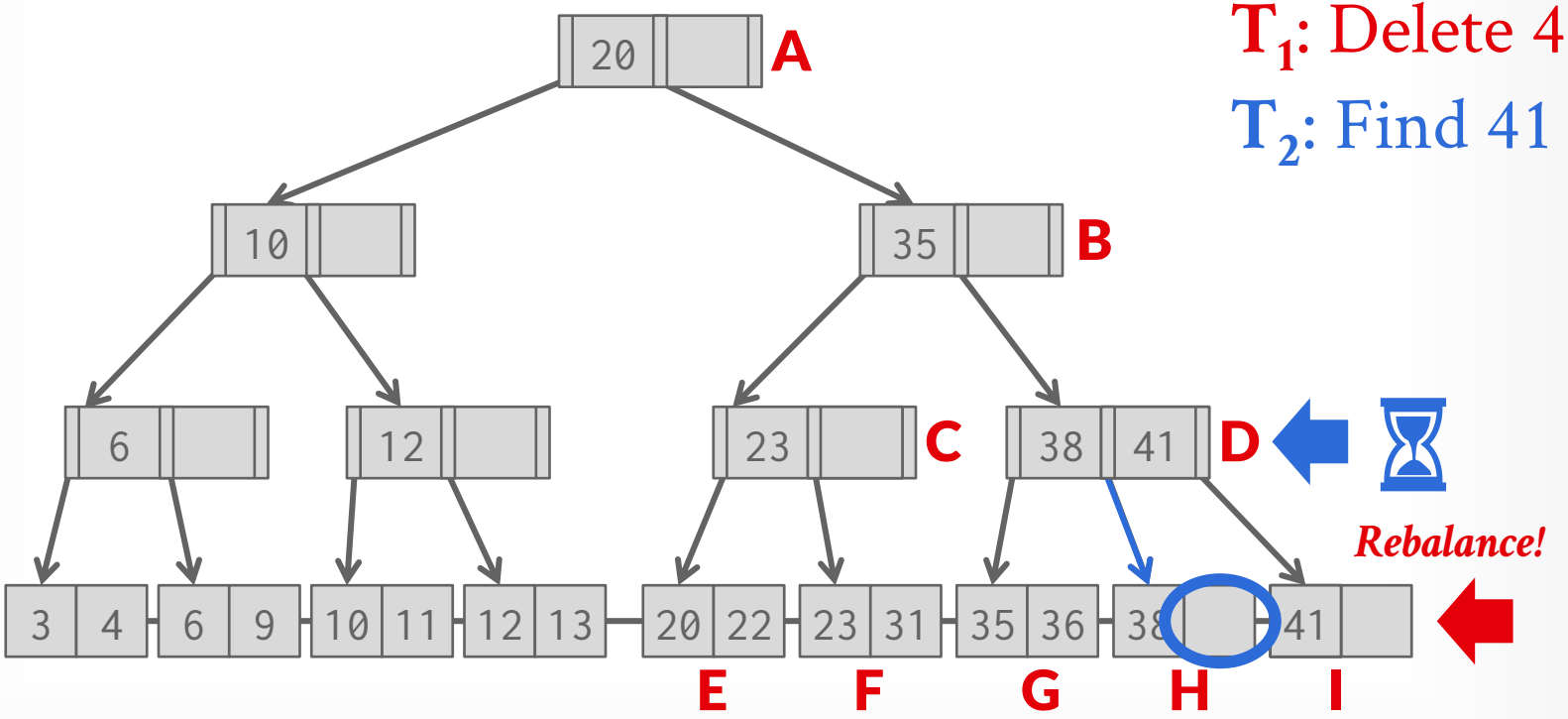
T_2: Find 41



B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44

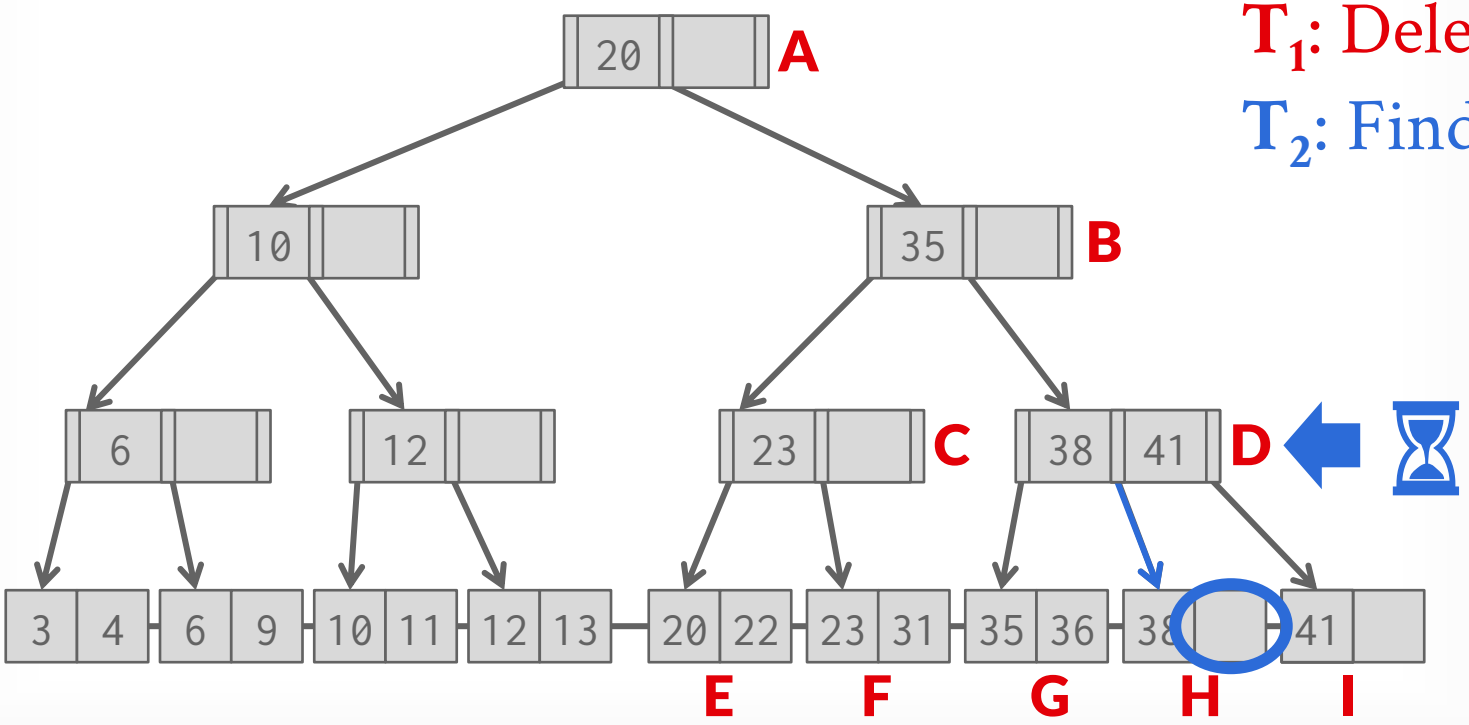
T_2: Find 41



B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44

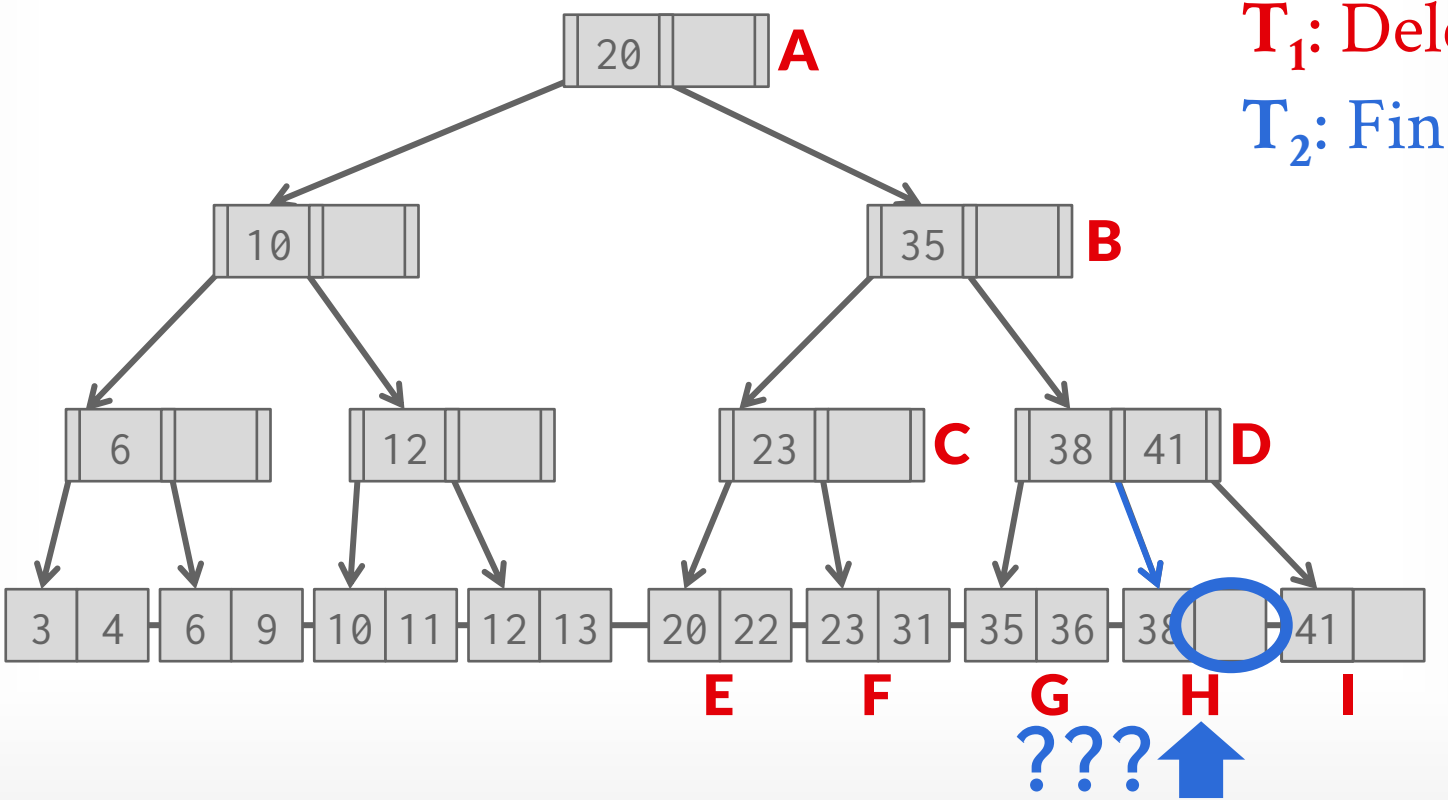
T_2: Find 41



B+TREE MULTI-THREADED EXAMPLE

T_1: Delete 44

T_2: Find 41



LATCH CRABBING/COUPLING

Protocol to allow multiple threads to access/modify B+Tree at the same time.

→ Get latch for parent
→ Get latch for child
→ Release latch for parent if "safe"

A **safe node** is one that will not split or merge when updated.

→ Not full (on insertion)
→ More than half-full (on deletion)

LATCH CRABBING/COUPLING

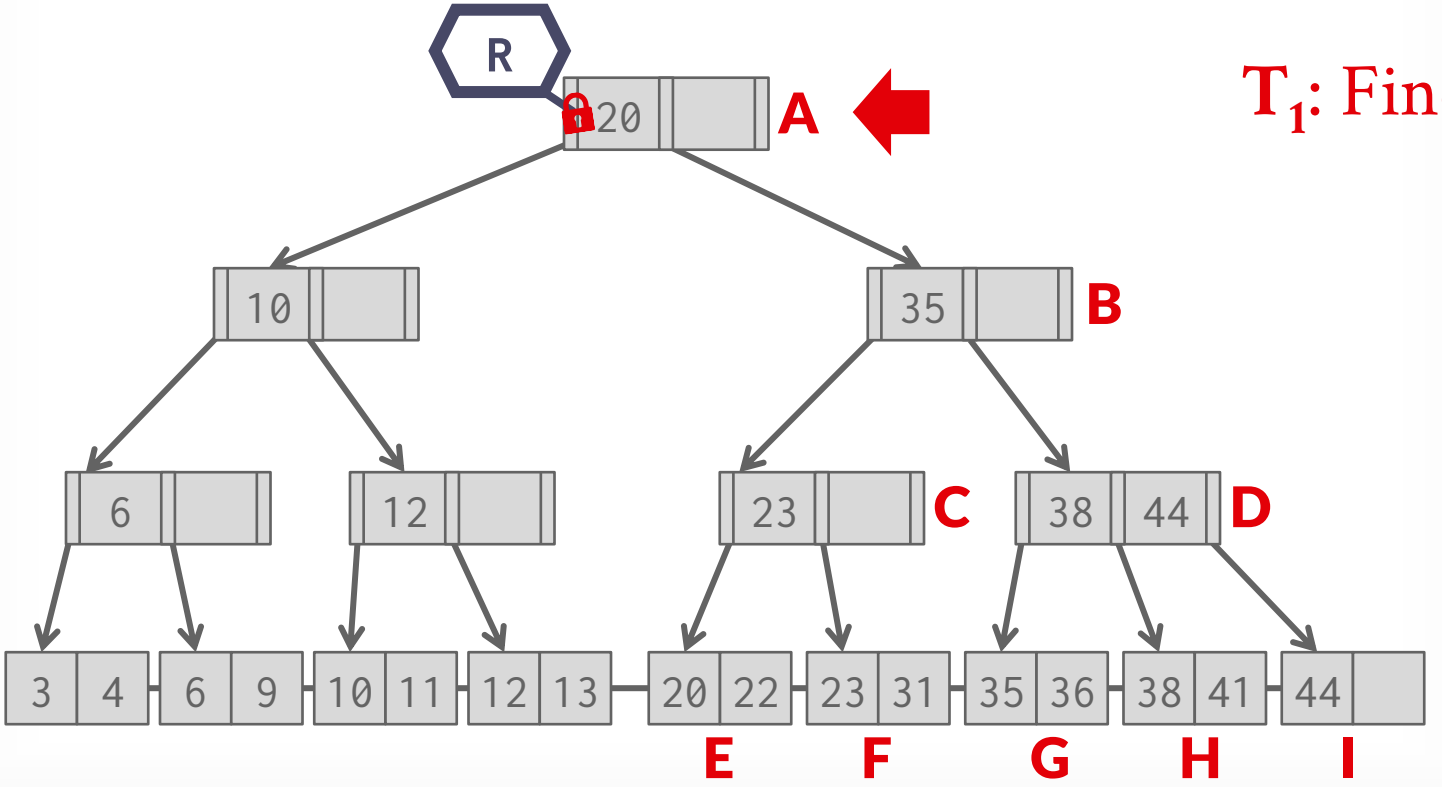
Find: Start at root and traverse down the tree:

→ Acquire **R** latch on child,
→ Then unlatch parent.
→ Repeat until we reach the leaf node.

Insert/Delete: Start at root and go down, obtaining **W** latches as needed. Once child is latched, check if it is safe:

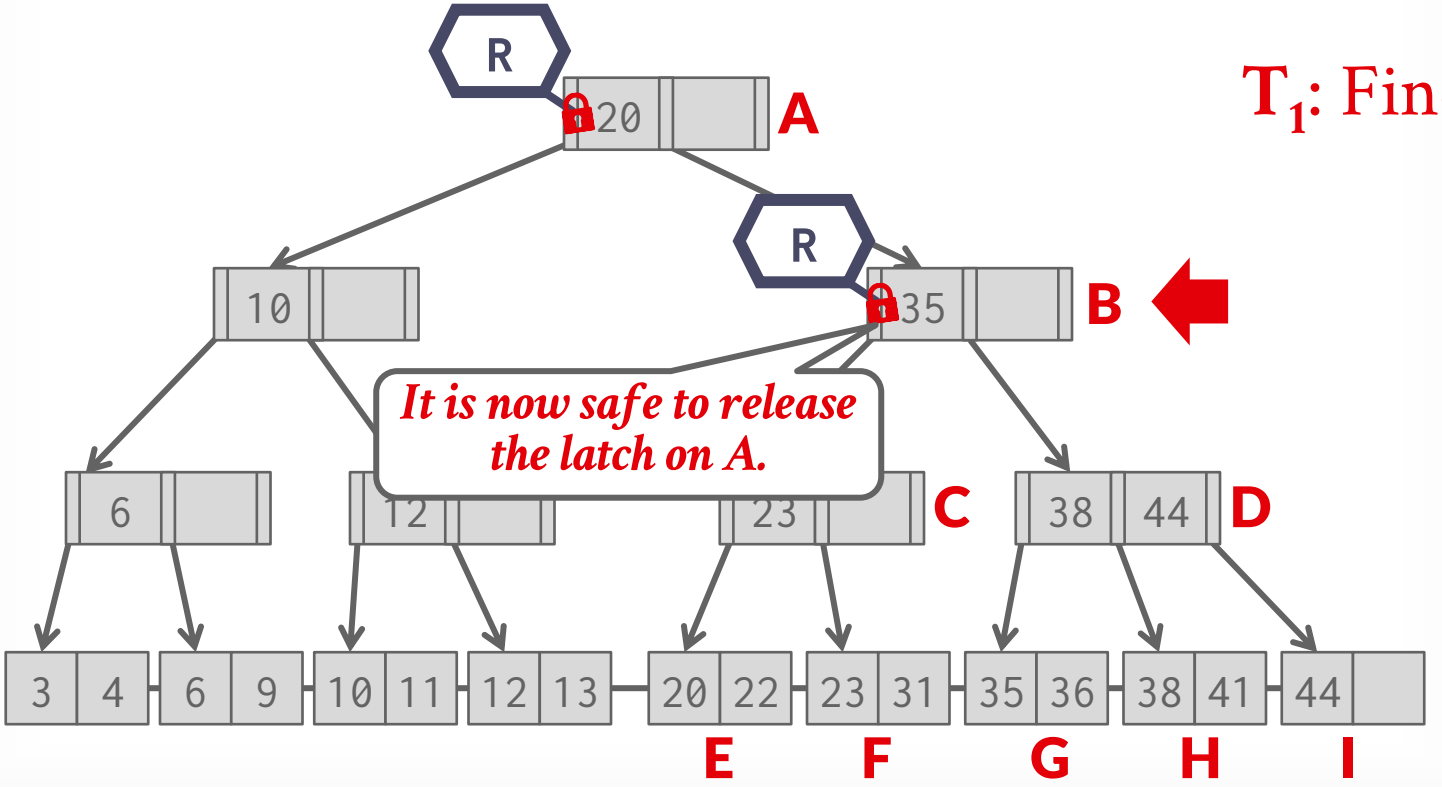
→ If child is safe, release all latches on ancestors.
→ Optimization: When at an inner node that is "safe" can release all latches on ancestors.

EXAMPLE #1: FIND 38



T_1: Find 38

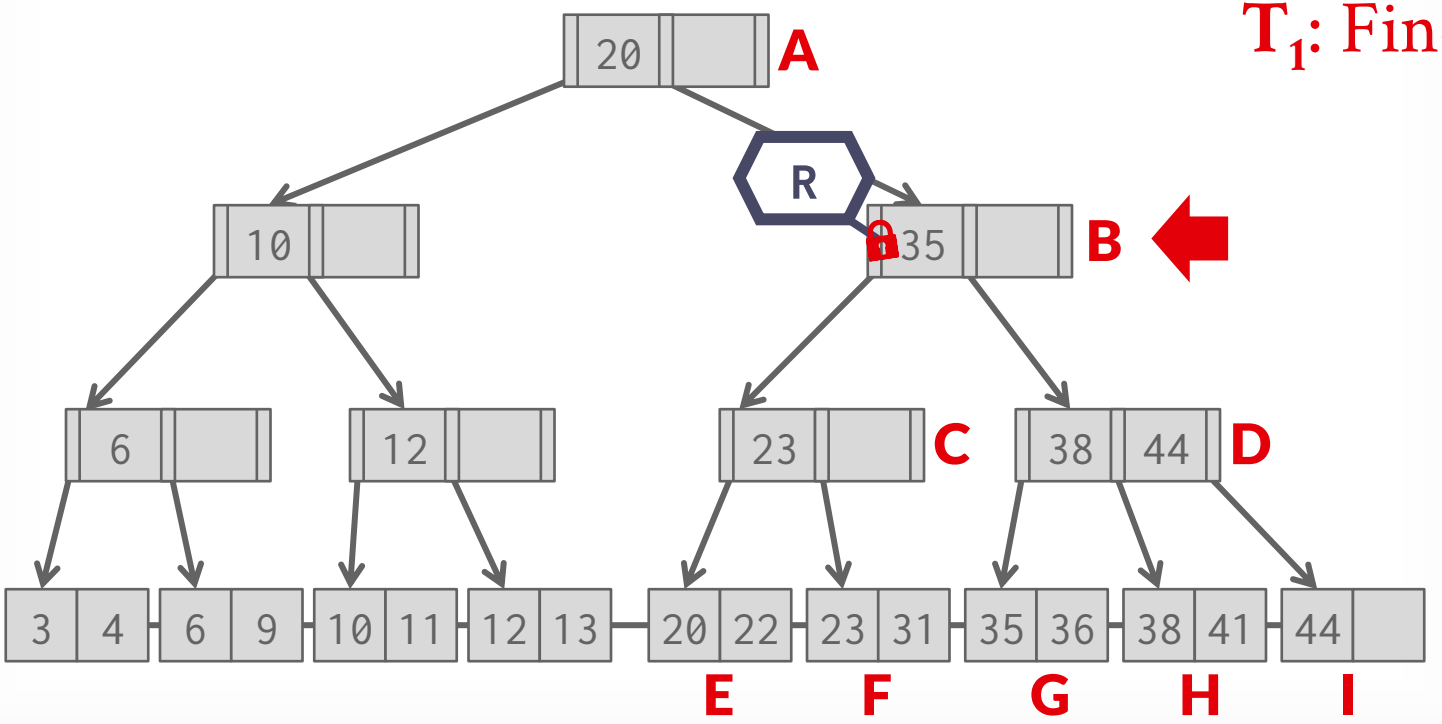
EXAMPLE #1: FIND 38



T_1: Find 38

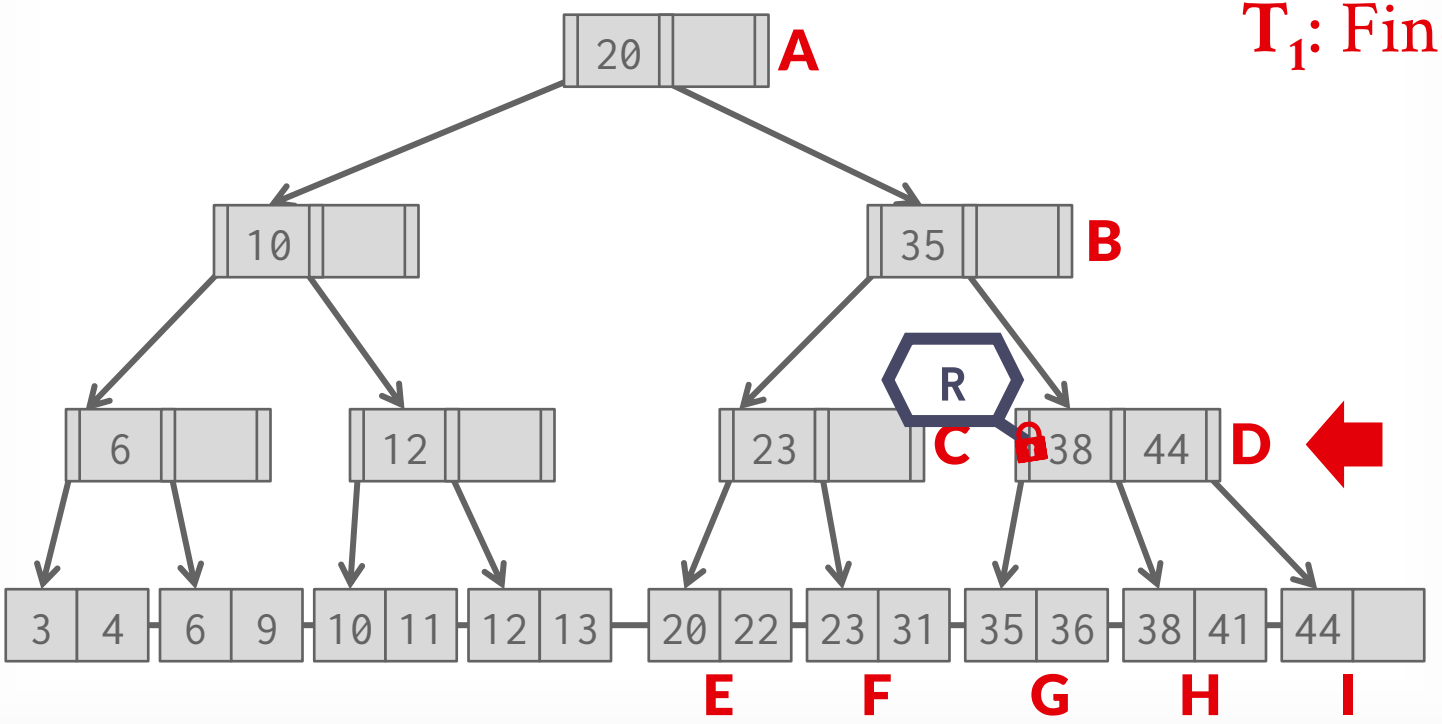
EXAMPLE #1: FIND 38

T_1: Find 38



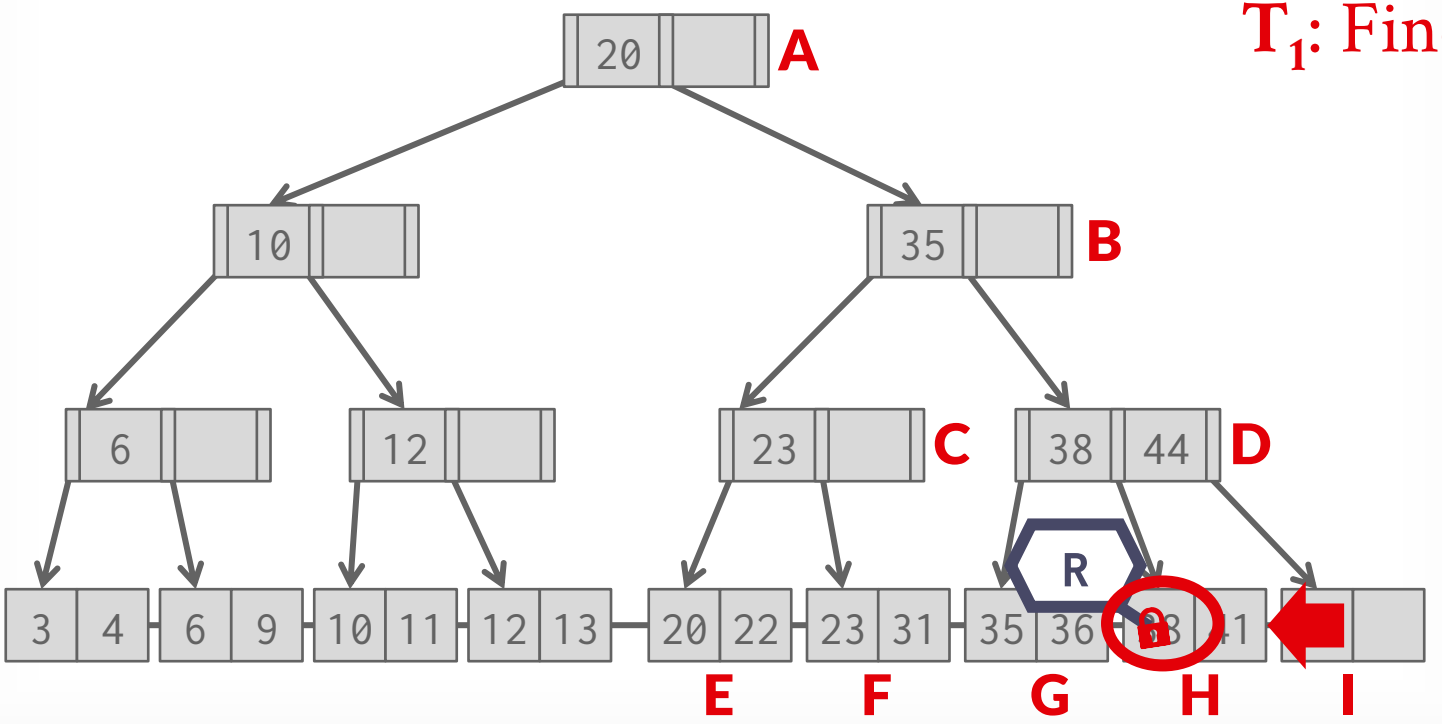
EXAMPLE #1: FIND 38

T_1: Find 38



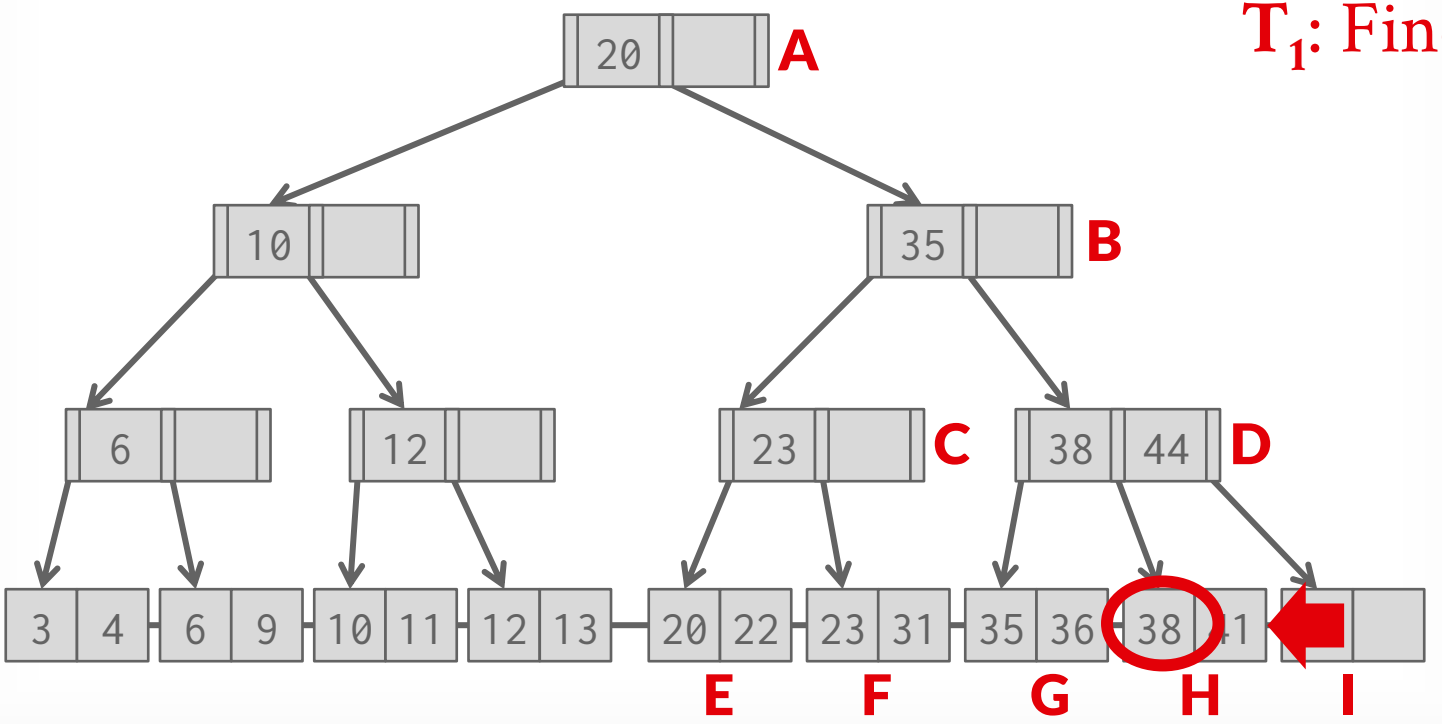
EXAMPLE #1: FIND 38

T_1: Find 38

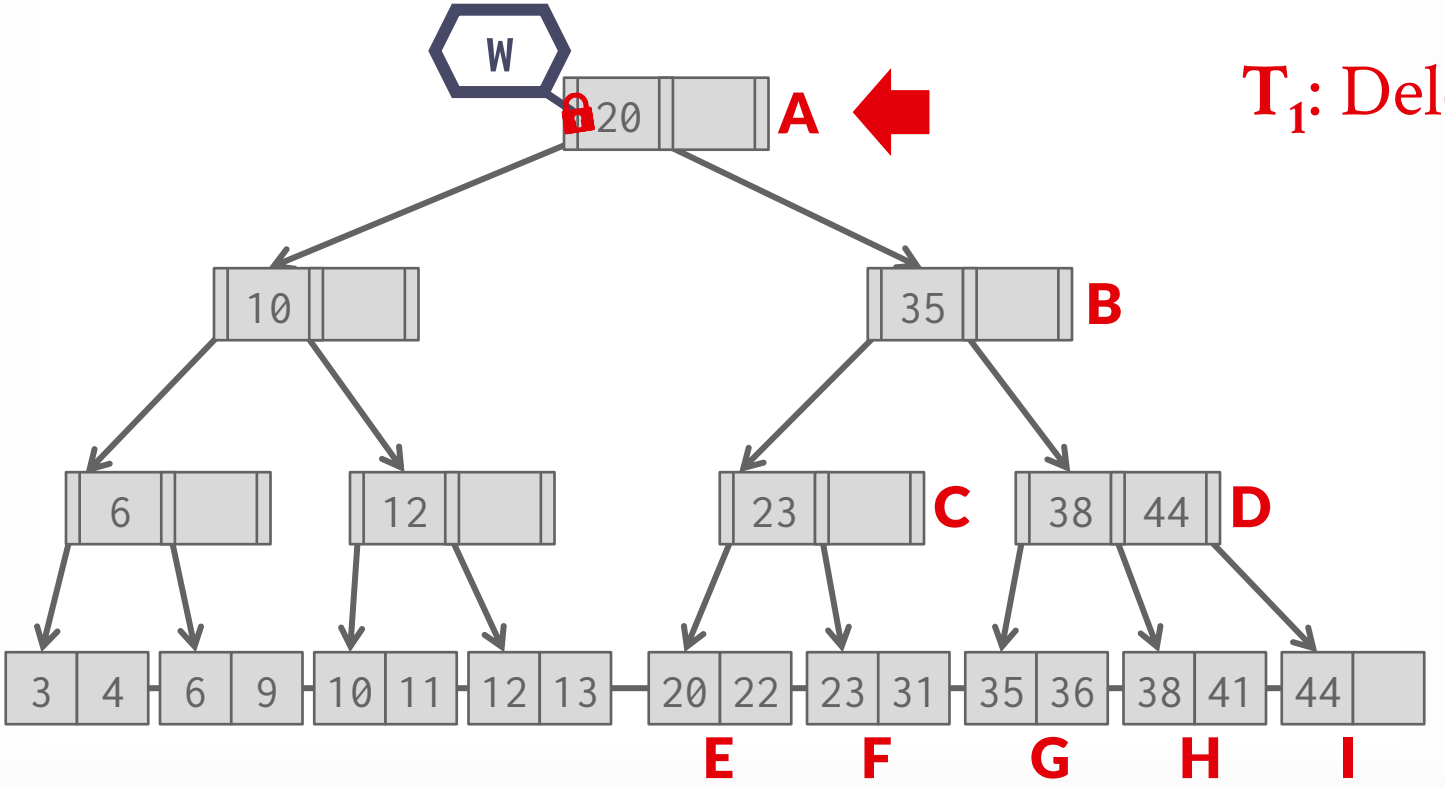


EXAMPLE #1: FIND 38

T_1: Find 38



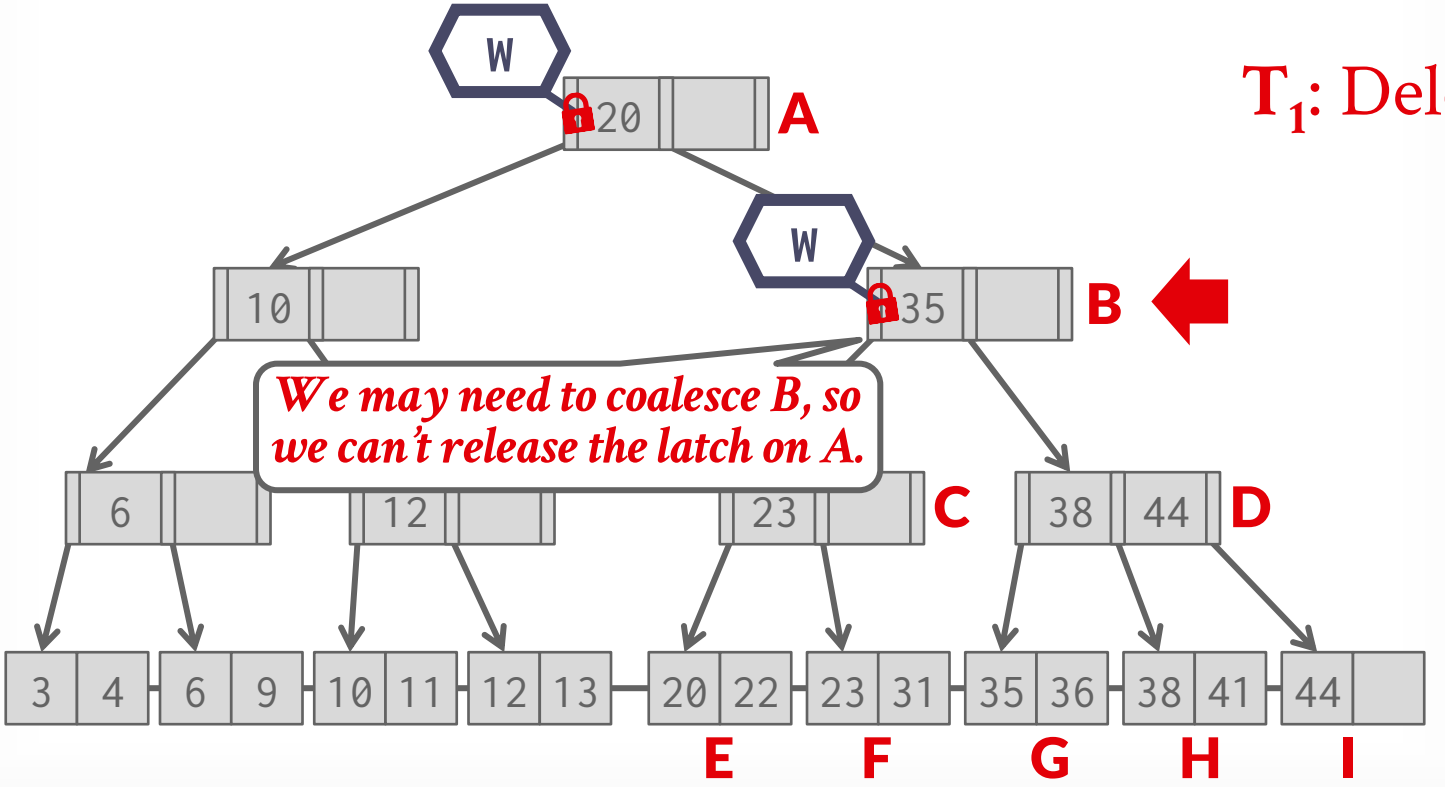
EXAMPLE #2: DELETE 38



T_1: Delete 38

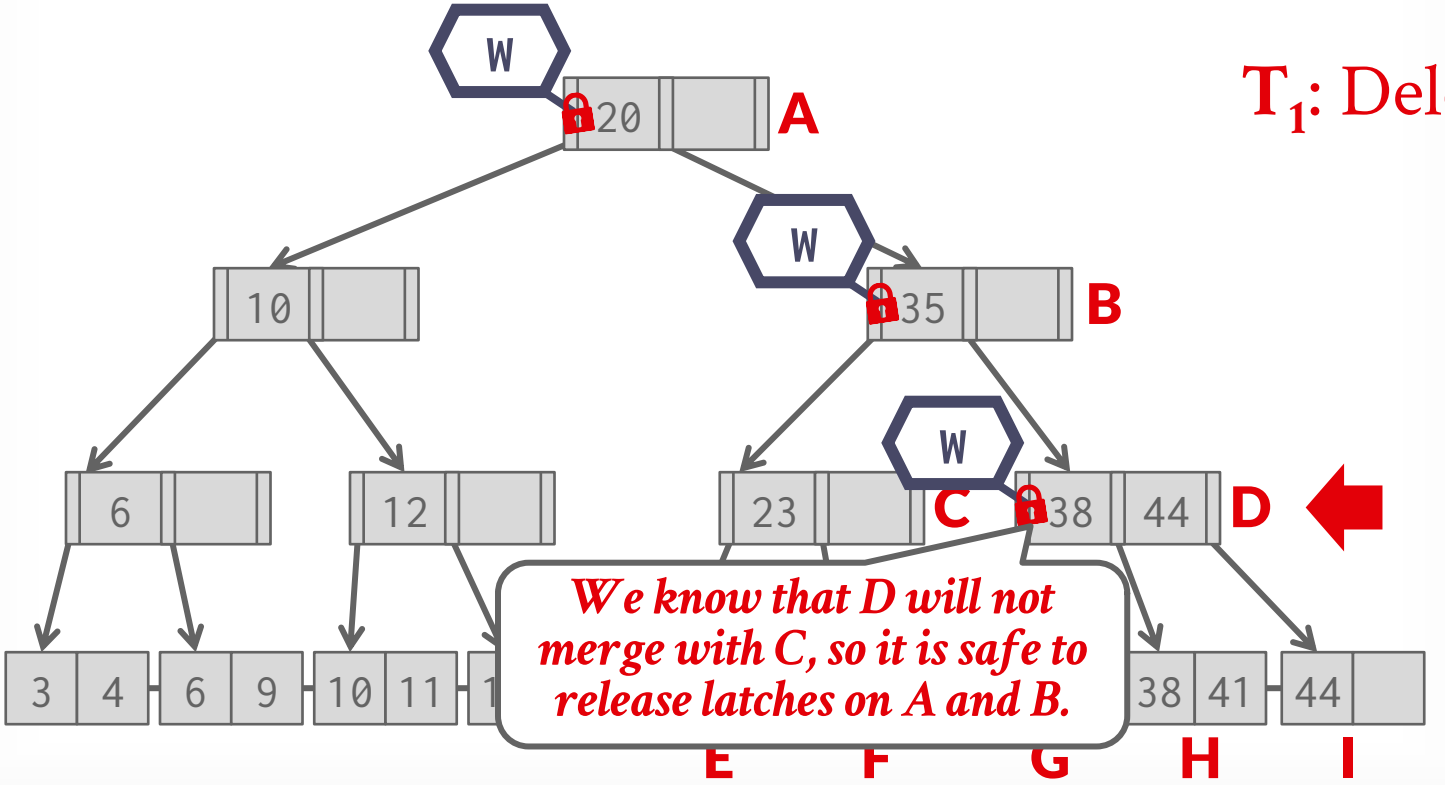
EXAMPLE #2: DELETE 38

T_1: Delete 38



EXAMPLE #2: DELETE 38

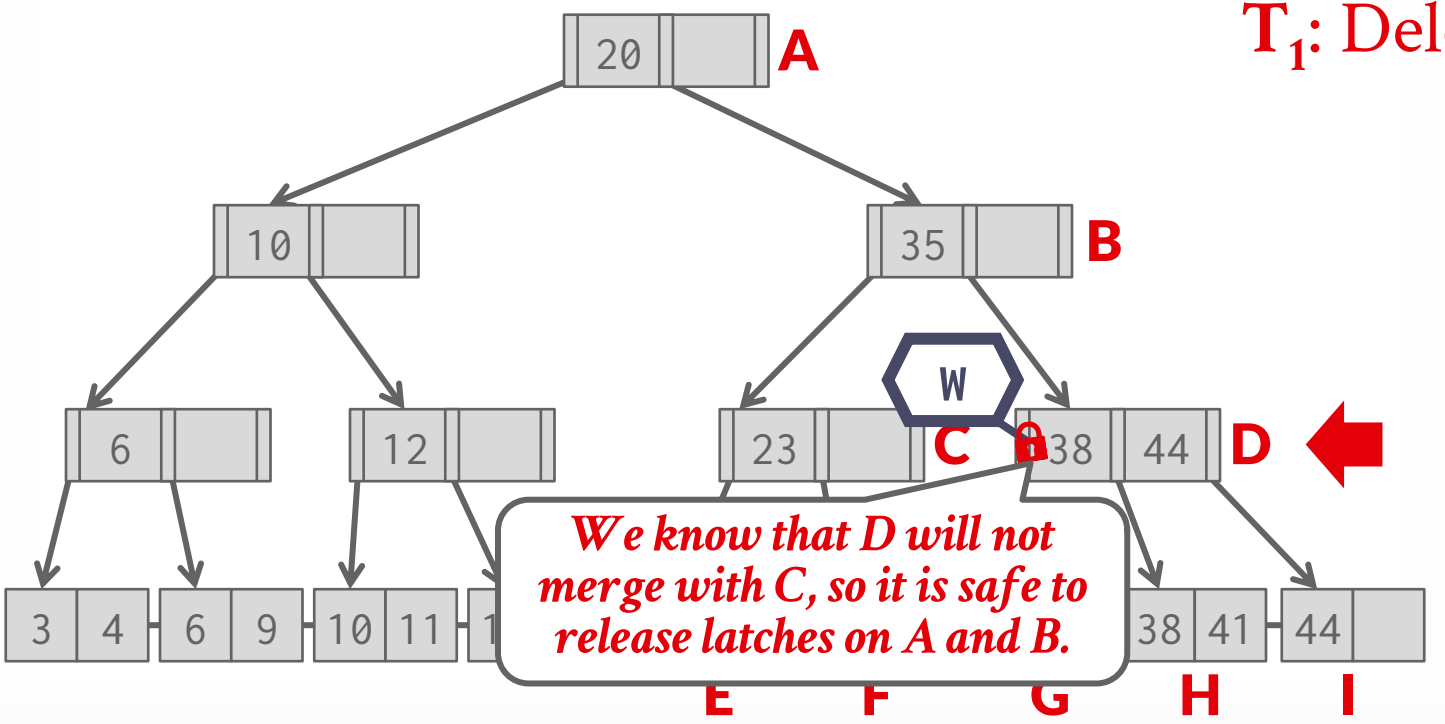
T_1: Delete 38



We know that D will not merge with C, so it is safe to release latches on A and B.

EXAMPLE #2: DELETE 38

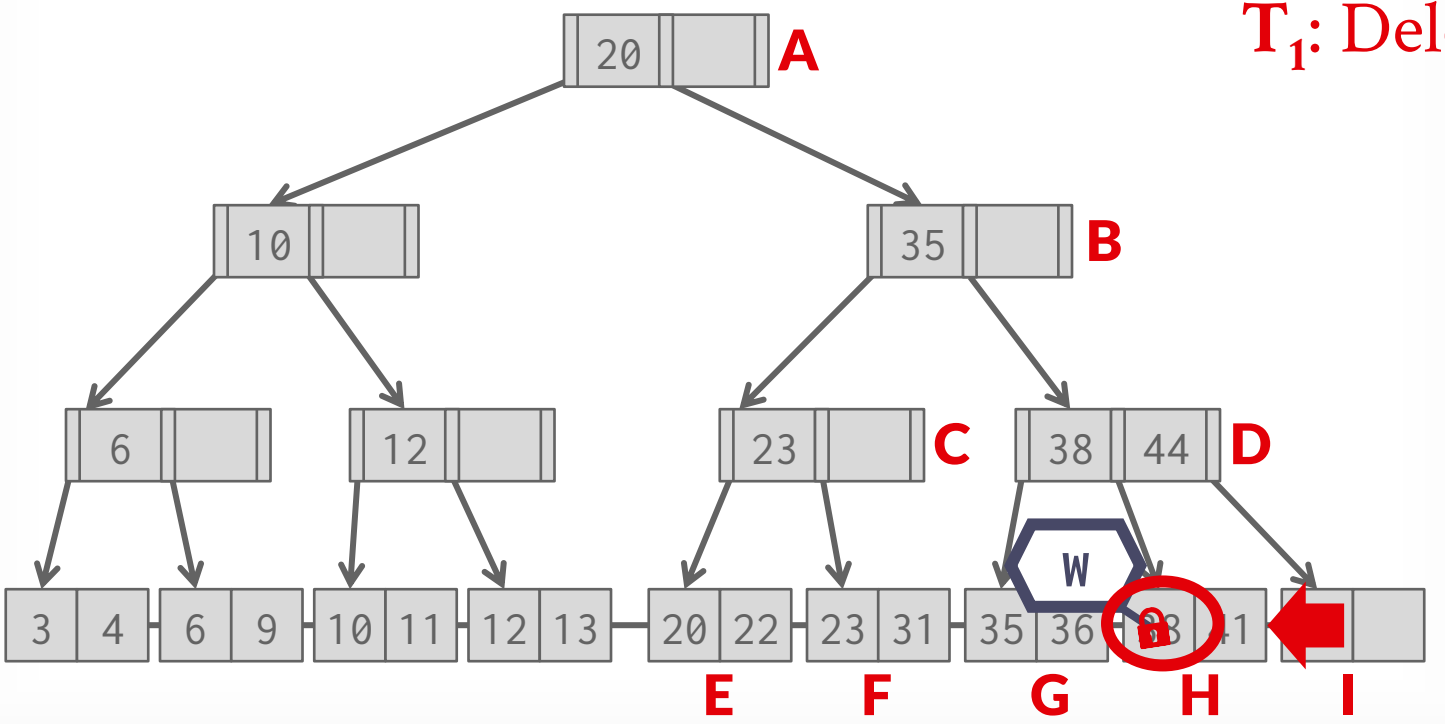
T_1: Delete 38



We know that D will not merge with C, so it is safe to release latches on A and B.

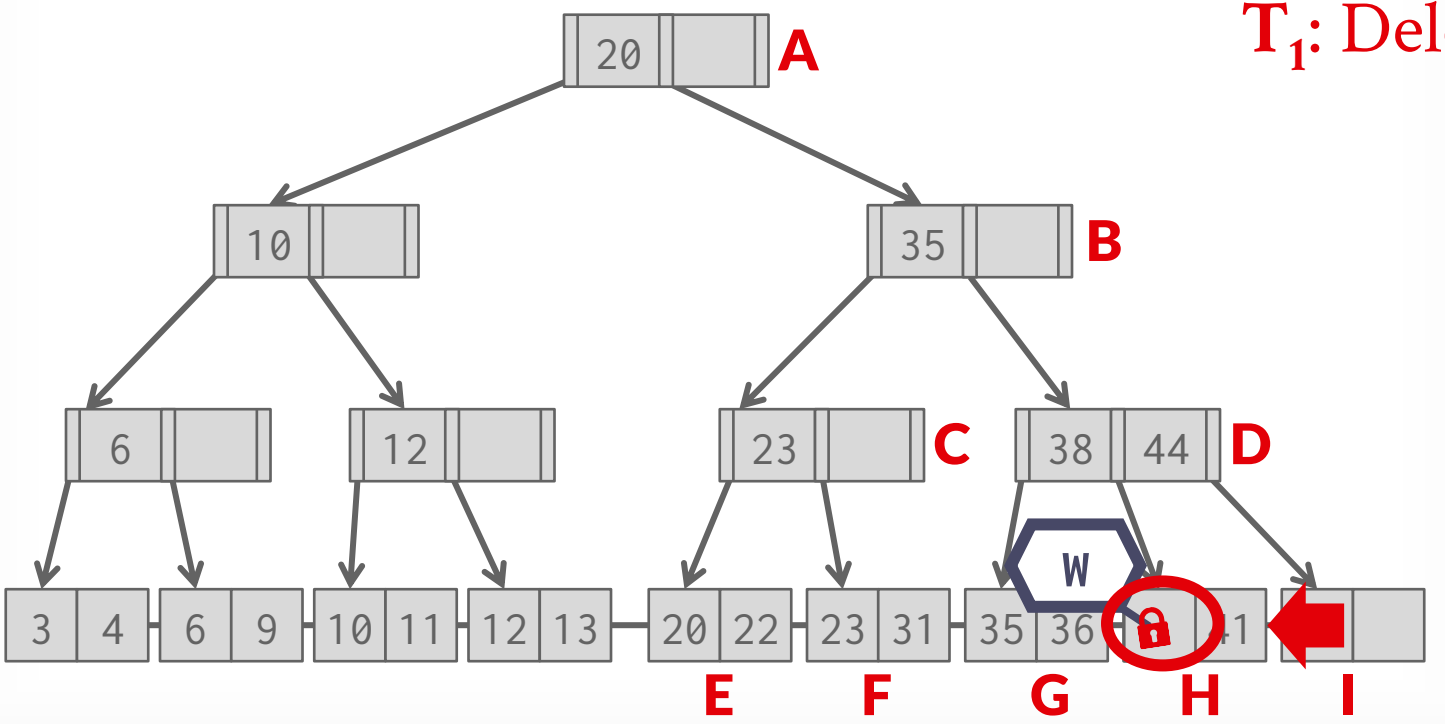
EXAMPLE #2: DELETE 38

T_1: Delete 38



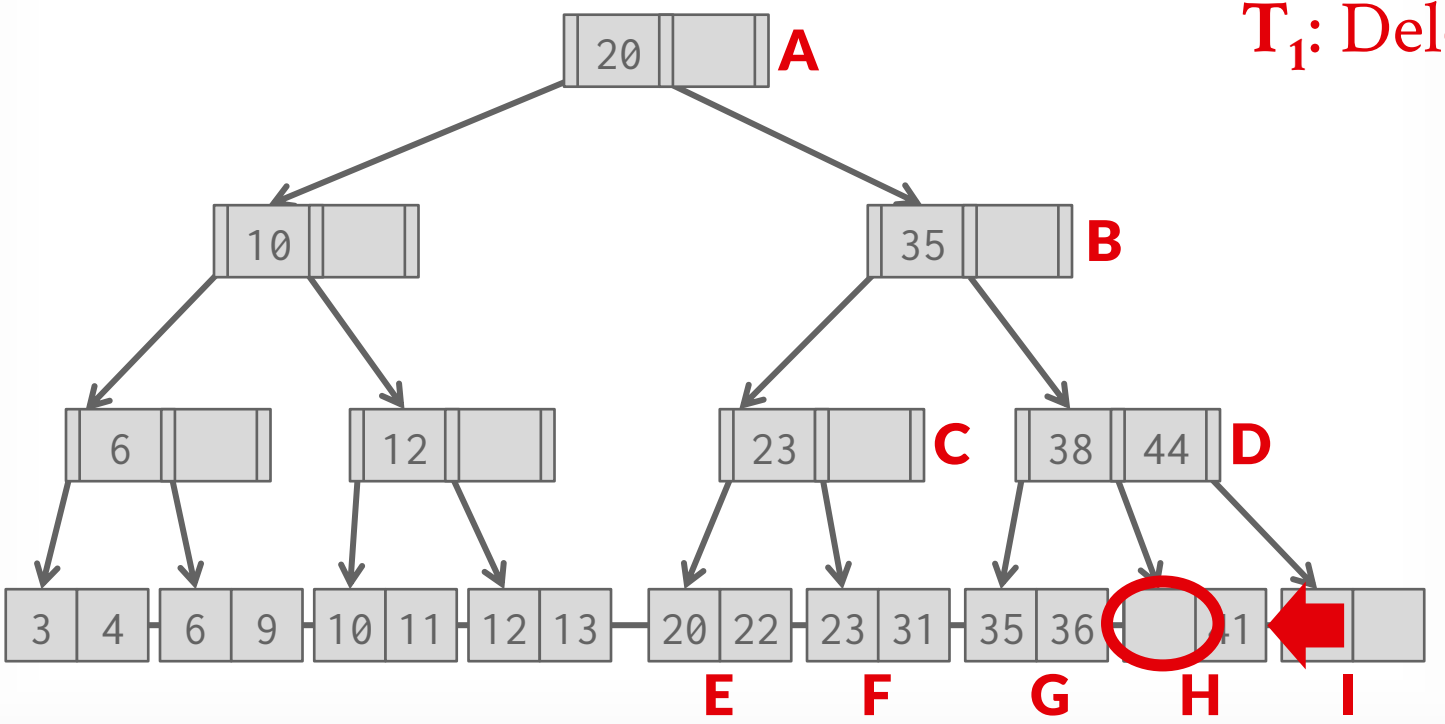
EXAMPLE #2: DELETE 38

T_1: Delete 38



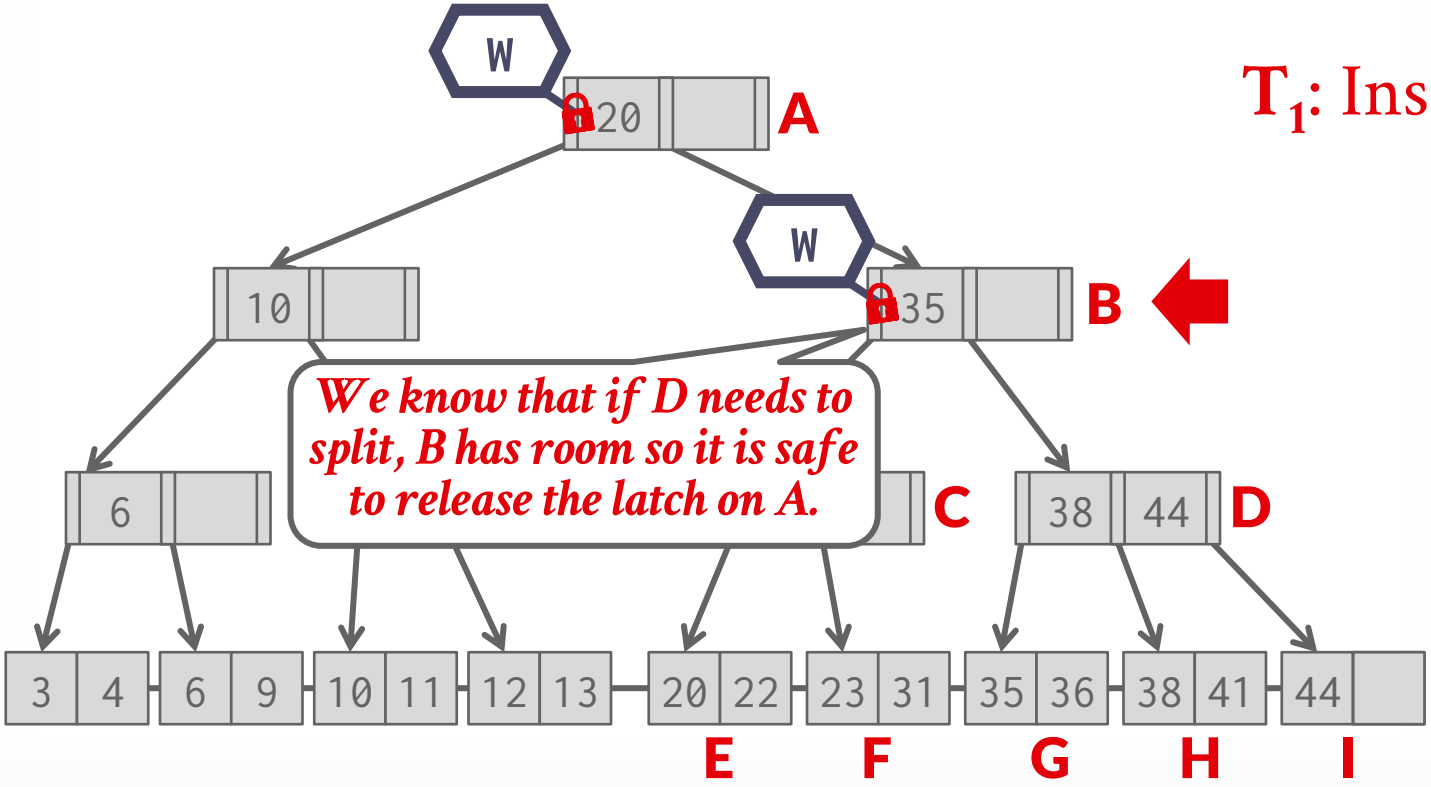
EXAMPLE #2: DELETE 38

T_1: Delete 38



EXAMPLE #3: INSERT 45

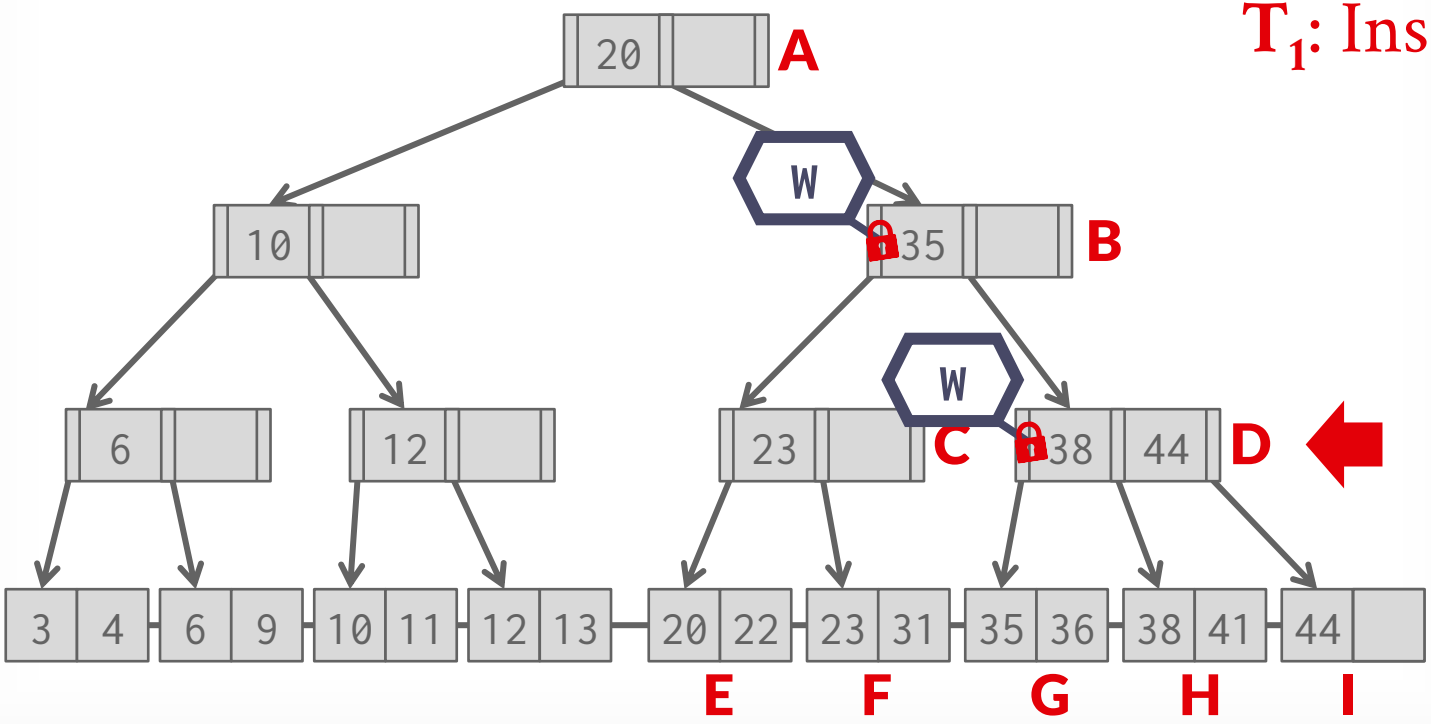
T_1: Insert 45



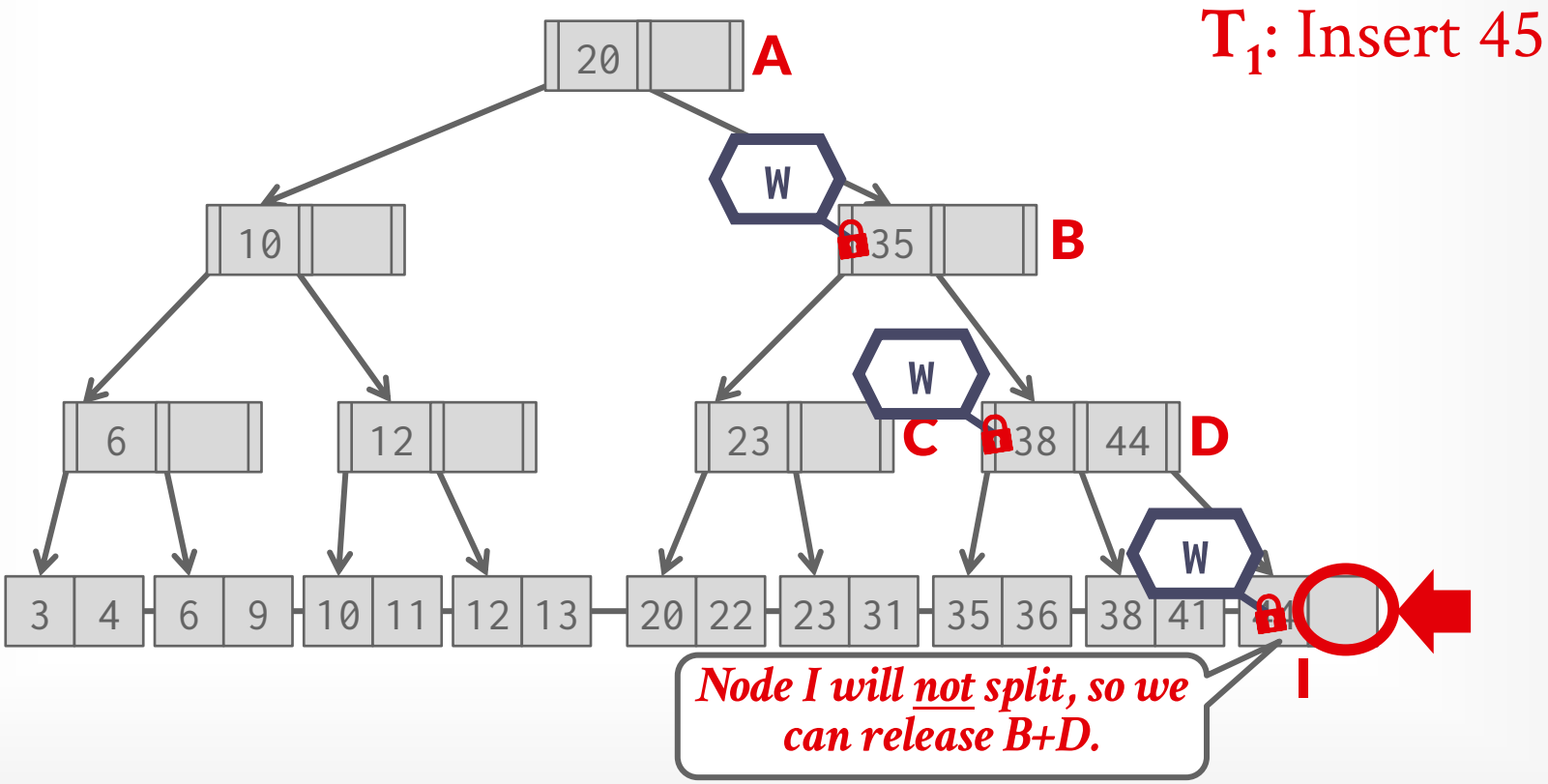
We know that if D needs to split, B has room so it is safe to release the latch on A.

EXAMPLE #3: INSERT 45

T_1: Insert 45

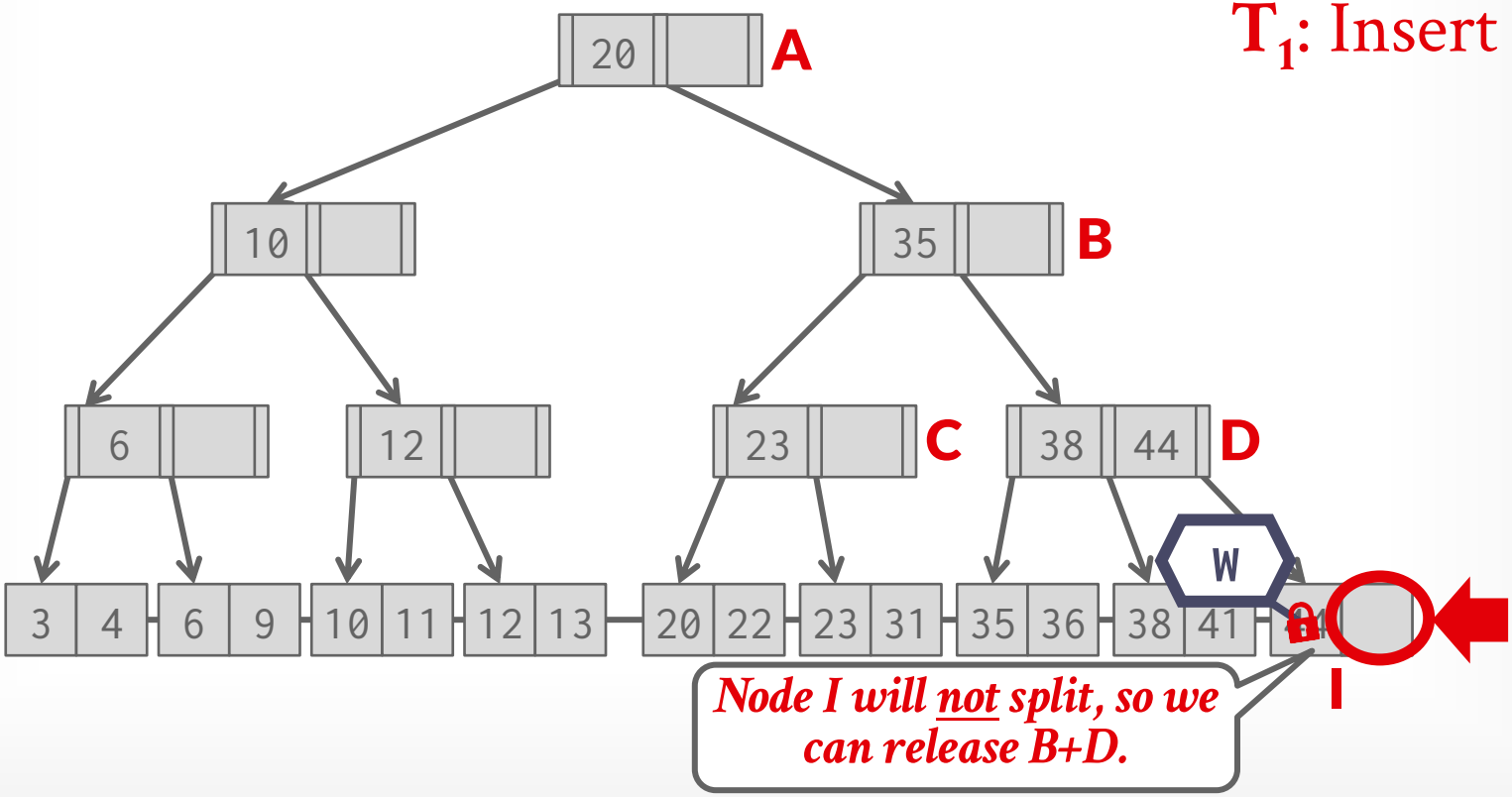


EXAMPLE #3: INSERT 45



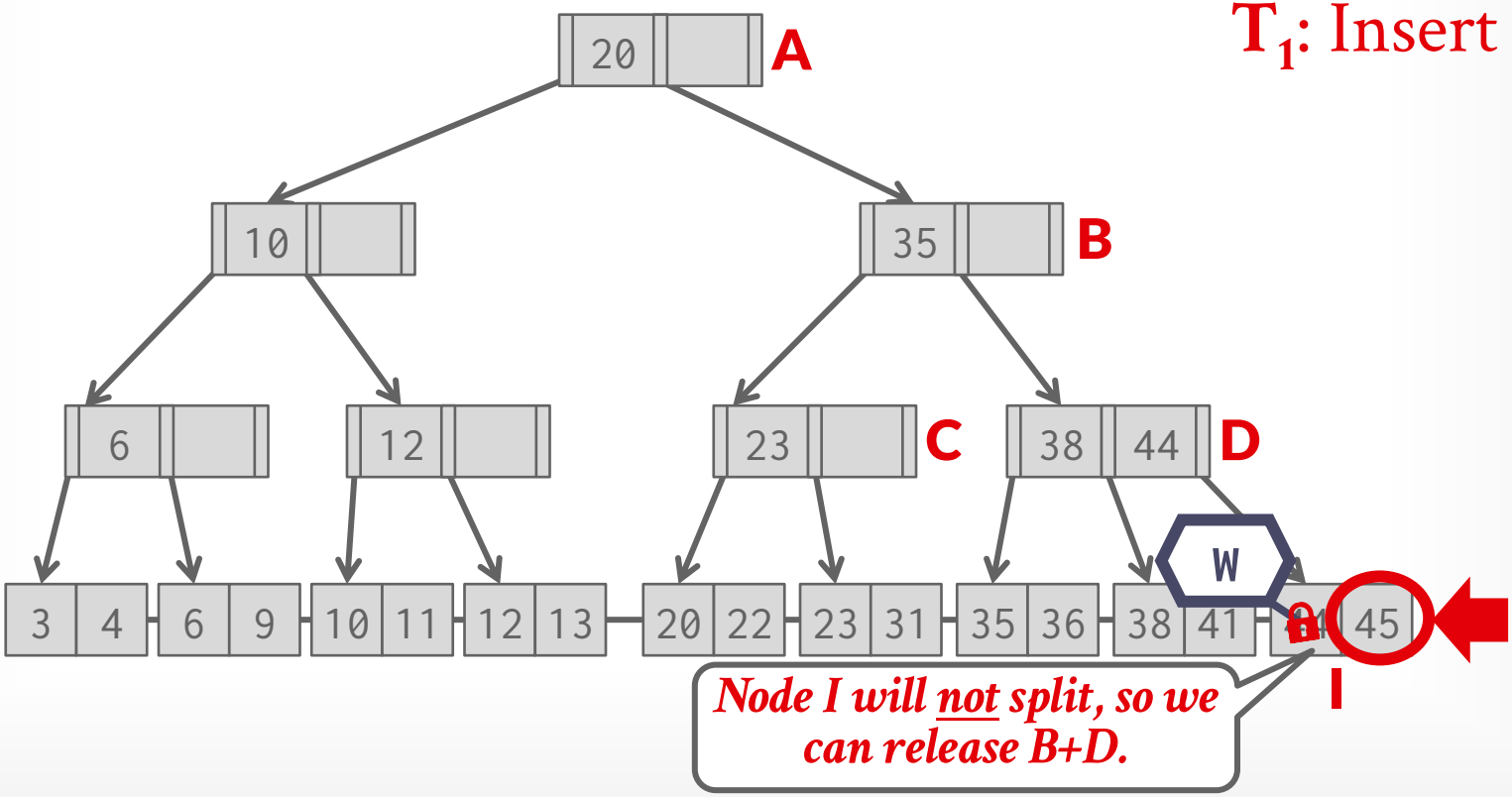
EXAMPLE #3: INSERT 45

T_1: Insert 45



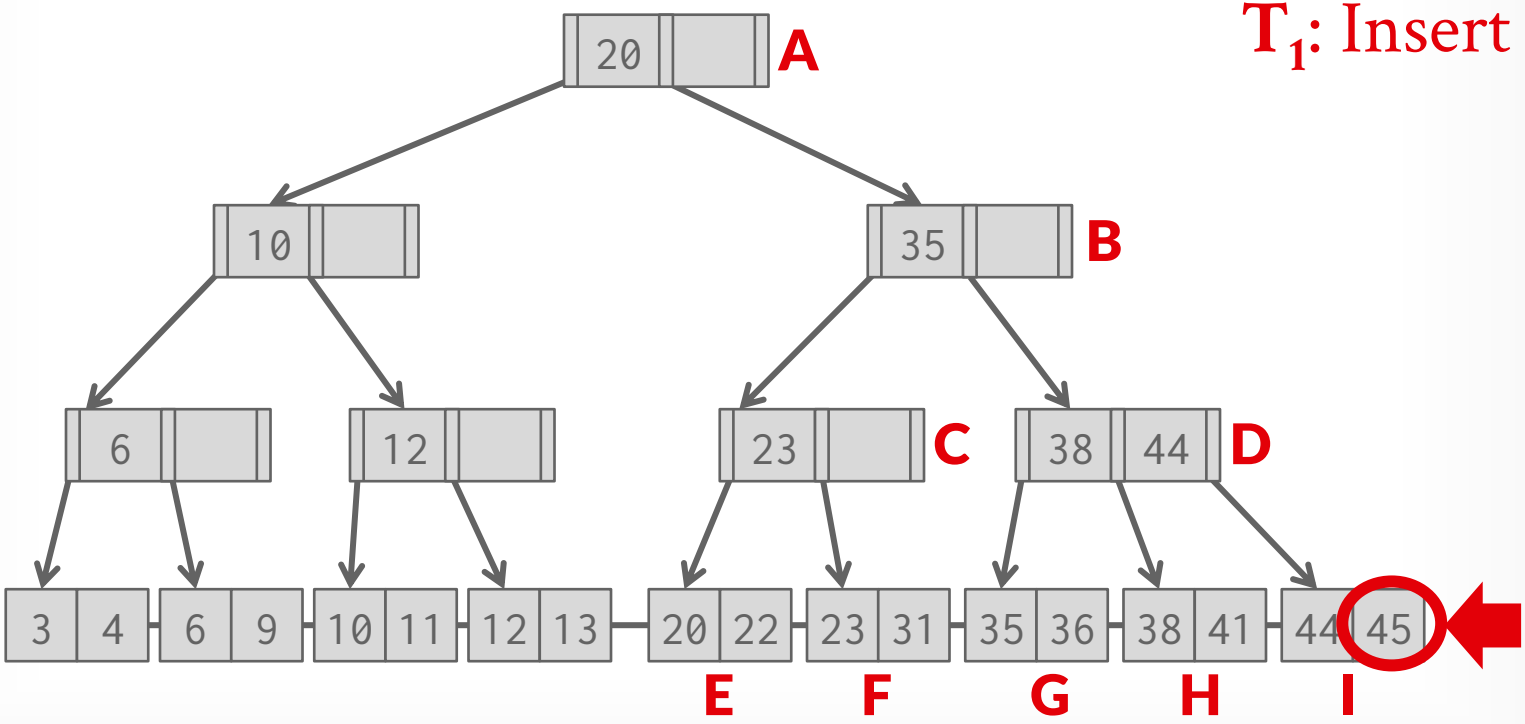
EXAMPLE #3: INSERT 45

T_1: Insert 45

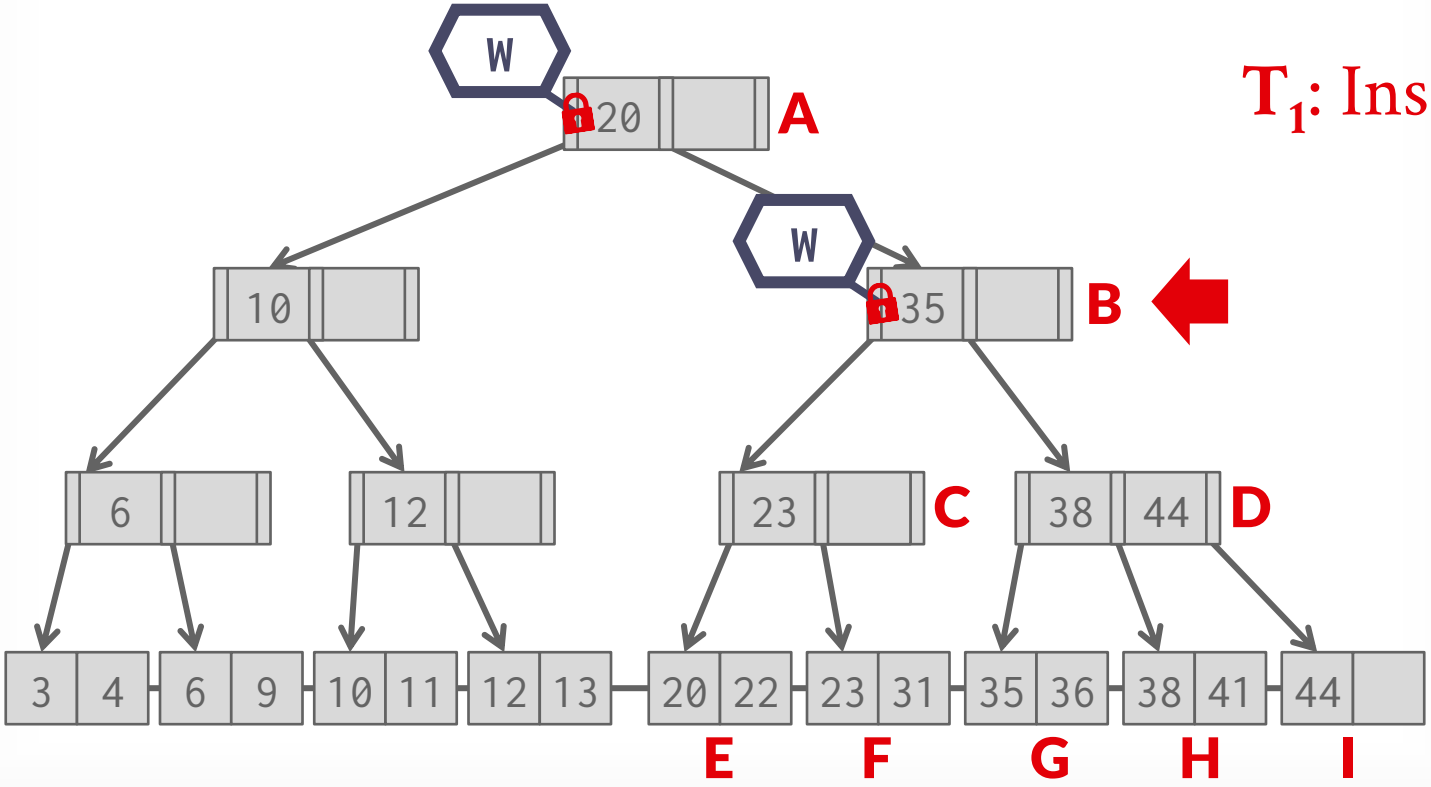


EXAMPLE #3: INSERT 45

T_1: Insert 45



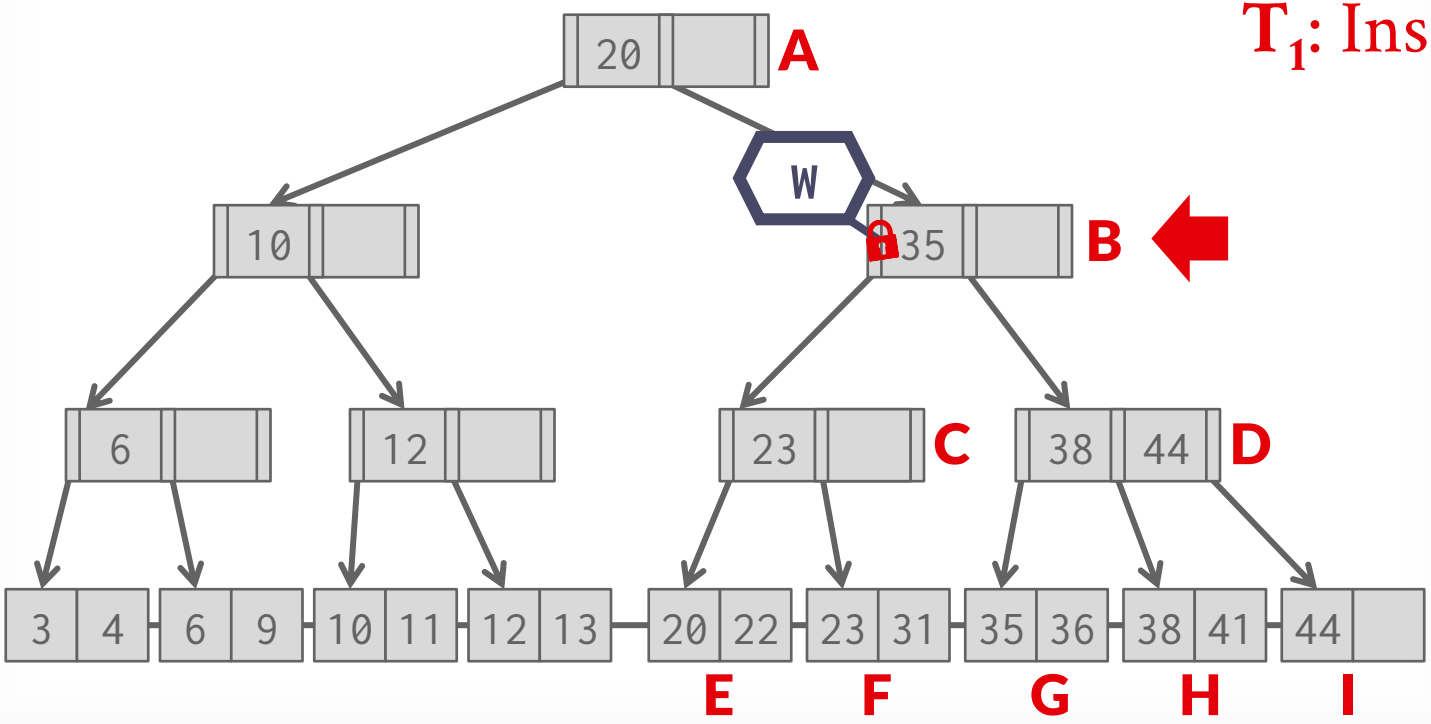
EXAMPLE #4: INSERT 25



T_1: Insert 25

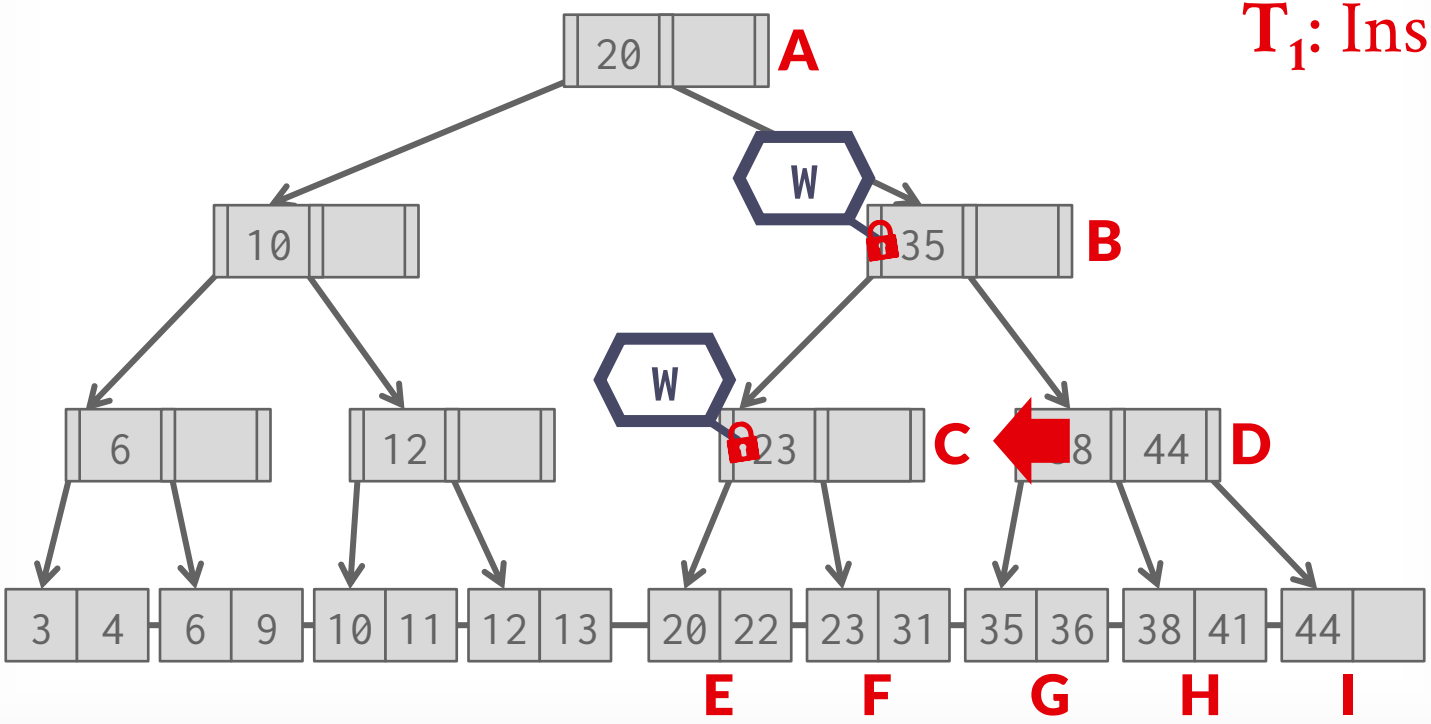
EXAMPLE #4: INSERT 25

T_1: Insert 25



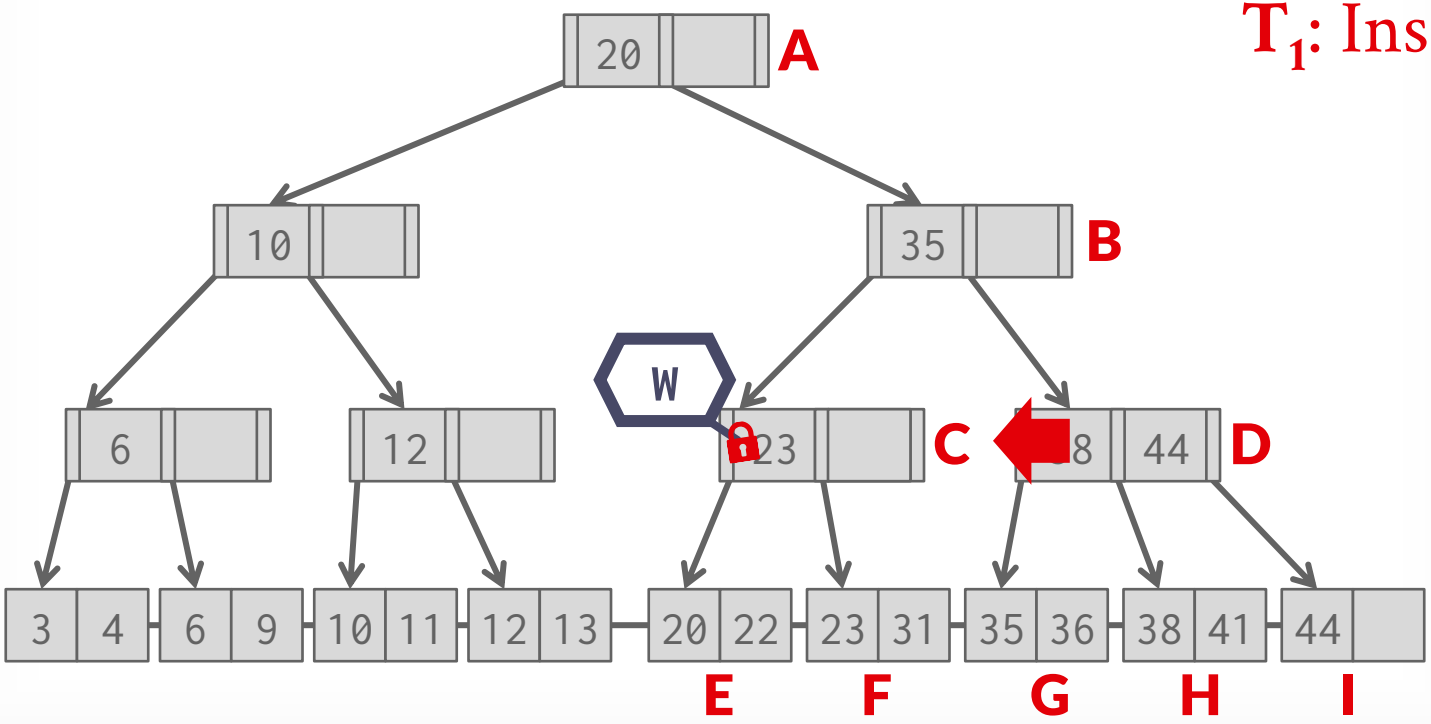
EXAMPLE #4: INSERT 25

T_1: Insert 25



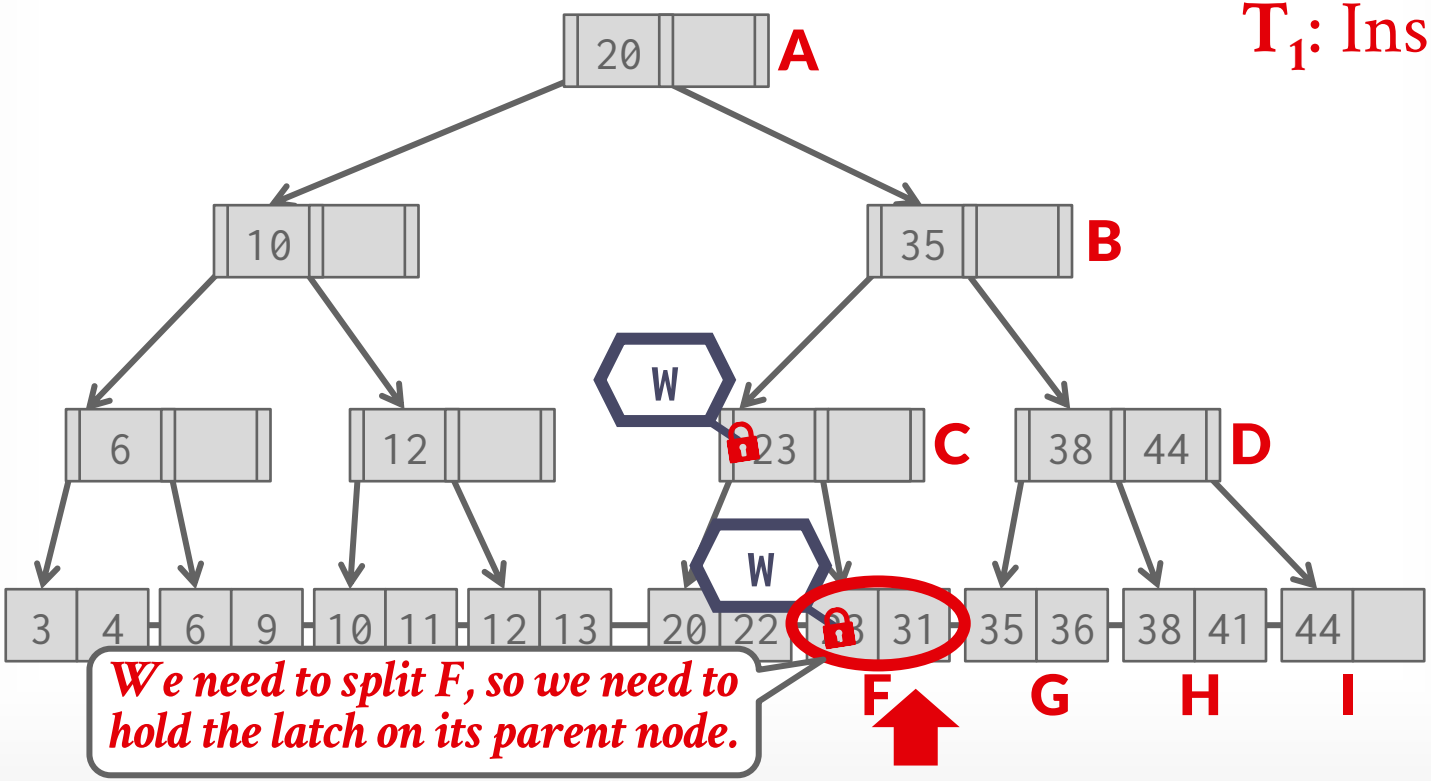
EXAMPLE #4: INSERT 25

T_1: Insert 25



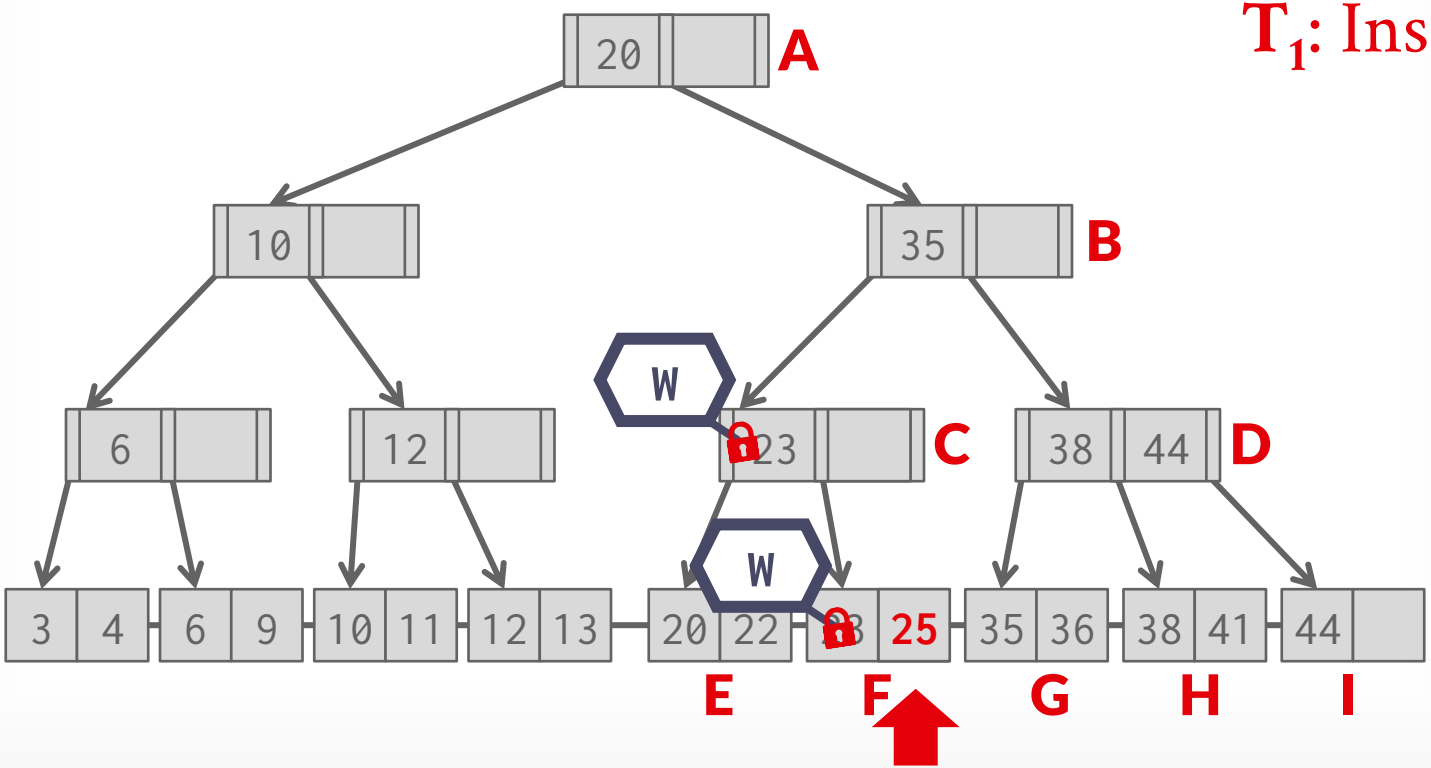
EXAMPLE #4: INSERT 25

T_1: Insert 25

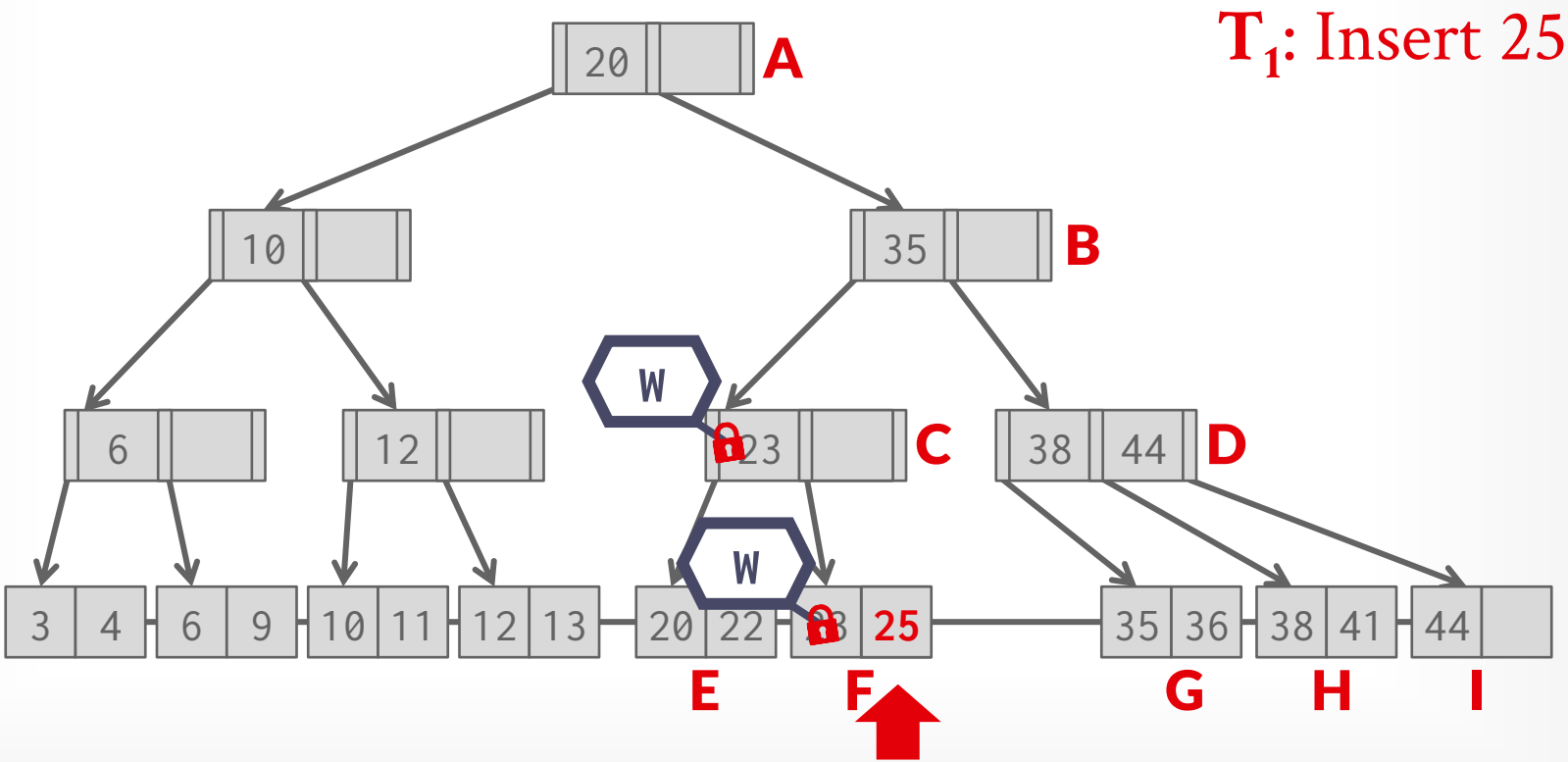


EXAMPLE #4: INSERT 25

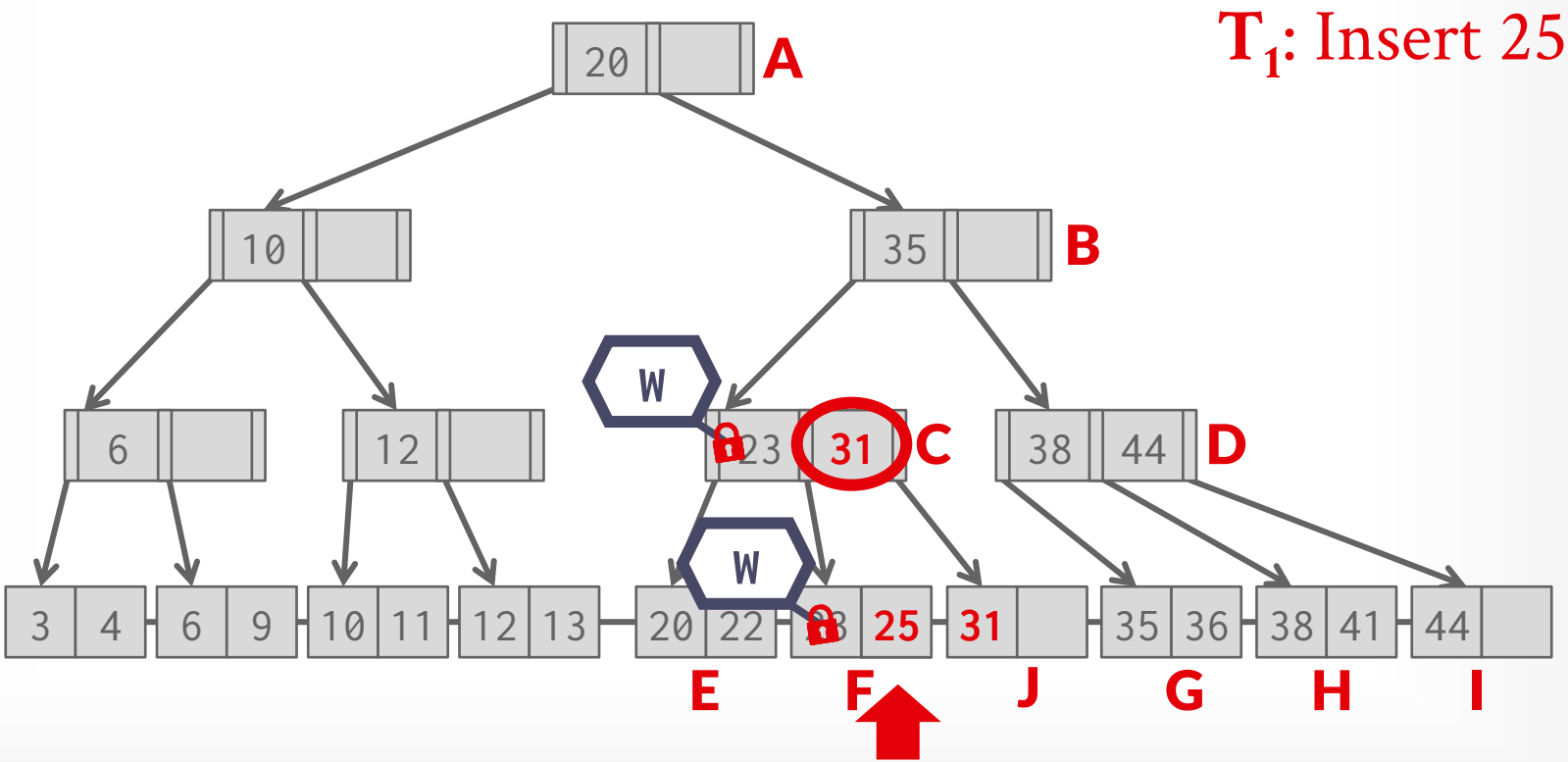
T_1: Insert 25



EXAMPLE #4: INSERT 25

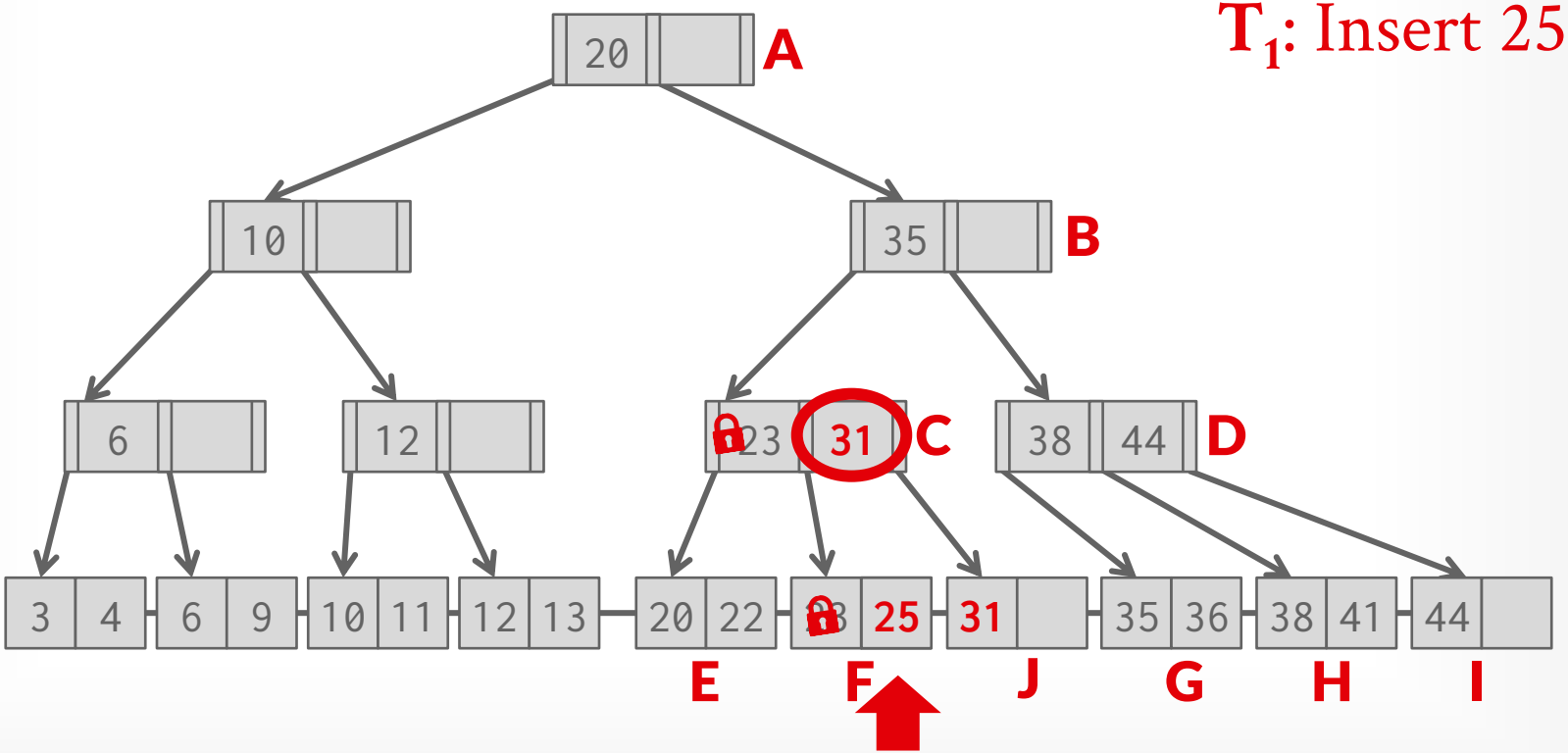


EXAMPLE #4: INSERT 25



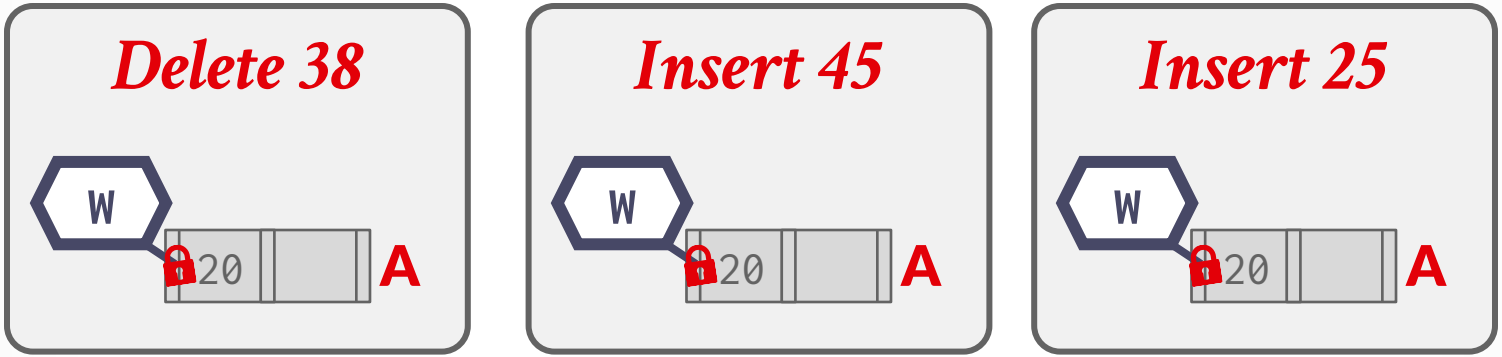
EXAMPLE #4: INSERT 25

T_1: Insert 25



OBSERVATION

What was the first step that all the update examples did on the B+Tree?



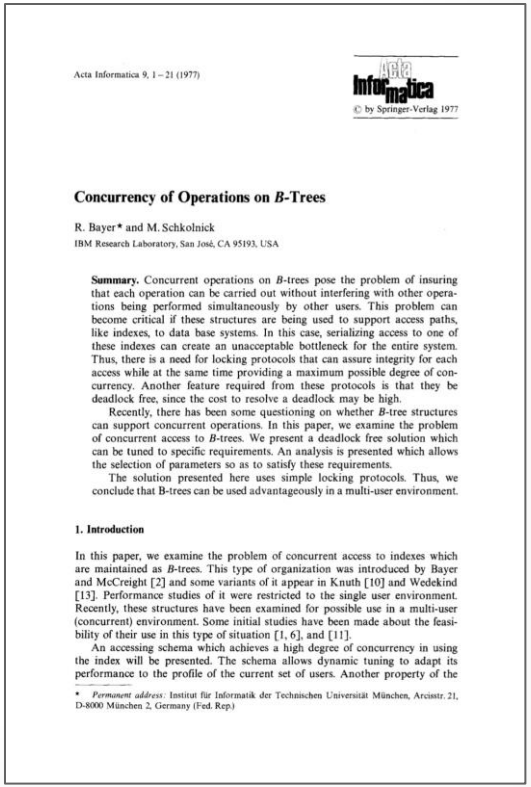
Taking a write latch on the root every time becomes a bottleneck with higher concurrency.

BETTER LATCHING ALGORITHM

Most modifications to a B+Tree will <u>not</u> require a split or merge.

Instead of assuming there will be a split/merge, optimistically traverse the tree using read latches.

If a worker guesses wrong, repeat traversal with pessimistic algorithm.



Acta Informatica 9, 1–21 (1977)

Acta Informatica

© by Springer-Verlag 1977

Concurrency of Operations on *B*-Trees

R. Bayer* and M. Schkolnick

IBM Research Laboratory, San José, CA 95193, USA

Summary. Concurrent operations on *B*-trees pose the problem of insuring that each operation can be carried out without interfering with other operations being performed simultaneously by other users. This problem can become critical if these structures are being used to support access paths, like indexes, to data base systems. In this case, serializing access to one of these indexes can create an unacceptable bottleneck for the entire system. Thus, there is a need for locking protocols that can assure integrity for each access while at the same time providing a maximum possible degree of concurrency. Another feature required from these protocols is that they be deadlock free, since the cost to resolve a deadlock may be high.

Recently, there has been some questioning on whether *B*-tree structures can support concurrent operations. In this paper, we examine the problem of concurrent access to *B*-trees. We present a deadlock free solution which can be tuned to specific requirements. An analysis is presented which allows the selection of parameters so as to satisfy these requirements.

The solution presented here uses simple locking protocols. Thus, we conclude that B-trees can be used advantageously in a multi-user environment.

1. Introduction

In this paper, we examine the problem of concurrent access to indexes which are maintained as *B*-trees. This type of organization was introduced by Bayer and McCreight [2] and some variants of it appear in Knuth [10] and Wedekind [13]. Performance studies of it were restricted to the single user environment. Recently, these structures have been examined for possible use in a multi-user (concurrent) environment. Some initial studies have been made about the feasibility of their use in this type of situation [1, 6], and [11].

An accessing schema which achieves a high degree of concurrency in using the index will be presented. The schema allows dynamic tuning to adapt its performance to the profile of the current set of users. Another property of the

* *Permanent address:* Institut für Informatik der Technischen Universität München, Arcisstr. 21, D-8000 München 2, Germany (Fed. Rep.)

BETTER LATCHING ALGORITHM

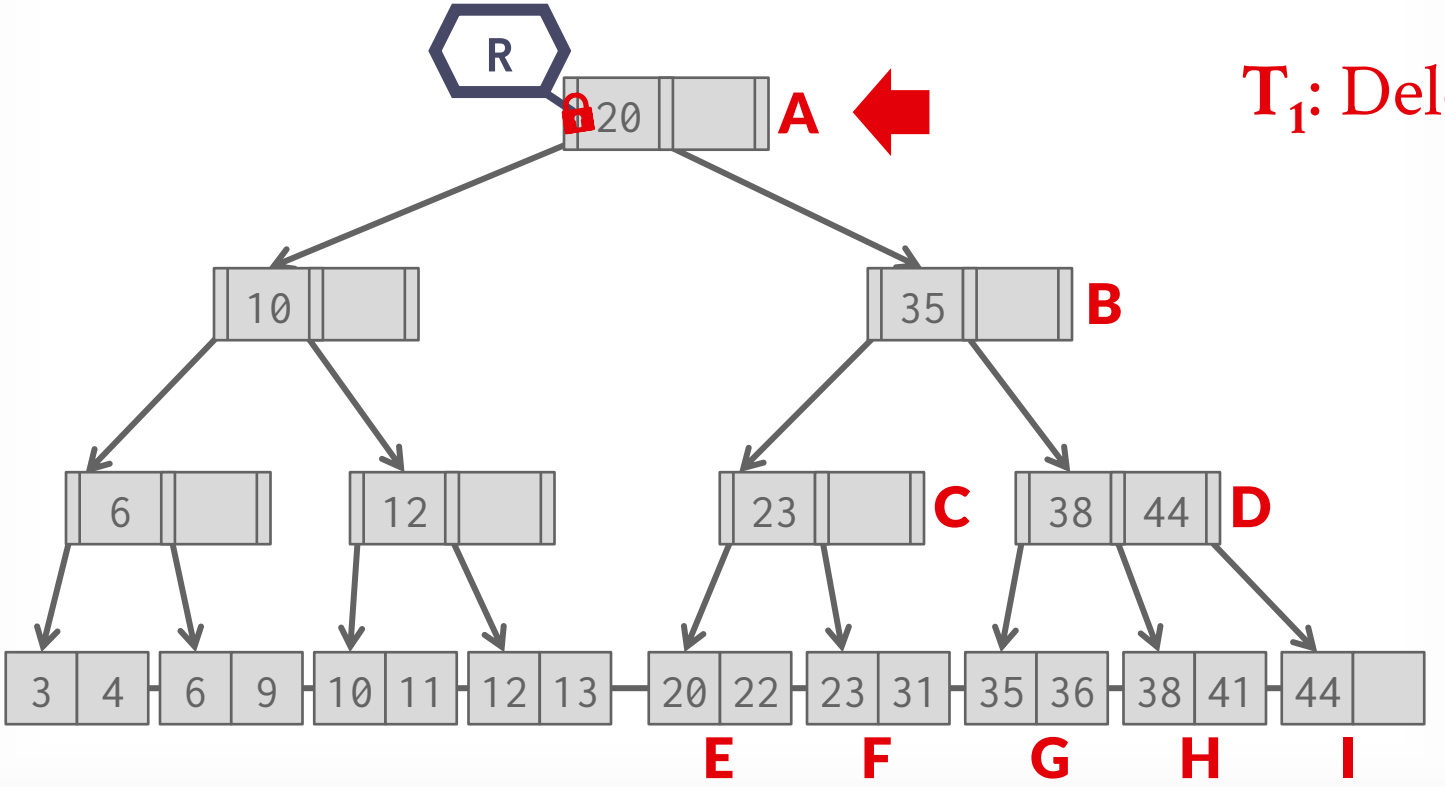
Search: Same as before.

Insert/Delete:

→ Set latches as if for search, get to leaf, and set **W** latch on leaf.
→ If leaf is not safe, release all latches, and restart thread using previous insert/delete protocol with write latches.

This approach optimistically assumes that only leaf node will be modified; if not, **R** latches set on the first pass to leaf are wasteful.

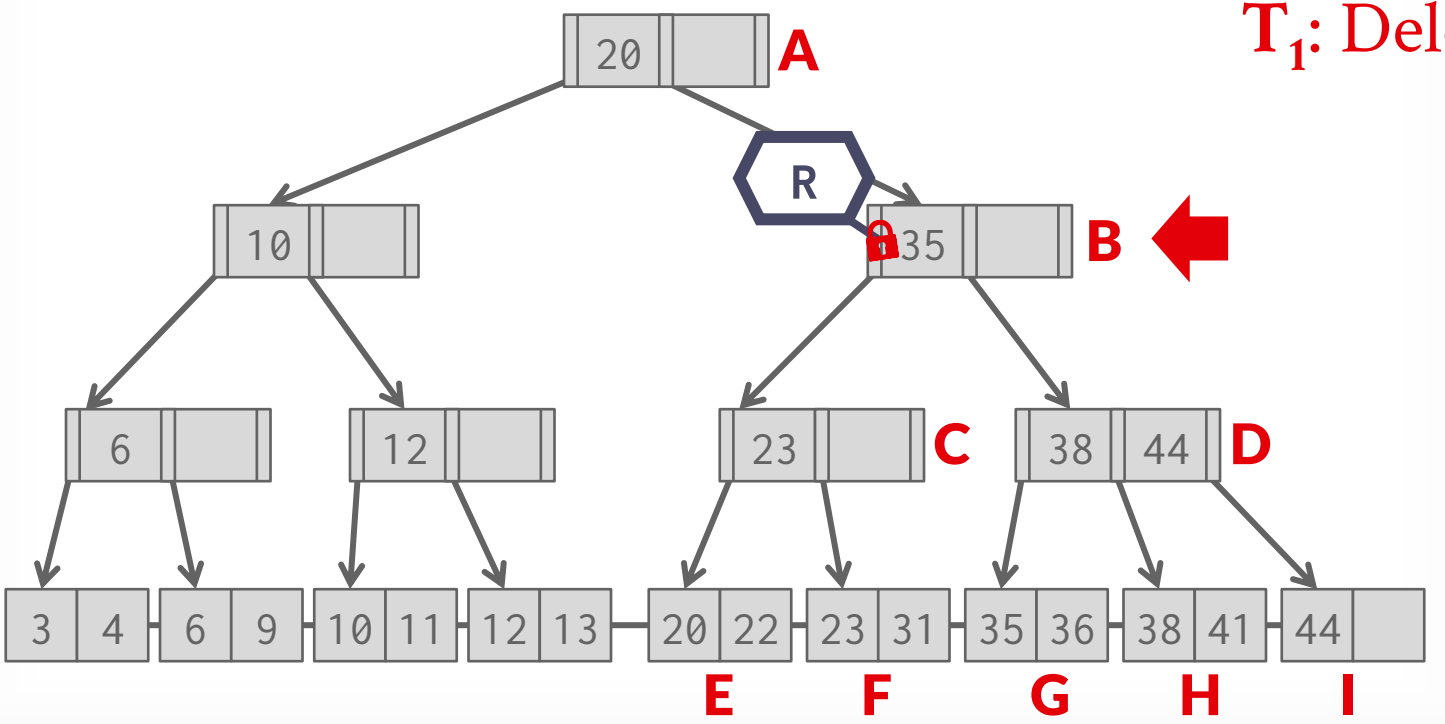
EXAMPLE #2: DELETE 38



T_1: Delete 38

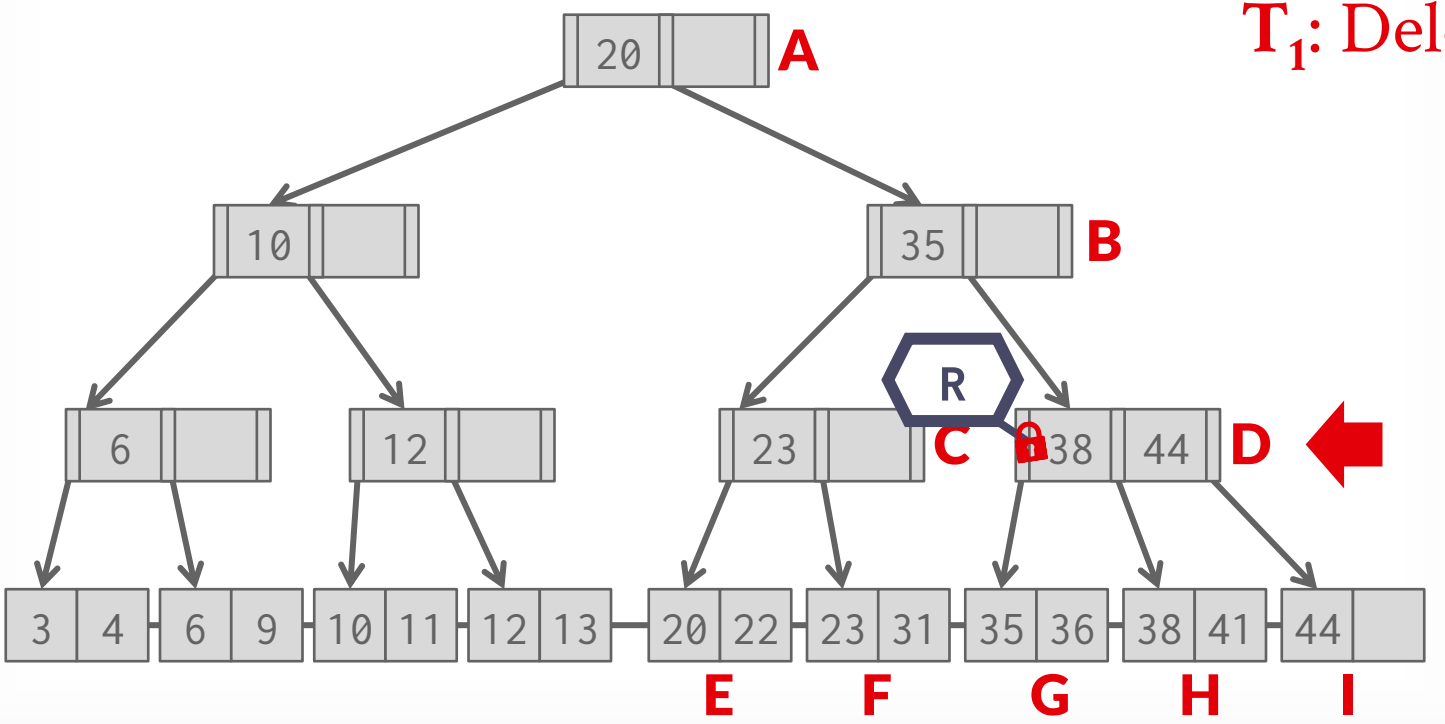
EXAMPLE #2: DELETE 38

T_1: Delete 38



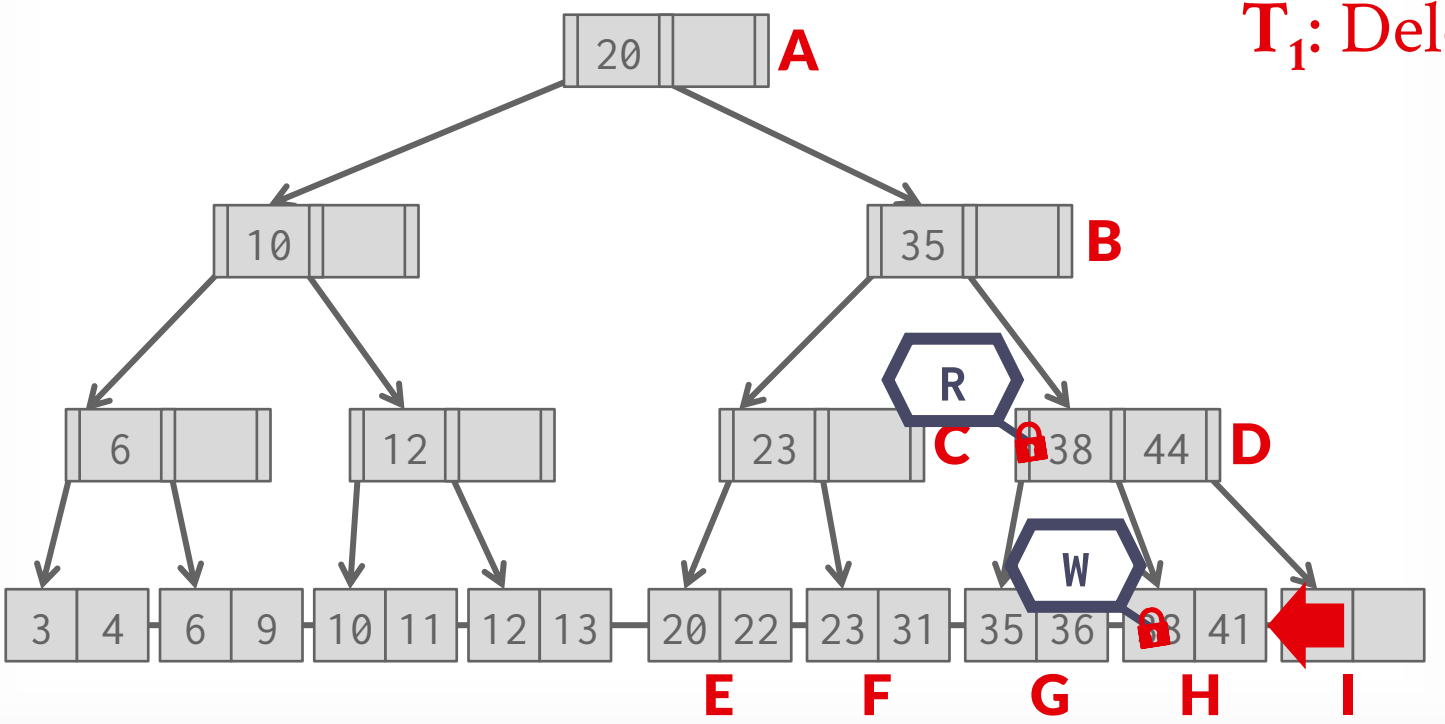
EXAMPLE #2: DELETE 38

T_1: Delete 38



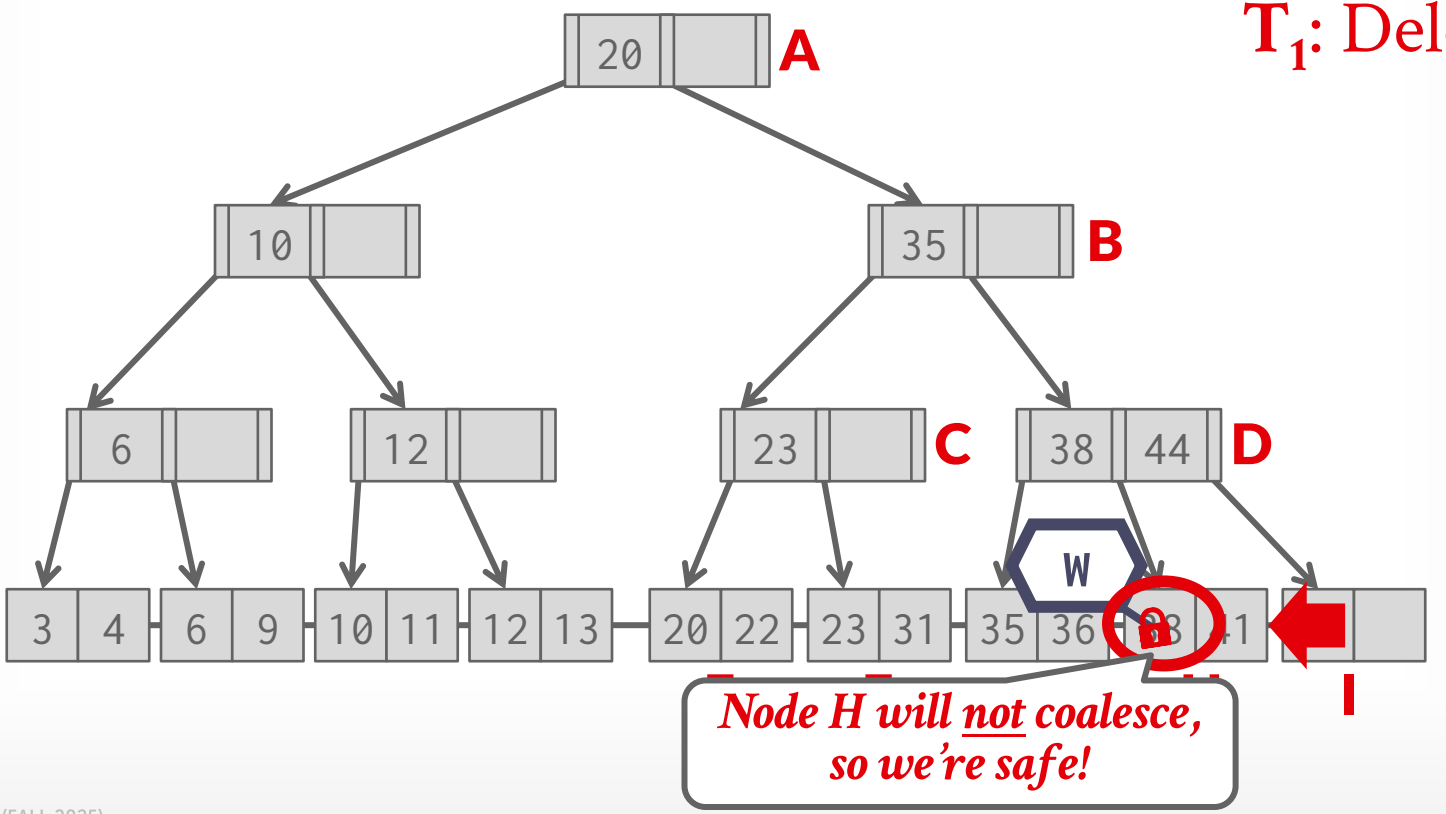
EXAMPLE #2: DELETE 38

T_1: Delete 38



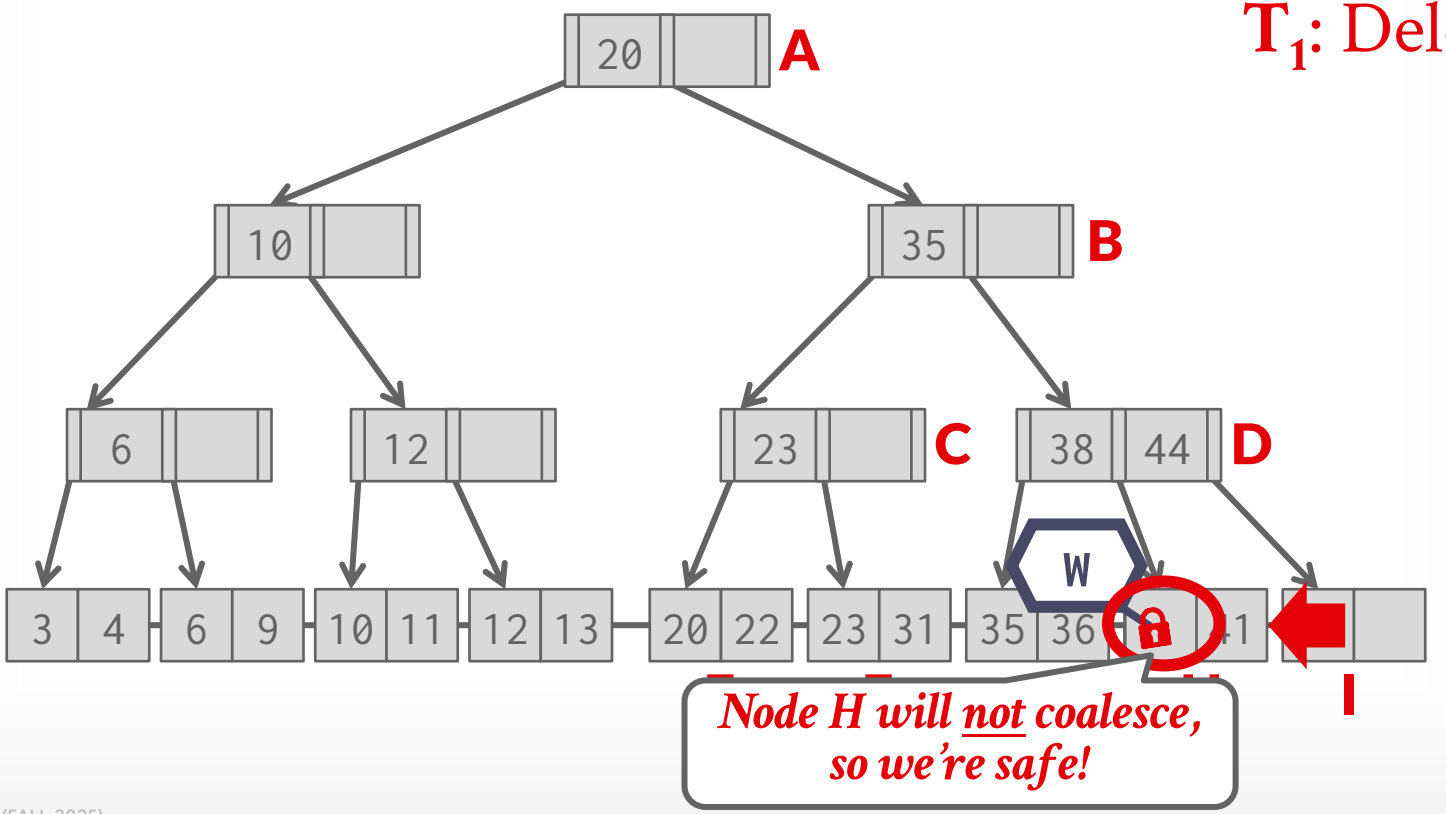
EXAMPLE #2: DELETE 38

T_1: Delete 38



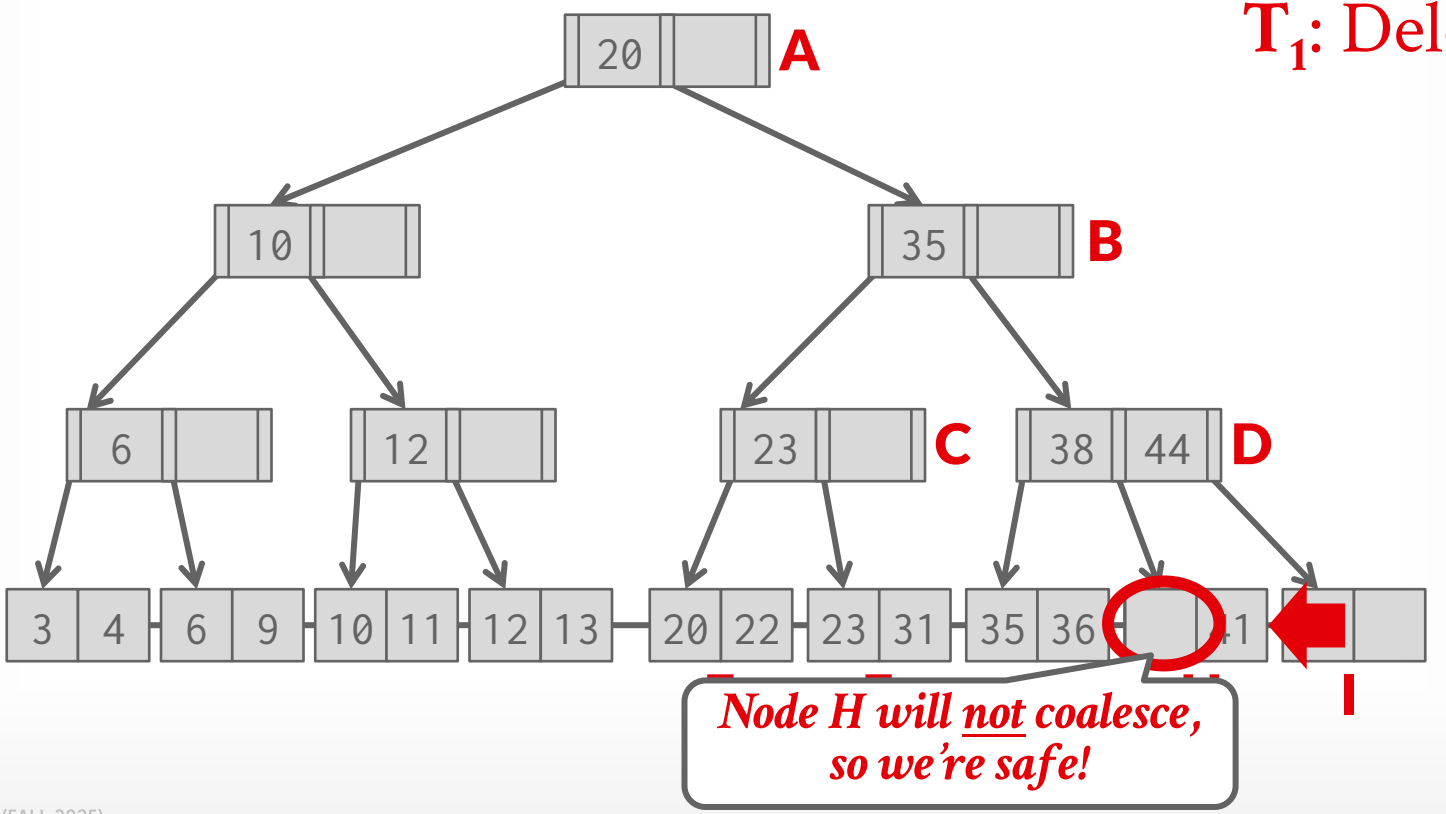
EXAMPLE #2: DELETE 38

T_1: Delete 38



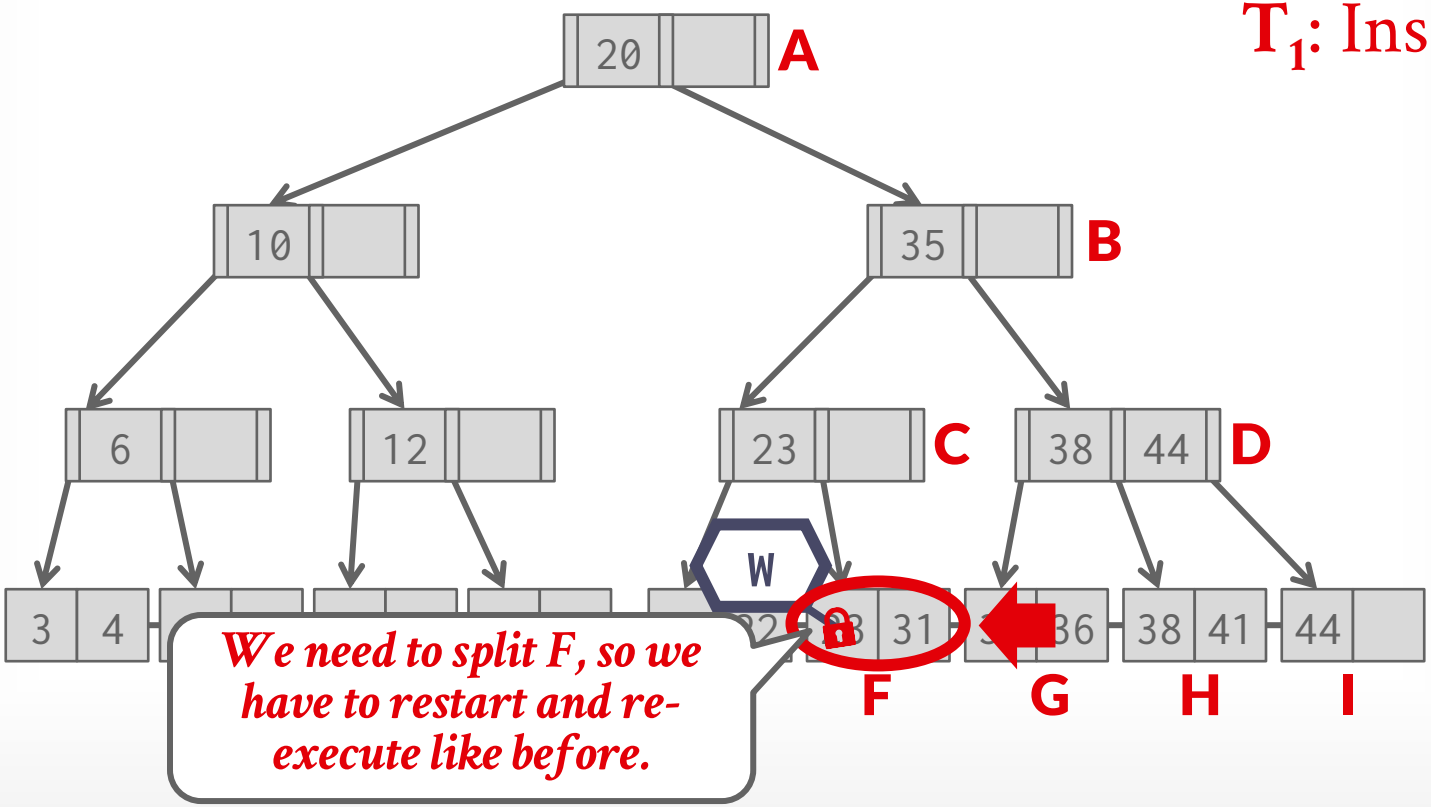
EXAMPLE #2: DELETE 38

T_1: Delete 38



EXAMPLE #4: INSERT 25

T_1: Insert 25



OBSERVATION

The threads in all the examples so far have acquired latches in a "top-down" manner.

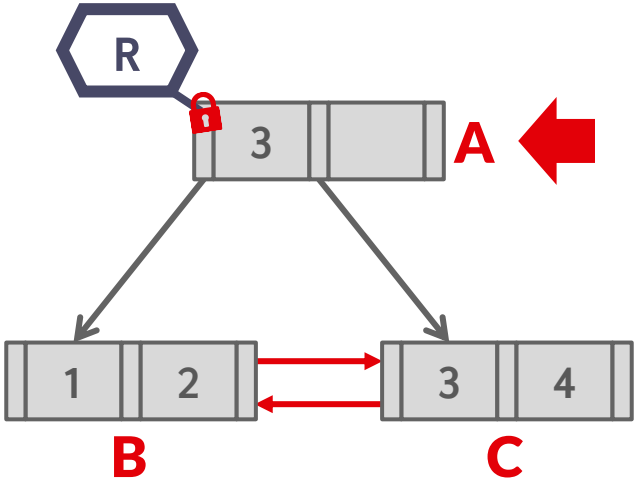
→ A thread can only acquire a latch from a node that is below its current node.

→ If the desired latch is unavailable, the thread must wait until it becomes available.

But what if threads want to move from one leaf node to another leaf node?

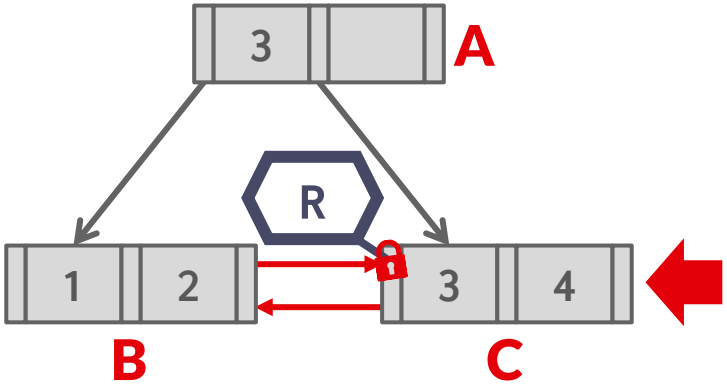
LEAF NODE SCAN EXAMPLE #1

T_1: Find Keys < 4



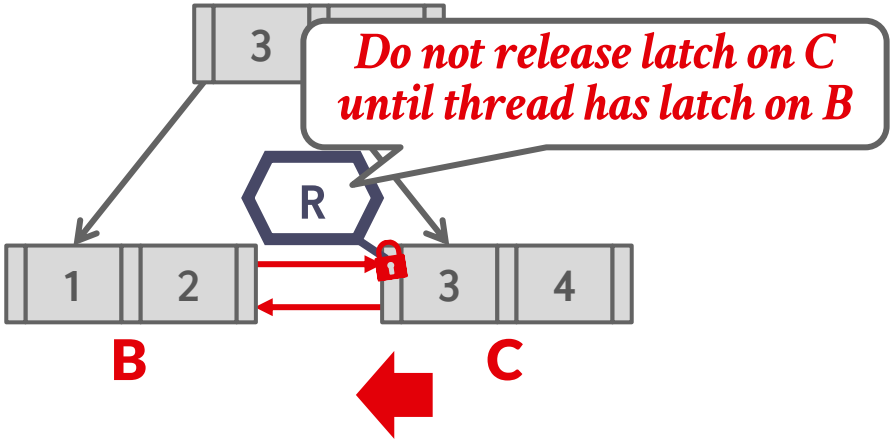
LEAF NODE SCAN EXAMPLE #1

T_1: Find Keys < 4



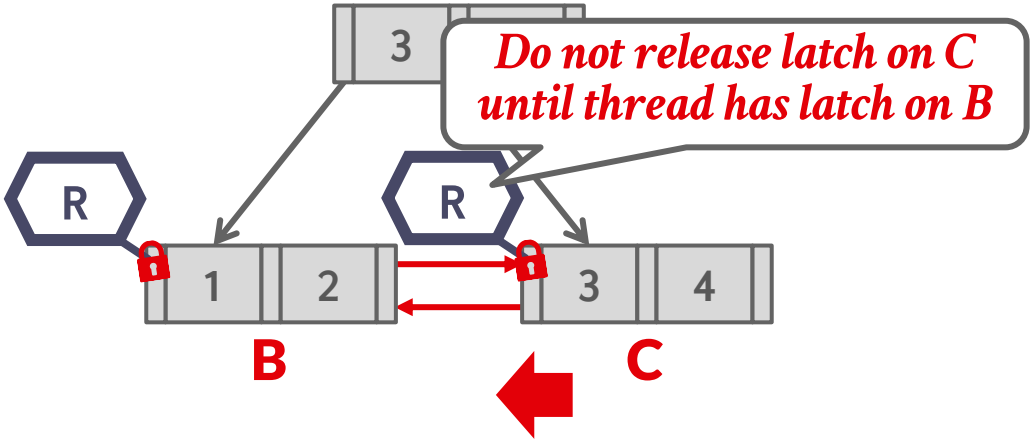
LEAF NODE SCAN EXAMPLE #1

T_1: Find Keys < 4



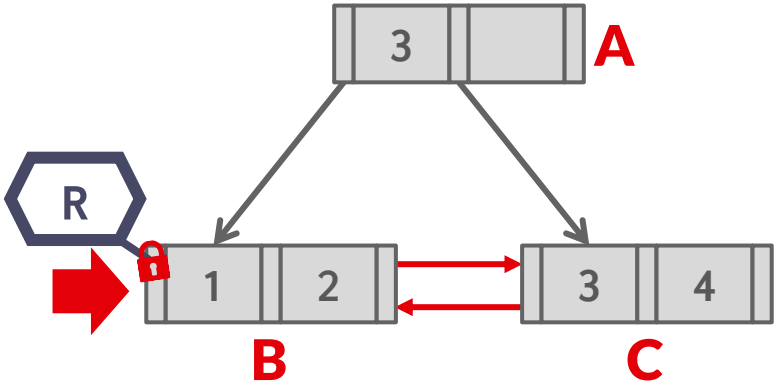
LEAF NODE SCAN EXAMPLE #1

T_1: Find Keys < 4

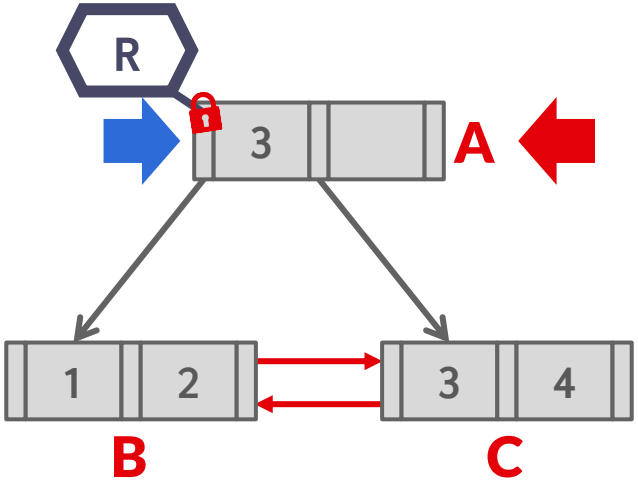


LEAF NODE SCAN EXAMPLE #1

T_1: Find Keys < 4



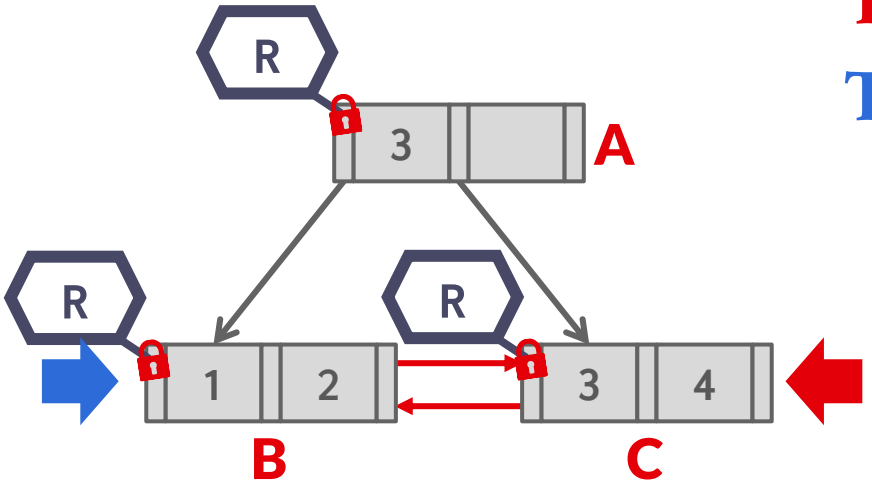
LEAF NODE SCAN EXAMPLE #2



T_1: Find Keys < 4

T_2: Find Keys > 1

LEAF NODE SCAN EXAMPLE #2



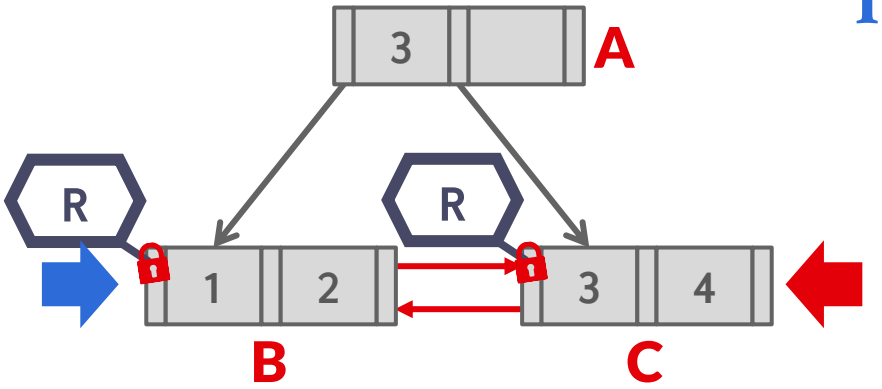
T_1: Find Keys < 4

T_2: Find Keys > 1

LEAF NODE SCAN EXAMPLE #2

T_1: Find Keys < 4

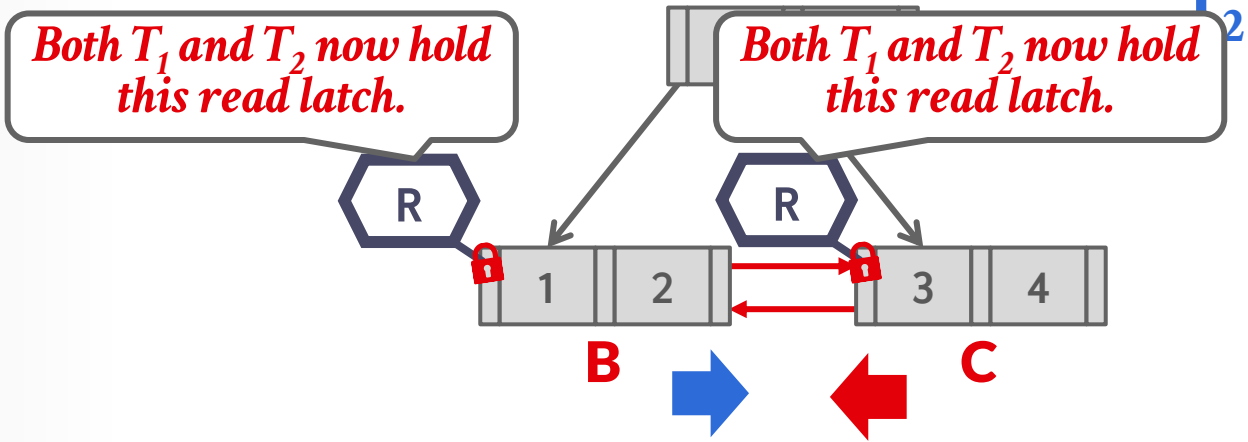
T_2: Find Keys > 1



LEAF NODE SCAN EXAMPLE #2

T_1: Find Keys < 4

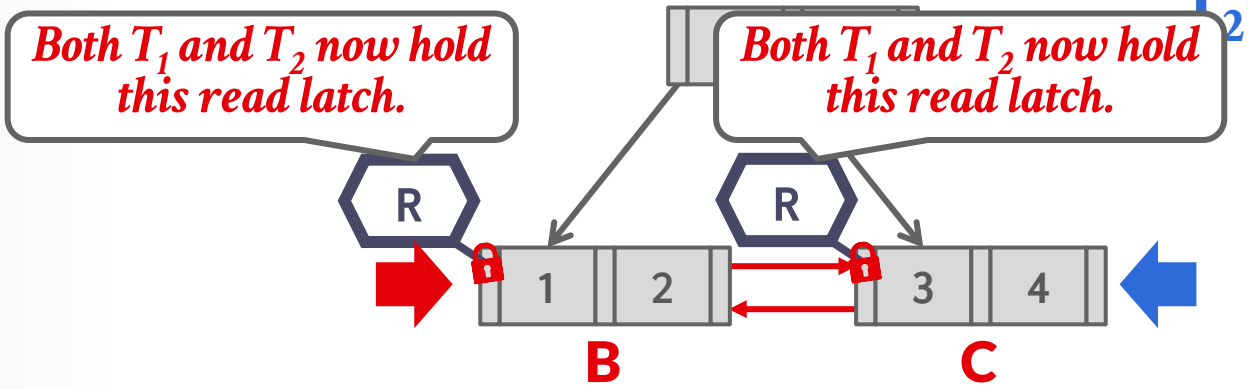
T_2: Find Keys > 1



LEAF NODE SCAN EXAMPLE #2

T_1: Find Keys < 4

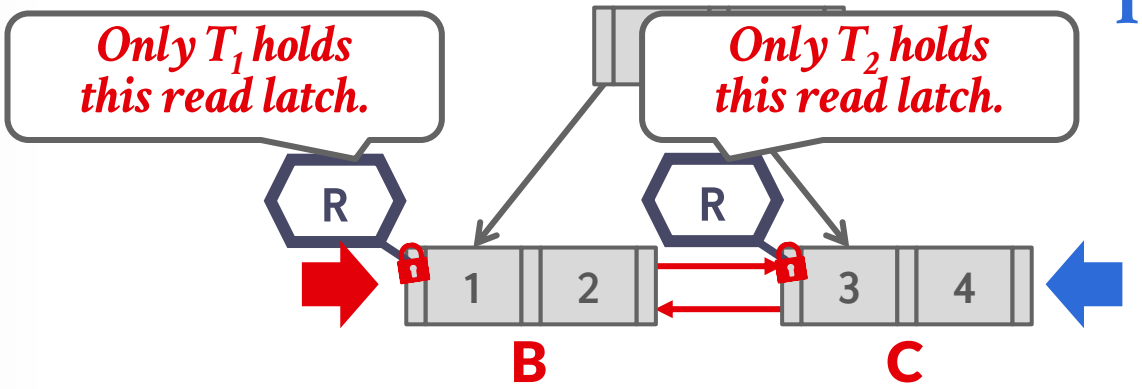
T_2: Find Keys > 1



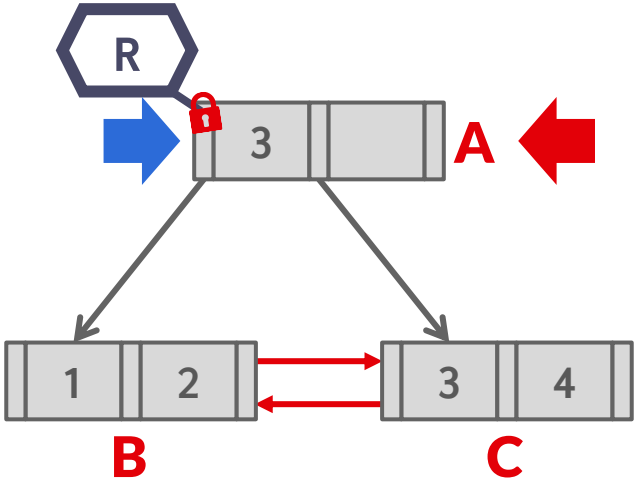
LEAF NODE SCAN EXAMPLE #2

T_1: Find Keys < 4

T_2: Find Keys > 1



LEAF NODE SCAN EXAMPLE #3



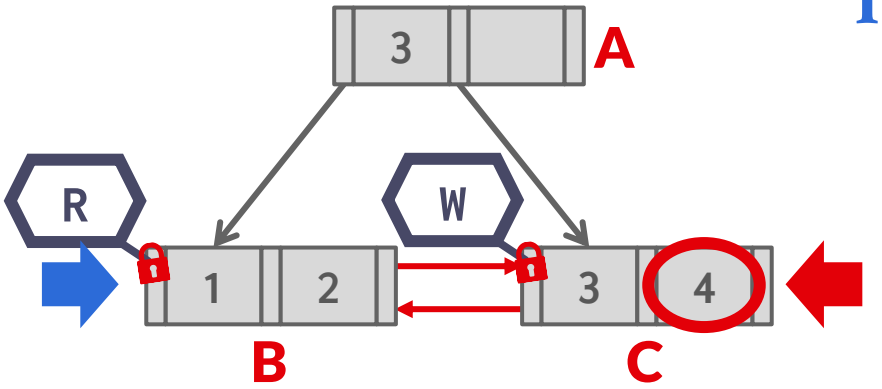
T_1: Delete 4

T_2: Find Keys > 1

LEAF NODE SCAN EXAMPLE #3

T_1: Delete 4

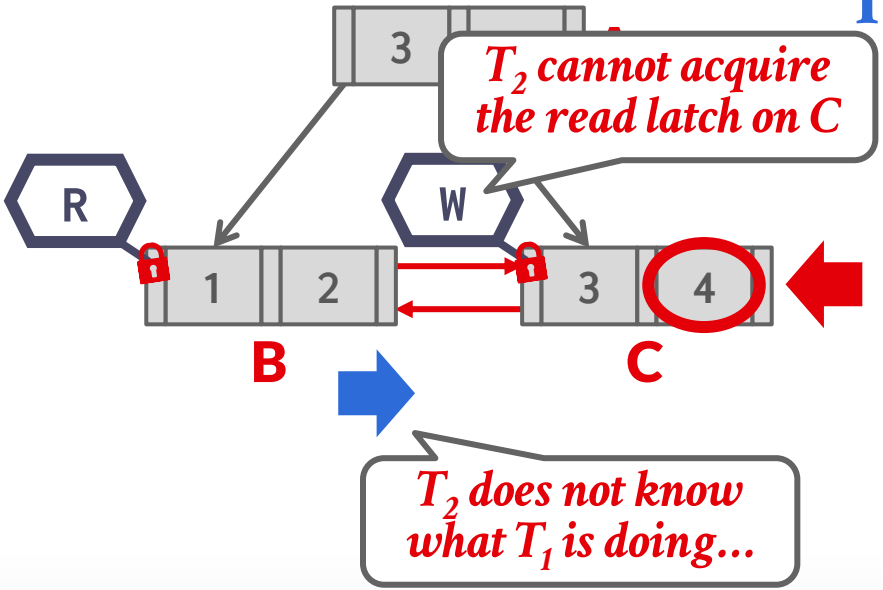
T_2: Find Keys > 1



LEAF NODE SCAN EXAMPLE #3

T_1: Delete 4

T_2: Find Keys > 1



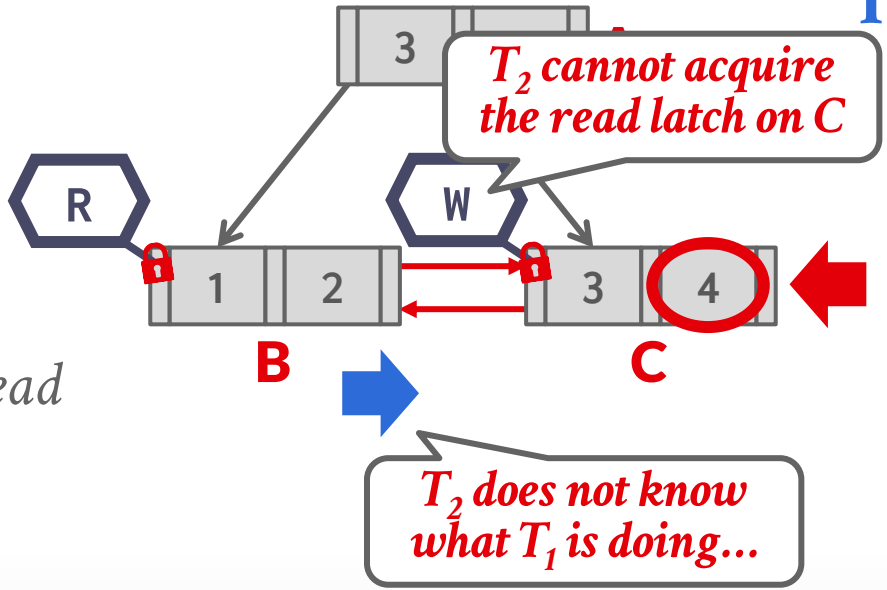
LEAF NODE SCAN EXAMPLE #3

T_1: Delete 4

T_2: Find Keys > 1

T_2 Choices?

- *Wait*
- *Kill Ourself*
- *Kill Other Thread*



LEAF NODE SCAN EXAMPLE #3

T_1: Delete 4

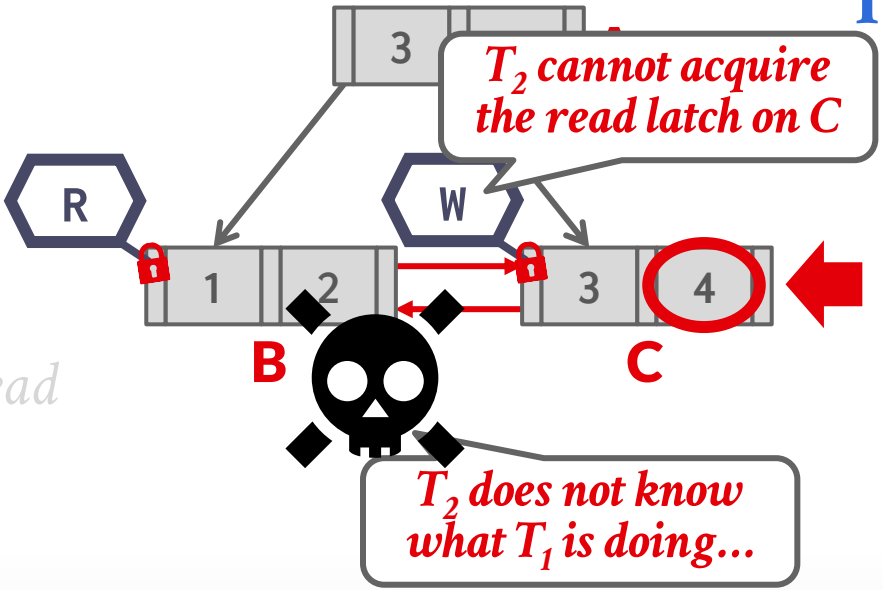
T_2: Find Keys > 1

T_2 Choices?

Wait

Kill Ourself

Kill Other Thread



LEAF NODE SCANS

Latches do not support deadlock detection or avoidance. The only way we can deal with this problem is through coding discipline.

The leaf node sibling latch acquisition protocol must support a “no-wait” mode.

The DBMS's data structures must cope with failed latch acquisitions.
→ Usually transparent to end-user / application.

B+TREE WRITE OPERATIONS

Each worker keeps of leaf node modifications in its private **write-set** during an operation.

→ List of node identifiers with keys inserted/deleted.

A worker holds write latches on modified nodes until it finishes that operation.

→ Ensures consistency by never exposing partial updates to other threads.

If a worker cannot acquire a write latch on a node:

→ Rollback modifications found in its write-set.

→ Abort the operation by releasing all latches.

CONCLUSION

Making a data structure thread-safe is notoriously difficult in practice.

We focused on B+Trees, but the same high-level techniques are applicable to other data structures.

NEXT CLASS

We are finally going to discuss how to execute some queries…